FPT UNIVERSITY

FPT UNIVERSITY
SoftwareDesignDocument

Frevia

GroupBar	
Group Members	Pham Kha Vy - CE182254 Nguyen Tien Thuan - CE181024 Tran Phat Tai - CE181632 Vo Nhut Tin - CE180619 Nguyen Lu Nhat Huy - CE181196
Supervisor	Mr. Vo Hoang Tu
Ext Supervisor	
Capstone Project code	FRE

- Can Tho, March 2025 -

Table of Contents

1 Product Overview **4**

2 User Requirement **6**

2.1 Actors 6

2.2 Use Cases 7

2.2.1 Diagram(s) 7

2.2.2 Use Case Description 13

2.2.3 Use Case Specification 18

2.2.1 Authentication 18

2.2.3.1 UC-01 Register 18

2.2.3.2 UC-02 Google Login 21

2.2.3.3 UC-05 Forgot Password 23

2.2.3.4 UC-07 Login 25

2.2.3.5 UC-08 Logout 27

2.2.2 Manage Role 28

2.2.3.1 UC-13.1 View Role List 28

2.2.3.2 UC-13.2 View Role Detail 29

2.2.3.3 UC-13.3 Create New Role 30

2.2.3.4 UC-13.4 Update Role 32

2.2.3.5 UC-13.5 Delete Role 34

2.2.3.6 UC-13.6 Update Role Permissions 35

2.2.3 Manage Freelancer Profile 38

2.2.3.1 UC-10.1 View Detailed Freelancer Profile 38

2.2.3.2 UC-10.2 Update Your Detailed Freelancer Profile . 39

2.2.4 Manage Freelancer Skills 40

2.2.3.1 UC-11.1 View List Freelancer Skill 40

2.2.3.2 UC-11.2 Add Freelancer Skill 41

2.2.3.3 UC-11.3 Delete Freelancer Skill 42

2.2.5 Manage Freelancer Portfolio 43

2.2.3.1 UC-12.1 View Portfolio List 43

2.2.3.2 UC-12.2 Add New Portfolio 44

2.2.3.3 UC-12.3 View Portfolio Detail 46

2.2.3.4 UC-12.4 Update Portfolio 47

2.2.3.5 UC-12.5 Delete Portfolio 48

2.2.6 Browse Jobs 49

2.2.3.1 UC-03.1 View List Job 49

2.2.3.2 UC-03.2 View Job Detail 51

2.2.3.3 UC-06.2 Post New Job 53

2.2.3.4 UC-06.3 Edit Job 55
2.2.3.5 UC-06.4 Delete Job 57
2.2.3.6 UC-06.5 Change Job Status 59
2.2.3.7 UC-06.6 View List Bookmarked Job 61
2.2.3.8 UC-06.7 Bookmark Job 63
2.2.3.9 UC-06.8 Remove Bookmark 65
2.2.7 Manage Contract 66
2.2.3.1 UC-09.1 Get Contract List 66
2.2.3.2 UC-09.2 Get Contract Detail 67
2.2.3.3 UC-09.3 Sign Contract 68
2.2.3.4 UC-09.4 Complete Contract 70
2.2.3.5 UC-09.5 Cancel Contract 71
2.2.3.6 UC-09.6 Contract Activities 72
2.2.3.7 UC-09.7 Contract Files 73
2.2.3.8 UC-09.8 Upload Contract File 75
2.2.3.9 UC-09.9 Delete Contract File 76
2.2.3.10 UC-09.10 Update Contract Terms 77
2.2.3.11 UC-09.11 Create Contract 79
2.2.3.12 UC-09.12 Progress Update 81
2.2.8 Manage Forum 82
2.2.3.1 UC-04.1 View Forum Categories 82
2.2.3.2 UC-04.2 View Forum Category Detail 84
2.2.3.3 UC-04.3 View Post List 86
2.2.3.4 UC-04.4 View Post Detail 88
2.2.3.5 UC-04.5 Create Post 90
2.2.3.6 UC-04.6 Edit Post 93
2.2.3.7 UC-04.7 Delete Post 95
2.2.3.8 UC-04.8 Like Post 97
2.2.3.9 UC-04.9 Unlike Post 99
2.2.3.10 UC-04.10 View Like Post Detail 101
2.2.3.11 UC-04.11 View Post Comments 103
2.2.3.12 UC-04.12 Add Comment 105
2.2.3.13 UC-04.13 Edit Comment 107
2.2.3.14 UC-04.14 Delete Comment 109
2.2.3.15 UC-04.15 Like Comment 111
2.2.3.16 UC-04.16 Unlike Comment 113
2.2.3.17 UC-04.17 Report Content 115

3 Business Rule 118

4 Functional Requirement 124
4.1 System Functional Overview 124
4.1.1 Screen Flow 124
4.1.2 Screen Details 127
4.1.3 Screen Authorization 131
4.1.4 Non-Screen Functions 133
4.1.5 Entity Relationship Diagram 141
4.1.1 Authentication & Role-Based Access 141
4.1.2 Manage Profile 142
4.1.3 Portfolio & Freelancer Network 143
4.1.4 Job Posting & Discovery 144
4.1.5 Manage Proposal 145
4.1.6 Manage Contract 146
4.1.7 Escrow, Payments & Dispute Resolution 147
4.1.8 Reviews & Reputation System 148
4.1.9 Forum & Community 149
4.1.10 Messaging & Notifications 149
4.1.11 Administration & Moderation 150
5 Non-Functional Requirement 150
5.1 Performance & Scalability 150
5.2 Security & Privacy 151
5.3 Availability & Reliability 151
5.4 Usability & User Experience 151

1 Product Overview

The Frevia project is a comprehensive freelancing platform that enables clients, freelancers, and administrators to build trusted professional relationships and efficiently manage the entire project lifecycle. Designed to connect businesses with skilled talent worldwide, Frevia provides intelligent job matching, structured contract workflows, and secure payment solutions to facilitate successful collaborations.

Through AI-powered recommendations, real-time communication, and advanced moderation mechanisms, the platform helps users discover suitable opportunities, deliver high-quality work, and maintain transparent interactions.

Frevia serves three primary stakeholder groups: clients seeking specialized expertise, freelancers showcasing their skills and services, and administrators responsible for maintaining platform integrity and operational efficiency. The platform offers an intuitive user experience, dynamic communication channels, and comprehensive management capabilities to support collaboration, professional growth, and secure transactions.

Key features of Frevia include:

- **Smart Job Matching and AI Recommendations:** The platform analyzes freelancer skills, experience, and preferences to recommend relevant job opportunities. AI-powered profile scoring provides quality assessments, while job alerts and saved searches help users stay informed about new opportunities.
- **End-to-End Contract and Milestone Management:** Clients and freelancers can negotiate project terms, define milestones and tasks, monitor progress, and securely manage payments. Integrated contract signing, activity logs, and file-sharing capabilities ensure transparency throughout the project lifecycle.
- **Community and Forum Engagement:** Frevia includes a community forum where users can discuss industry topics, exchange knowledge, and seek professional advice. Features such as categories, posts, comments, reactions, and reporting mechanisms encourage active participation and self-moderation.
- **Identity Verification and Content Moderation:** Freelancers can verify their identities through document submission, while administrators utilize moderation tools, banned keyword management, and AI-assisted violation detection to maintain a secure and trustworthy environment.
- **Wallet, Payments, and Gas Fee Subsidies:** Users can manage account balances, withdraw earnings, and review transaction histories. The platform also supports gas fee sub-

sidy mechanisms for blockchain-based transactions, improving payment accessibility and reducing operational costs.

- **Administrative Management Tools:** Administrators have access to comprehensive management functionalities, including user administration, role and permission control, system configuration, advertisement management, activity monitoring, and dispute resolution, ensuring fair and reliable platform operations.

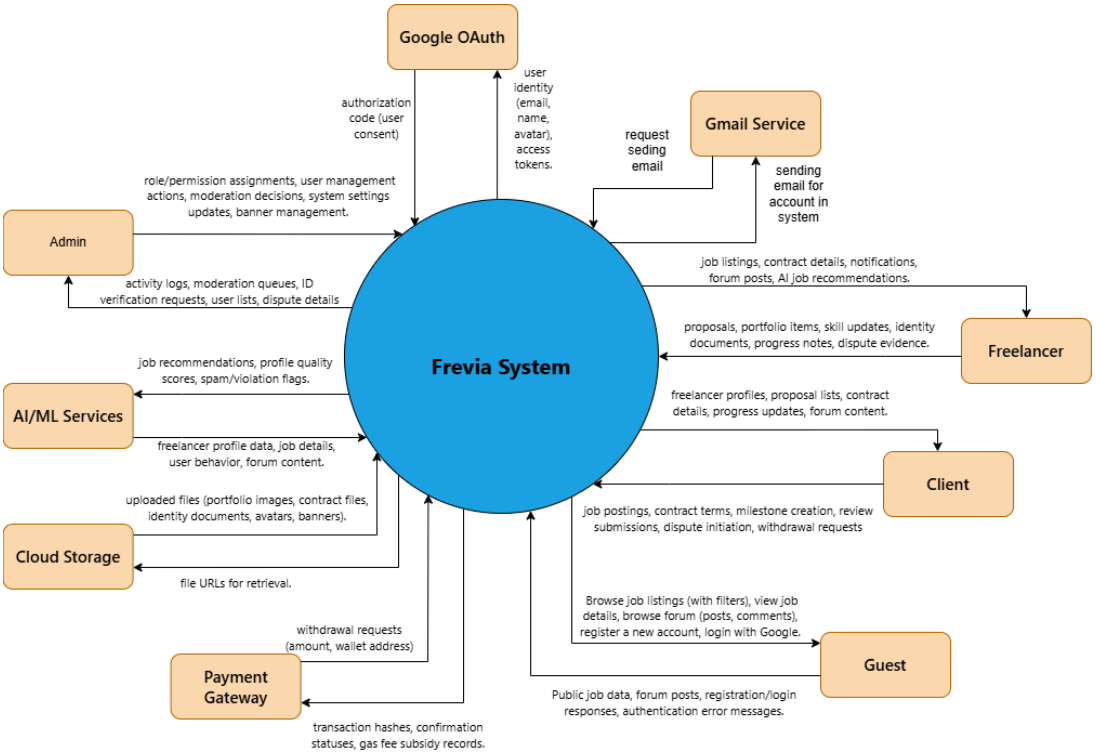


Figure 1: Context diagram

2 User Requirement

2.1 Actors

Table 1: Actors of the Frevia Platform

No	Actor	Description
1	Guest	Guests are unregistered visitors who have not yet created an account on the platform. They can browse publicly available information such as job listings, freelancer profiles, forum content, and platform features. Guests may also register a new account or authenticate using supported login methods to access platform services.
2	Client	Clients are individuals or organizations seeking professional services from freelancers. They can create and manage job postings, review proposals, hire freelancers, establish contracts, manage escrow payments, monitor project progress, initiate disputes, and provide reviews upon project completion.
3	Freelancer	Freelancers are service providers who offer professional skills and expertise through the platform. They can create and manage profiles, portfolios, and skills; search and apply for jobs; submit proposals; collaborate with clients; complete project tasks and milestones; and receive payments after successful project delivery.
4	Admin	Administrators are responsible for managing and maintaining the platform's operations, security, and compliance. Their responsibilities include user management, role and permission management, identity verification, content moderation, dispute resolution, system configuration, and monitoring platform activities to ensure a secure and reliable environment.

2.2 Use Cases

2.2.1 Diagram(s)

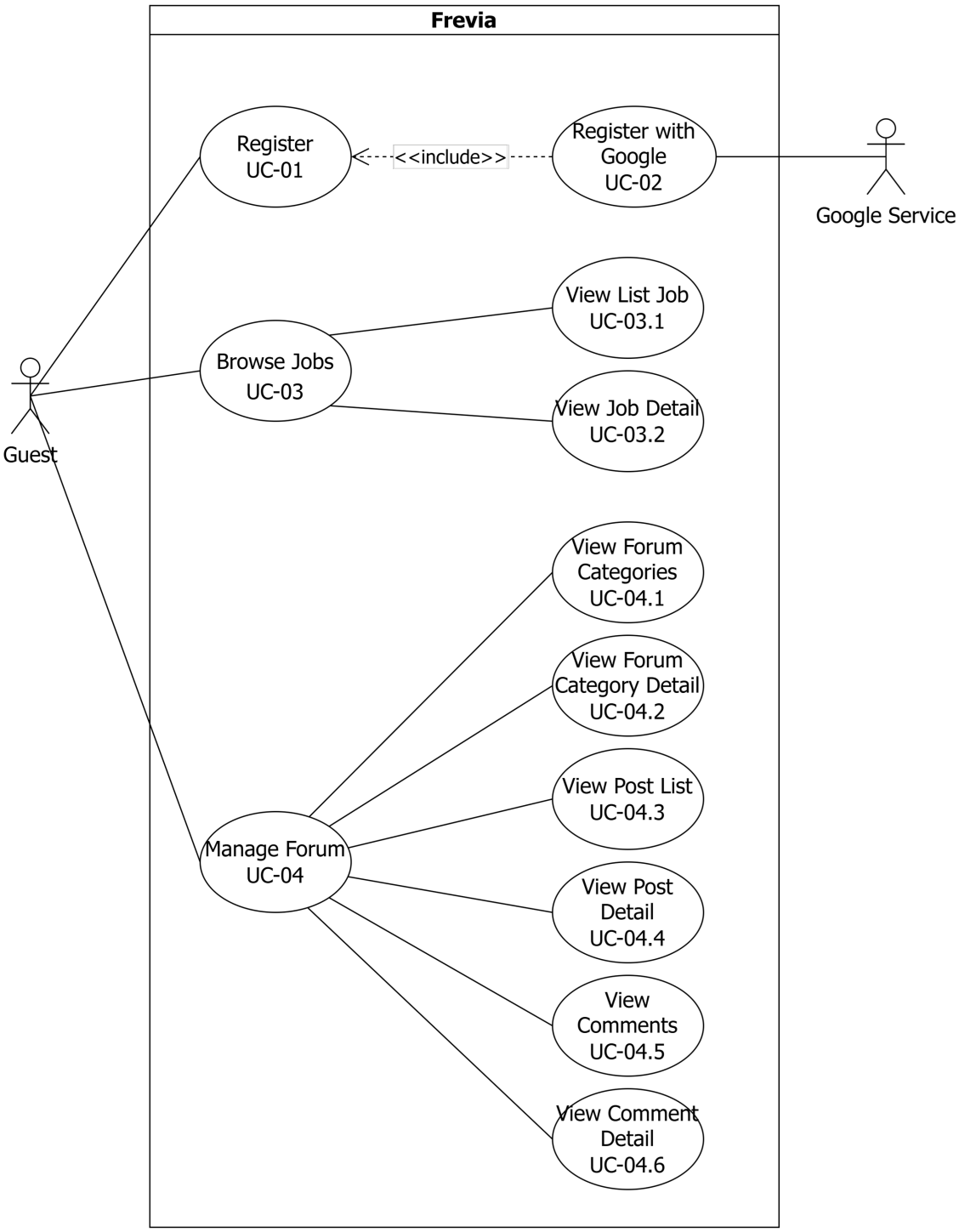


Figure 2: Guest Usecase

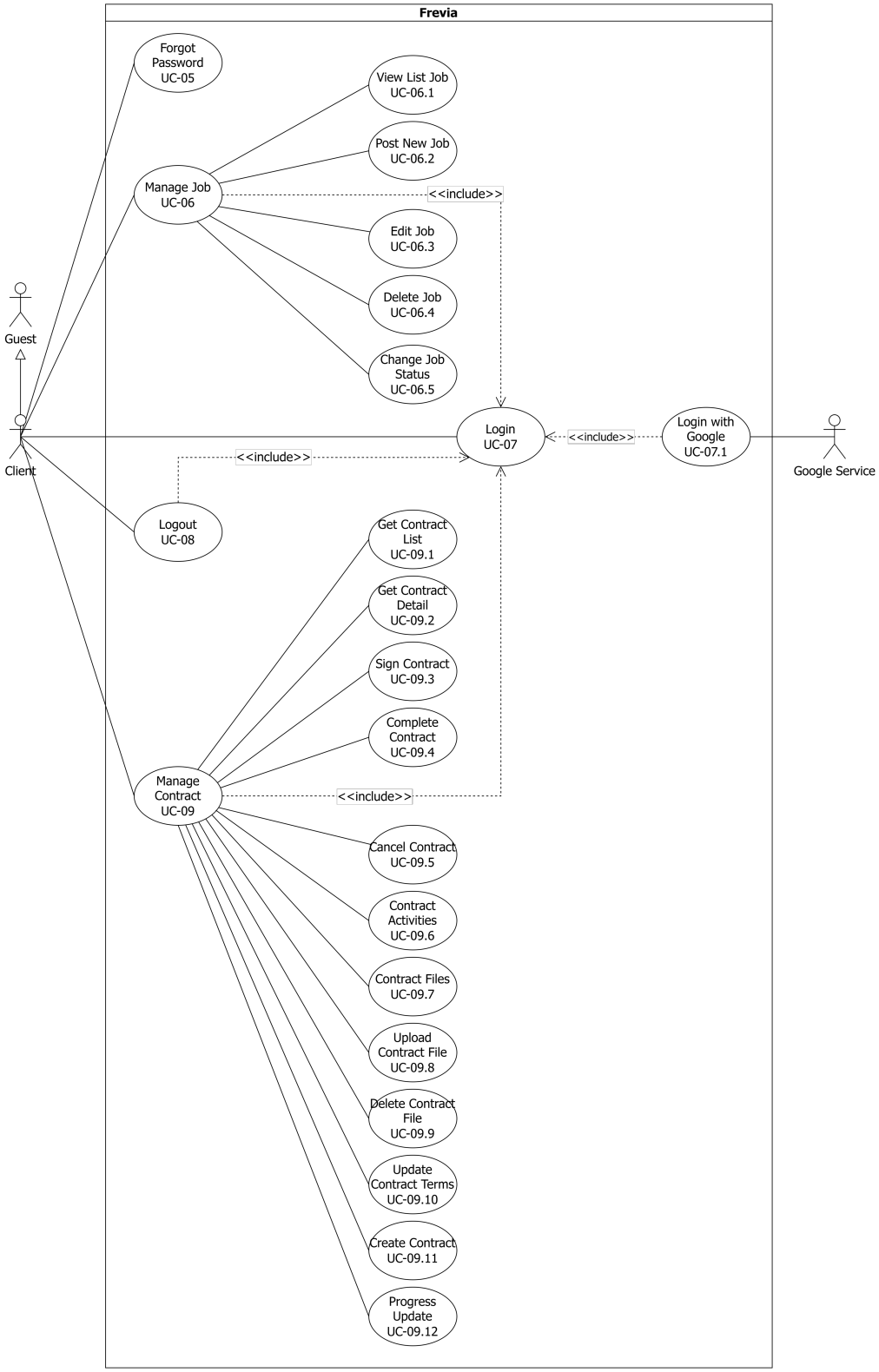


Figure 3: Client Usecase

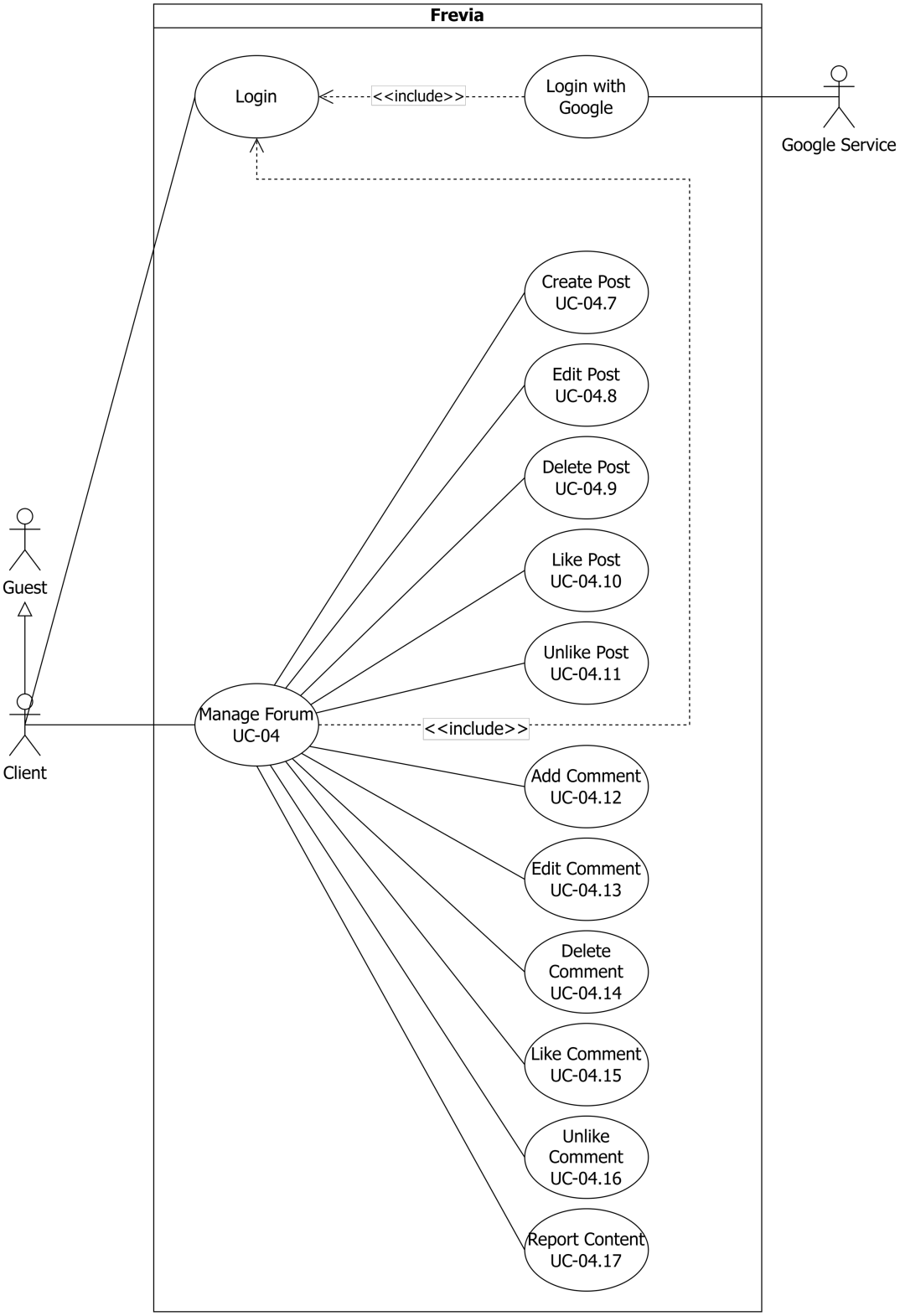


Figure 4: Client Usecase

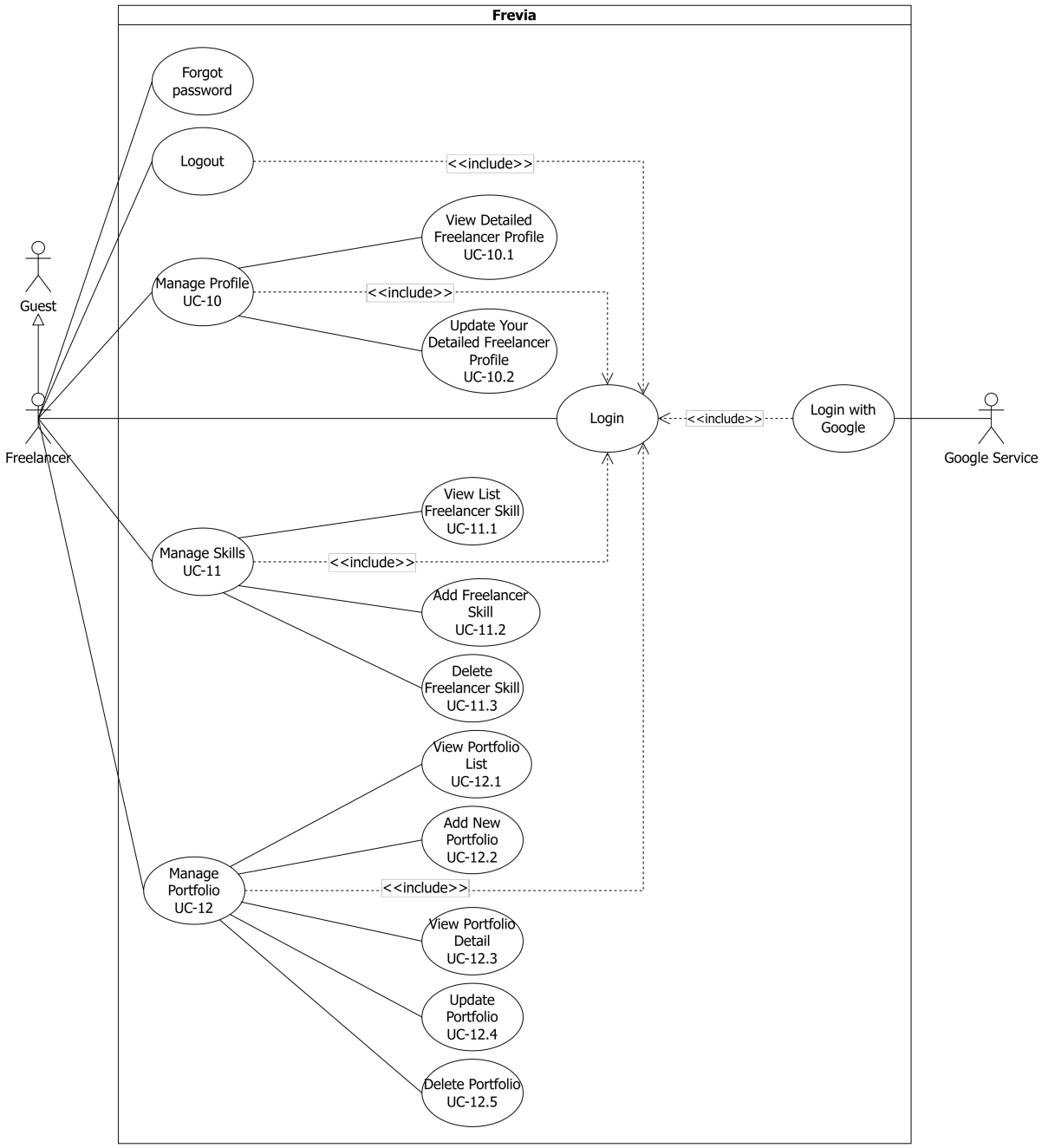


Figure 5: Freelancer Usecase

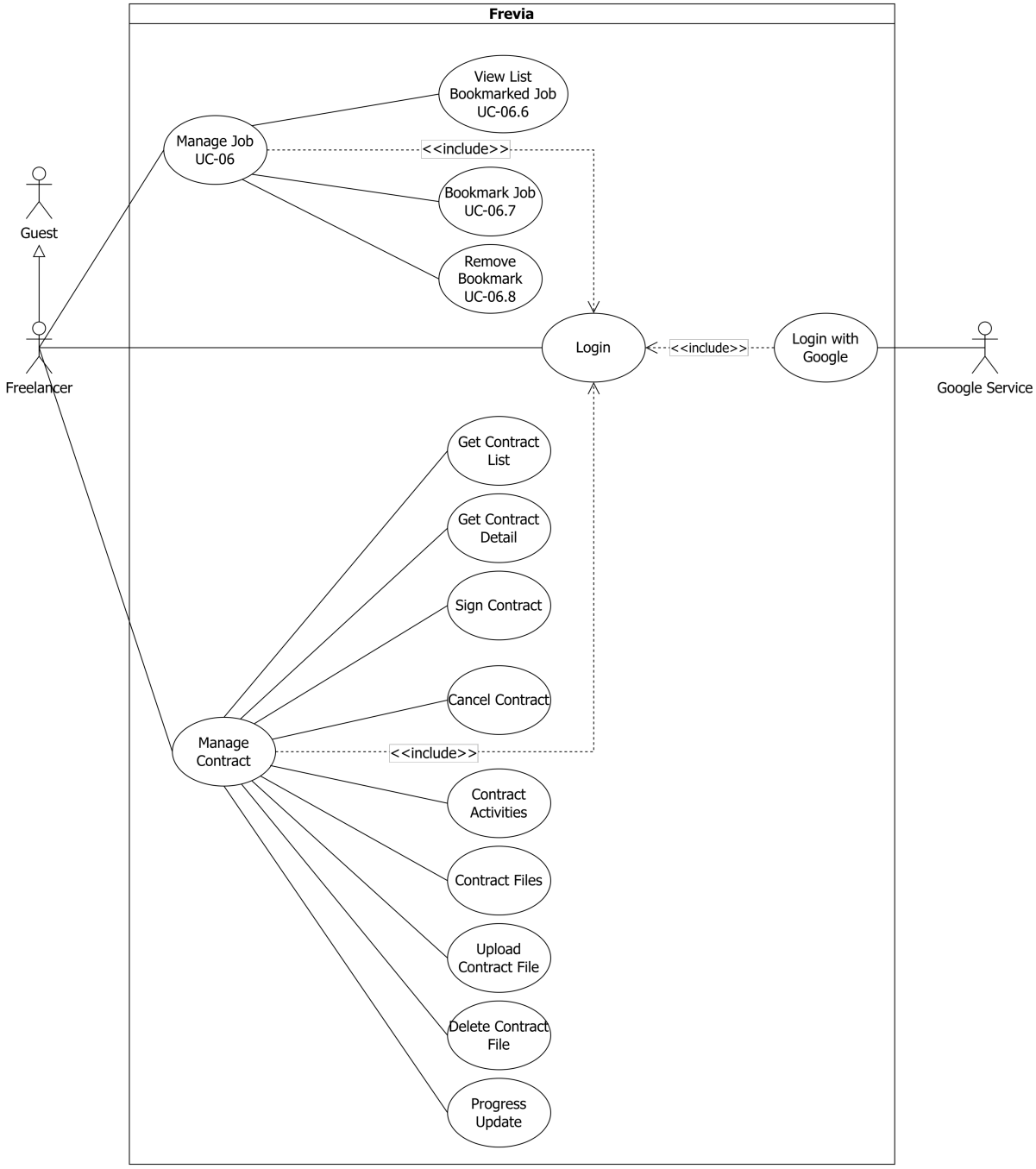


Figure 6: Freelancer Usecase

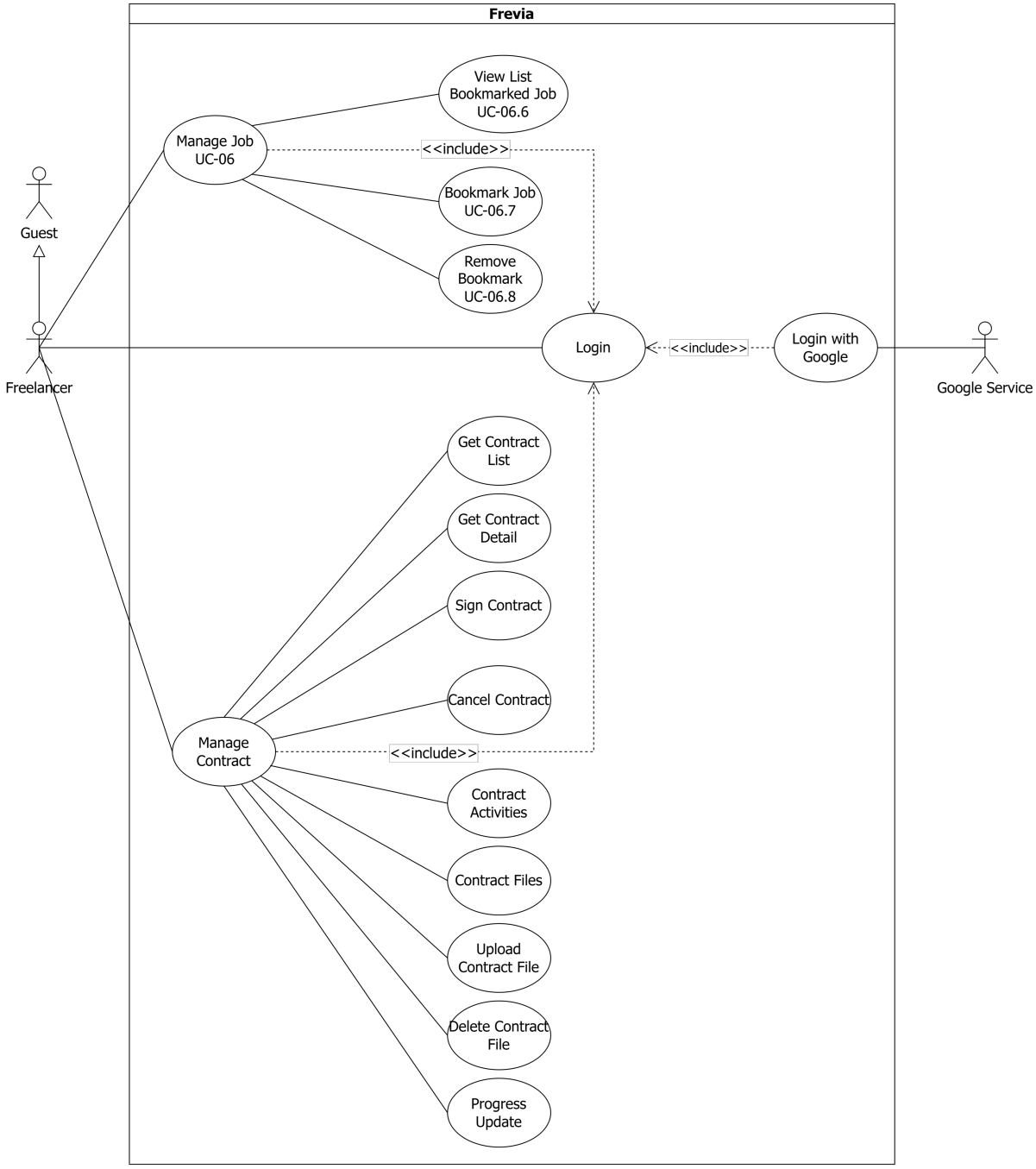


Figure 7: Freelancer Usecase

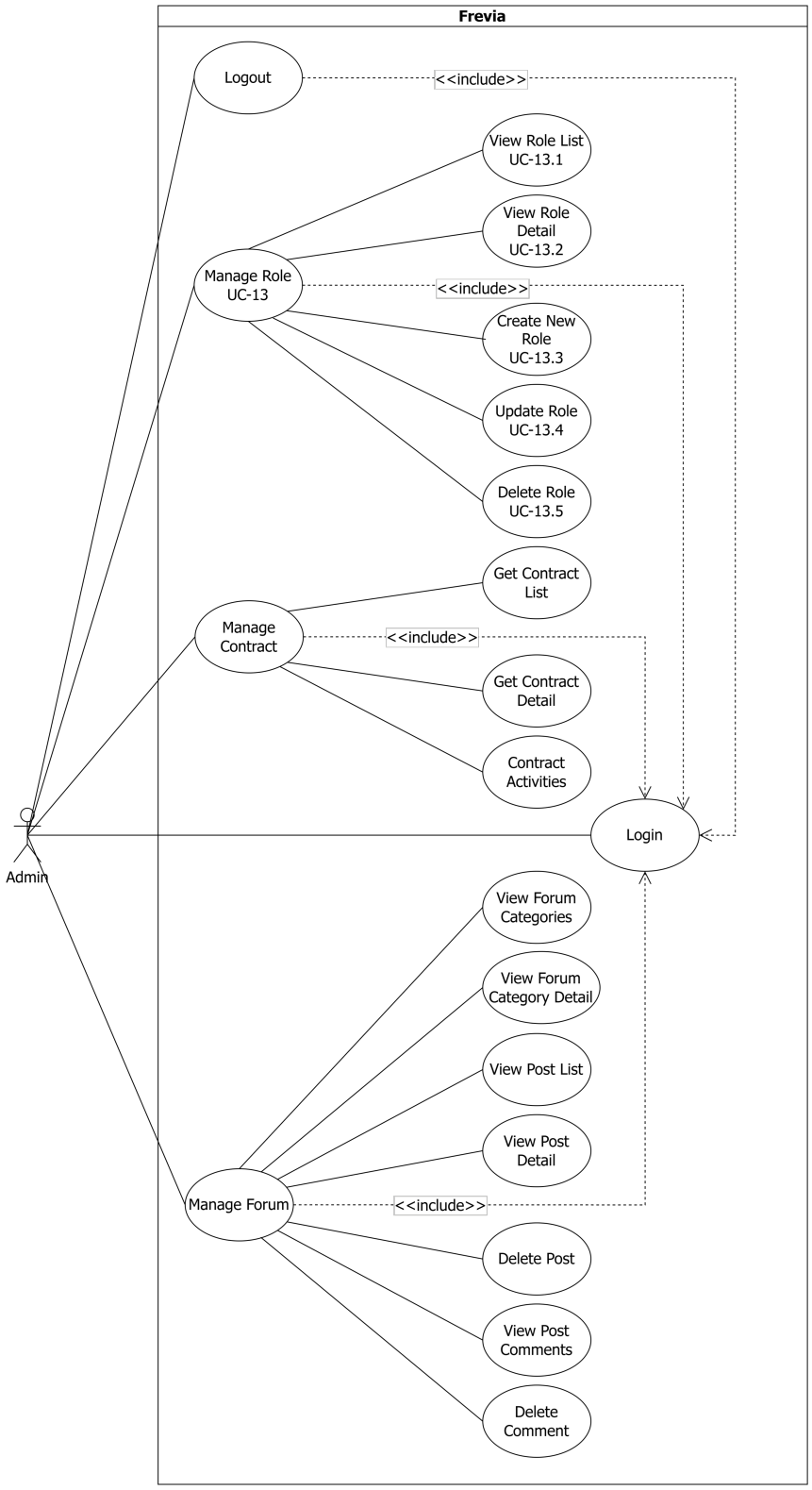


Figure 8: Admin Usecase

2.2.2 Use Case Description

ID	Use Case	Actors	Description
UC-01	Register	Guest, Client, Freelancer	Allows users to create new accounts using email and password.
UC-02	Google Login	Guest, Client, Freelancer	Allows users to login/register using Google OAuth.
UC-03	Job Browsing	Guest, Client, Freelancer	Enables guests to browse and search for available job listings.
UC-03.1	View List Job	Guest, Client, Freelancer	Retrieve paginated job list with filters (category, budget, skills, status, keywords) and sorting options.
UC-03.2	View Job Detail	Guest, Client, Freelancer	Get full job details: title, description, budget, skills, client info, proposals count, attachments, timeline.
UC-04	Manage Forum	Guest, Client, Freelancer, Admin	Manages forum operations including viewing, creating, editing, and moderating posts and comments.
UC-04.1	View Forum Categories	Guest, Client, Freelancer, Admin	Displays all available forum categories for browsing.
UC-04.2	View Forum Category Detail	Guest, Client, Freelancer, Admin	Shows detailed information of a specific forum category and its sub-forums.
UC-04.3	View Post List	Guest, Client, Freelancer, Admin	Retrieves paginated list of posts within a forum category with filters and sorting.
UC-04.4	View Post Detail	Guest, Client, Freelancer, Admin	Displays full post content, author info, creation date, and associated tags.
UC-04.5	View Comments	Guest, Client, Freelancer, Admin	Retrieves all comments related to a specific forum post.
UC-04.6	View Comment Detail	Guest, Client, Freelancer, Admin	Shows detailed information of a specific comment, including content, author, and timestamps.
UC-04.7	Create Post	Client, Freelancer	Allows freelancers to create new discussion posts or questions in the forum.

Continued on next page

ID	Use Case	Actors	Description
UC-04.8	Edit Post	Client, Freelancer	Allows freelancers to edit their own forum posts.
UC-04.9	Delete Post	Client, Freelancer, Admin	Allows post owners or admins to delete a forum post.
UC-04.10	Like Post	Client, Freelancer	Allows freelancers to like a forum post.
UC-04.11	Unlike Post	Client, Freelancer	Allows freelancers to remove their like from a forum post.
UC-04.12	Add Comment	Client, Freelancer	Allows freelancers to add comments to a forum post.
UC-04.13	Edit Comment	Client, Freelancer	Allows freelancers to edit their own comments.
UC-04.14	Delete Comment	Client, Freelancer	Allows comment owners or admins to delete a comment.
UC-04.15	Like Comment	Client, Freelancer	Allows freelancers to like a comment.
UC-04.16	Unlike Comment	Client, Freelancer	Allows freelancers to remove their like from a comment.
UC-04.17	Report Content	Client, Freelancer	Allows freelancers to report inappropriate content in forum posts or comments.
UC-05	Forgot Password	Client, Freelancer	Allows users to reset their password via email verification.
UC-06	Manage Job	Client, Freelancer	Manages job postings including creation, editing, deletion, and status changes.
UC-06.1	View List Job	Guest, Client, Freelancer	Retrieve paginated job list with filters and sorting options.
UC-06.2	Post New Job	Client	Allows clients to create and publish new job postings.
UC-06.3	Edit Job	Client	Allows clients to edit their own job postings.
UC-06.4	Delete Job	Client	Allows clients to delete their own job postings.
UC-06.5	Change Job Status	Client	Allows clients to update job status (Open, In Progress, Closed, Cancelled).
UC-06.6	View List Bookmarked Job	Freelancer	Retrieves list of jobs bookmarked by the freelancer.

Continued on next page

ID	Use Case	Actors	Description
UC-06.7	Bookmark Job	Freelancer	Allows freelancers to save/bookmark a job for later reference.
UC-06.8	Remove Bookmark	Freelancer	Allows freelancers to remove a bookmarked job from their list.
UC-07	Login	Client, Freelancer, Admin	Allows users to authenticate using email and password.
UC-08	Logout	Client, Freelancer, Admin	Allows users to securely log out of the system.
UC-09	Manage Contract	Client, Freelancer, Admin	Manages contracts between clients and freelancers.
UC-09.1	Get Contract List	Client, Freelancer, Admin	Retrieves paginated list of contracts with status filters and sorting.
UC-09.2	Get Contract Detail	Client, Freelancer, Admin	Displays full contract details including terms, parties, status, and milestones.
UC-09.3	Sign Contract	Client, Freelancer	Allows freelancers to digitally sign and accept a contract.
UC-09.4	Complete Contract	Client, Freelancer	Marks a contract as completed successfully.
UC-09.5	Cancel Contract	Client, Freelancer	Allows either party to cancel a contract before completion.
UC-09.6	Contract Activities	Client, Freelancer, Admin	Tracks all activities and updates related to a contract.
UC-09.7	Contract Files	Client, Freelancer	Manages files and attachments associated with a contract.
UC-09.8	Upload Contract File	Client, Freelancer	Allows users to upload files to a contract (e.g., deliverables, documents).
UC-09.9	Delete Contract File	Client, Freelancer	Allows users to delete files from a contract.
UC-09.10	Update Contract Terms	Client	Allows clients to update contract terms and conditions.

Continued on next page

ID	Use Case	Actors	Description
UC-09.11	Create Contract	Client	Allows clients to create and initiate a new contract with a freelancer.
UC-09.12	Progress Update	Client, Freelancer	Allows freelancers to submit progress updates on contract milestones.
UC-10	Manage Profile	Freelancer	Manages freelancer profile information and settings.
UC-10.1	View Detailed Freelancer Profile	Freelancer	Displays full freelancer profile including skills, portfolio, experience, and ratings.
UC-10.2	Update Your Detailed Freelancer Profile	Freelancer	Allows freelancers to edit and update their profile information.
UC-11	Manage Skills	Freelancer	Manages freelancer skill set and proficiency levels.
UC-11.1	View List Freelancer Skill	Freelancer	Retrieve the list of skills added by the freelancer.
UC-11.2	Add Freelancer Skill	Freelancer	Allows freelancers to add new skills to their profile.
UC-11.3	Delete Freelancer Skill	Freelancer	Allows freelancers to remove skills from their profile.
UC-12	Manage Portfolio	Freelancer	Manages freelancer portfolio projects and work samples.
UC-12.1	View Portfolio List	Freelancer	Retrieve the list of portfolios created by the freelancer.
UC-12.2	Add New Portfolio	Freelancer	Allows freelancers to add new portfolio items with images and descriptions.
UC-12.3	View Portfolio Detail	Freelancer	Displays full details of a portfolio item including media, description, and technologies used.
UC-12.4	Update Portfolio	Freelancer	Allows freelancers to edit and update their portfolio items.
UC-12.5	Delete Portfolio	Freelancer	Allows freelancers to remove portfolio items.
UC-13	Manage Role	Admin	Manages user roles and permissions within the system.

Continued on next page

ID	Use Case	Actors	Description
UC-13.1	View Role List	Admin	Retrieves list of all roles in the system.
UC-13.2	View Role Detail	Admin	Displays detailed information of a specific role and its permissions.
UC-13.3	Create New Role	Admin	Allows admins to create new custom roles with a name and description.
UC-13.4	Update Role	Admin	Allows admins to update a role's name and description.
UC-13.5	Delete Role	Admin	Allows admins to delete roles from the system.
UC-13.6	Update Role Permissions	Admin	Assigns route access rights to roles by granting or revoking permissions.

2.2.3 Use Case Specification

2.2.1 Authentication

2.2.3.1 UC-01 Register

UC ID and Name	UC-01 Register		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026
Primary Actor:	Guest	Secondary Actors:	Email Server
Trigger:	When a guest submits the registration form on the registration page.		
Description:	Allows guests to create a new account by providing their full name, email, password, and selecting a role (Client or Freelancer). The system sends a one-time password (OTP) to the provided email for inline verification. Upon successful OTP verification and form submission, the account is created and activated immediately.		
Preconditions:	**PRE-1** The user is not currently authenticated. **PRE-2** The system is connected to the email server.		
Post-conditions:	**POST-1** A new user account is created in the database with the selected role (Client or Freelancer) and an active email verification status. **POST-2** The guest is redirected to the appropriate onboarding or dashboard page.		

Normal Flow:	**A. Successful Registration** 1. Guest navigates to the registration page. 2. Guest enters their full name, email address, password, and password confirmation. 3. Guest selects their role (Client or Freelancer). 4. Guest clicks the "Send OTP" button. 5. The system validates the input fields. 6. The system checks that the email is not already registered. 7. The system generates a time-limited OTP and sends it to the provided email address. 8. Guest enters the received OTP code. 9. Guest checks the "I agree to the Terms of Service and Privacy Policy" checkbox. 10. Guest clicks the "Continue to Profile" button. 11. The system validates the OTP. 12. The system creates the new user account and sets the email verification status to verified. 13. The system redirects the Guest to the appropriate onboarding or dashboard page.
Alternative Flows:	**A.5 Required Fields Missing** 1. The system detects that a required field is empty. 2. The system displays the message "All fields are required." 3. Return to step A.2. **A.5 Invalid Email Format** 1. The system detects that the entered email does not match a valid format. 2. The system displays the message "Please enter a valid email address." 3. Return to step A.2. **A.5 Password Too Weak** 1. The system detects that the password does not meet the minimum strength requirements. 2. The system displays the message "Password must be at least 8 characters and include at least one uppercase letter, one number, and one special character." 3. Return to step A.2. **A.5 Password Confirmation Mismatch** 1. The system detects that the password confirmation does not match the password. 2. The system displays the message "Passwords do not match." 3. Return to step A.2.

	A.6 Email Already Registered 1. The system finds an existing account associated with the entered email. 2. The system displays the message “This email is already registered. Please log in or use a different email.” 3. Return to step A.2. **A.11 OTP Expired** 1. The system detects that the OTP has expired. 2. The system displays the message “OTP has expired. Please request a new one.” 3. Return to step A.4. **A.11 OTP Not Found** 1. The system detects that the entered OTP does not match any issued code. 2. The system displays the message “Invalid OTP. Please check your email and try again.” 3. Return to step A.8. **A.10 Terms of Service Not Accepted** 1. Guest clicks “Continue to Profile” without checking the Terms of Service checkbox. 2. The system displays the message “You must agree to the Terms of Service and Privacy Policy to continue.” 3. Return to step A.9.
Exceptions:	**EX-01 Email Server Failure** 1. The system fails to send the OTP email. 2. The system displays the message “Unable to send OTP. Please try again later.” 3. The use case ends. **EX-02 Database Connection Failure** 1. The system fails to create the user account. 2. The system displays the message “Registration failed. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	Medium

Business Rules:	BR-UA-10: Only Guest users may register a new account. BR-UA-11: Email must be unique across all accounts in the system. BR-UA-12: The user must select either the Client or Freelancer role at registration; no default role is assigned. BR-UA-13: Passwords must be stored as hashed values; plain-text passwords shall never be stored. BR-UA-14: The guest must accept the Terms of Service and Privacy Policy before the account is created. BR-UA-15: Email verification is performed inline via OTP during registration; the account is activated upon successful OTP validation.
Other Information:	The OTP sent via email shall expire after 10 minutes. If expired, the guest may click "Send OTP" again to receive a new code. Each OTP is single-use and is invalidated immediately after successful verification.
Assumptions:	The email server is configured and accessible at the time of registration. Password hashing is handled server-side before storage.

2.2.3.2 UC-02 Google Login

UC ID and Name	UC-02 Google Login		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026
Primary Actor:	Guest	Secondary Actors:	Google OAuth Server
Trigger:	When a user clicks the "Login with Google" button on the login or registration page.		
Description:	Allows guests to authenticate or register using their Google account via OAuth 2.0. Upon success, the system issues a JWT stored in an httpOnly cookie and redirects the user to the appropriate dashboard based on their role.		
Preconditions:	**PRE-1** The user is not currently authenticated. **PRE-2** The system is connected to the Google OAuth Server.		
Post-conditions:	**POST-1** The user is authenticated and a JWT is issued and stored in an httpOnly cookie. **POST-2** If the Google account is new to the system, a user account is automatically created with the Client role by default. **POST-3** The user is redirected to their role-appropriate dashboard.		

Normal Flow:	**A. Successful Google Login** 1. Guest clicks the "Login with Google" button. 2. The system redirects the Guest to the Google OAuth consent screen. 3. Guest grants permission on the Google consent screen. 4. Google redirects back to the system with an authorization code. 5. The system exchanges the authorization code for a Google access token. 6. The system retrieves the user's profile information (email, name) from Google. 7. The system checks whether an account with the returned email already exists. 8. The system loads the existing user record. 9. The system issues a JWT and stores it in an httpOnly cookie. 10. The system redirects the Guest to the role-appropriate dashboard.
Alternative Flows:	**A.3 Google Permission Denied** 1. Guest denies permission on the Google consent screen. 2. The system displays the message "Google login was cancelled. Please try again." 3. Return to step A.1. **A.7 New Google Account** 1. The system determines that no account exists for the returned email. 2. The system automatically creates a new account using the Google profile data, assigning the Client role by default. 3. Proceed to step A.9. **A.8 Account Suspended** 1. The system finds that the matched account has been suspended. 2. The system displays the message "Your account has been suspended. Please contact support." 3. The use case ends.
Exceptions:	**EX-01 Google Token Exchange Failure** 1. The system fails to exchange the authorization code for a Google access token. 2. The system displays the message "Unable to connect to Google. Please try again later." 3. The use case ends. **EX-02 Database Connection Failure** 1. The system fails to issue or store the JWT. 2. The system displays the message "Authentication failed. Please try again later." 3. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	BR-UA-01: JWT shall be stored exclusively in an httpOnly cookie to prevent client-side script access. BR-UA-02: A Google account not previously registered in the system shall be automatically registered with the Client role. BR-UA-03: Suspended accounts shall not be permitted to authenticate.
Other Information:	Auto-registered accounts via Google OAuth do not require email verification, as Google has already verified the email address. The redirect destination after login is determined by the user's role (Client or Freelancer dashboard).
Assumptions:	Google OAuth 2.0 credentials are configured in the system. The Google OAuth Server is accessible at the time of login.

2.2.3.3 UC-05 Forgot Password

UC ID and Name	UC-05 Forgot Password		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026
Primary Actor:	Guest	Secondary Actors:	Email Server
Trigger:	When a user clicks the "Forgot Password" link on the login page and submits their email address.		
Description:	Allows users who have forgotten their password to request a password reset link via email. The user receives a time-limited link to set a new password.		
Preconditions:	**PRE-1** The user is not currently authenticated. **PRE-2** The system is connected to the email server.		
Post-conditions:	**POST-1** A password reset link is sent to the provided email address if an account exists. **POST-2** The user is informed to check their email regardless of whether the account exists (to prevent email enumeration). **POST-3** Upon visiting the reset link and submitting a new password, the account password is updated and all existing JWTs are invalidated.		

Normal Flow:	**A. Successful Password Reset** 1. Guest clicks the “Forgot Password” link on the login page. 2. The system displays the Forgot Password form. 3. Guest enters their registered email address. 4. Guest clicks the “Send Reset Link” button. 5. The system validates the email format. 6. The system looks up the account by email. 7. The system generates a time-limited password reset token. 8. The system sends the password reset link to the provided email address. 9. The system displays the message “If an account exists for this email, a reset link has been sent.” 10. Guest opens the email and clicks the reset link. 11. The system validates the reset token. 12. The system displays the Reset Password form. 13. Guest enters and confirms the new password. 14. Guest clicks the “Reset Password” button. 15. The system validates the new password and updates the account password. 16. The system invalidates all existing JWTs associated with the account. 17. The system displays the message “Password reset successful. You may now log in with your new password.”
Alternative Flows:	**A.5 Invalid Email Format** 1. The system detects that the submitted email does not match a valid format. 2. The system displays the message “Please enter a valid email address.” 3. Return to step A.3. **A.6 Account Not Found** 1. The system finds no account associated with the provided email. 2. The system displays the message “If an account exists for this email, a reset link has been sent.” 3. No email is sent; the use case ends. **A.11 Reset Token Not Found** 1. The system detects that the reset token does not exist. 2. The system displays the message “This reset link is invalid. Please request a new one.” 3. Return to step A.10. **A.11 Reset Token Expired** 1. The system detects that the reset token has expired. 2. The system displays the message “This reset link has expired. Please request a new one.” 3. Return to step A.10.

	A.13 Password Validation Failure 1. The system detects that the new password does not meet the strength requirements or the confirmation does not match. 2. The system displays the message “Passwords do not match or do not meet the requirements.” 3. Return to step A.13.
Exceptions:	**EX-01 Database Connection Failure** 1. The system fails to generate or store the reset token. 2. The system displays the message “An error occurred. Please try again later.” 3. The use case ends. **EX-02 Email Server Failure** 1. The system fails to send the reset email. 2. The system displays the message “Unable to send reset email. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-UA-15: The system shall return the same response message regardless of whether the submitted email is registered, to prevent email enumeration. BR-UA-16: Password reset tokens shall expire after 1 hour. BR-UA-17: Each reset token is single-use; it is invalidated immediately after the password is successfully reset. BR-UA-18: Upon successful password reset, all active JWTs for the account shall be invalidated to terminate any existing sessions.
Other Information:	This use case applies only to accounts registered via email and password. Accounts authenticated exclusively via Google OAuth do not have a stored password and shall be directed to use Google Login instead.
Assumptions:	The email server is configured and accessible. A token storage mechanism is available to persist and validate reset tokens.

2.2.3.4 UC-07 Login

UC ID and Name	UC-07 Login		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026
Primary Actor:	Guest	Secondary Actors:	N/A
Trigger:	When a user submits their email and password on the login page.		

Description:	Allows guests to authenticate using their registered email and password. Upon success, the system issues a JWT stored in an httpOnly cookie and redirects the user to their role-appropriate dashboard.
Preconditions:	**PRE-1** The user is not currently authenticated. **PRE-2** The user has a registered account with email and password.
Post-conditions:	**POST-1** The user is authenticated and a JWT is issued and stored in an httpOnly cookie. **POST-2** The user is redirected to their role-appropriate dashboard.
Normal Flow:	**A. Successful Login** 1. Guest navigates to the login page. 2. Guest enters their email and password. 3. Guest clicks the "Login" button. 4. The system validates that both fields are not empty. 5. The system looks up the account by email. 6. The system verifies the provided password against the stored hash. 7. The system checks that the account's email has been verified. 8. The system issues a JWT and stores it in an httpOnly cookie. 9. The system redirects the Guest to the role-appropriate dashboard.
Alternative Flows:	**A.4 Required Fields Missing** 1. The system detects that the email or password field is empty. 2. The system displays the message "Email and password are required." 3. Return to step A.2. **A.5 Account Not Found** 1. The system finds no account associated with the provided email. 2. The system displays the message "Invalid email or password." 3. Return to step A.2. **A.5 Account Suspended** 1. The system finds that the matched account has been suspended. 2. The system displays the message "Your account has been suspended. Please contact support." 3. The use case ends. **A.6 Incorrect Password** 1. The system detects that the provided password does not match the stored hash. 2. The system displays the message "Invalid email or password." 3. Return to step A.2.

	A.7 Email Not Verified 1. The system detects that the account's email has not been verified. 2. The system displays the message "Please verify your email before logging in. Resend verification email?" 3. Return to step A.2.
Exceptions:	**EX-01 Database Connection Failure** 1. The system fails to issue or store the JWT. 2. The system displays the message "Login failed. Please try again later." 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-UA-04: JWT shall be stored exclusively in an httpOnly cookie. BR-UA-05: The system shall not distinguish between "account not found" and "incorrect password" in error messages to prevent account enumeration. BR-UA-06: Accounts with unverified emails shall not be permitted to log in. BR-UA-07: Suspended accounts shall not be permitted to authenticate.
Other Information:	The redirect destination after login is determined by the user's role (Client, Freelancer, or Admin dashboard).
Assumptions:	Passwords are stored as hashed values in the database.

2.2.3.5 UC-08 Logout

UC ID and Name	UC-08 Logout		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When an authenticated user clicks the "Logout" button.		
Description:	Allows authenticated users to terminate their session by invalidating the JWT on the server side and clearing the httpOnly cookie on the client side.		
Preconditions:	**PRE-1** The user is currently authenticated with a valid JWT stored in an httpOnly cookie.		

Post-conditions:	**POST-1** The JWT is blacklisted on the server and can no longer be used for authentication. **POST-2** The httpOnly cookie is cleared on the client side. **POST-3** The user is redirected to the login page.
Normal Flow:	**A. Successful Logout** 1. Authenticated user clicks the “Logout” button. 2. The system sends a logout request to the server, including the current JWT. 3. The server validates the JWT and adds it to the blacklist. 4. The server clears the httpOnly cookie. 5. The system redirects the user to the login page.
Alternative Flows:	N/A
Exceptions:	**EX-01 Database Connection Failure** 1. The server fails to blacklist the token. 2. The system displays the message “Logout failed. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-UA-08: Logout must invalidate the JWT on the server side before clearing the cookie to prevent token reuse. BR-UA-09: Blacklisted tokens shall be rejected on all subsequent authenticated requests.
Other Information:	Blacklisted tokens should be purged from storage after their natural expiry time to prevent indefinite growth of the blacklist.
Assumptions:	A token blacklist mechanism (e.g., Redis or database table) is available in the system.

2.2.2 Manage Role

2.2.3.1 UC-13.1 View Role List

UC ID and Name	UC-13.1 View Role List		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin navigates to the Role Management section from the admin dashboard.		
Description:	Allows administrators to retrieve and view a list of all active roles in the system along with their assigned permissions.		

Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The system is connected to the database.
Post-conditions:	**POST-1** A list of all active roles is displayed along with their assigned permissions.
Normal Flow:	**A. View Role List** 1. Admin navigates to the Role Management page. 2. The system retrieves all active (non-deleted) roles from the roles table. 3. The system retrieves the associated permissions for each role. 4. The system displays the role list including role names, descriptions, and assigned permissions. 5. Admin selects a role to view its details (UC-13.2).
Alternative Flows:	**A.4 No Roles Found** 1. The system finds no active roles. 2. The system displays the message “No roles available.” 3. Return to step A.1.
Exceptions:	**EX-01 Database Connection Failure** 1. The system displays the message “Unable to load role list. Please try again later.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-RM-01: Only authenticated Admins can access the Role Management section. BR-RM-02: Each role displayed shall include its name, description, and list of assigned permissions. BR-RM-03: Soft-deleted roles shall not appear in the active role list.
Other Information:	Roles are displayed in ascending order by creation date. Soft-deleted roles are excluded from the list.
Assumptions:	Roles and permissions are maintained in the system database.

2.2.3.2 UC-13.2 View Role Detail

UC ID and Name	UC-13.2 View Role Detail		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin selects a specific role from the role list, or navigates directly to a role detail URL.		

Description:	Allows administrators to view detailed information of a specific active role, including its name, description, and the full list of assigned permissions.
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The selected role exists and has not been soft-deleted.
Post-conditions:	**POST-1** The detailed information of the selected role is displayed.
Normal Flow:	**A. View Role Detail** 1. Admin selects a role from the role list or navigates directly to the role detail URL. 2. The system validates that the role exists and is not soft-deleted. 3. The system retrieves the role information using the role ID. 4. The system retrieves all permissions assigned to the selected role. 5. The system displays the role name, description, and list of assigned permissions.
Alternative Flows:	**A.2 Role Not Found** 1. The system determines the role does not exist or is soft-deleted. 2. The system displays the message “Role not found.” 3. Return to step A.1.
Exceptions:	**EX-01 Database Connection Failure** 1. The system displays the message “Unable to load role details. Please try again later.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-RM-04: Only authenticated Admins can view role details. BR-RM-05: Role details shall include name, description, and the complete list of assigned permissions. BR-RM-06: Soft-deleted roles are not accessible via the detail view.
Other Information:	If the admin navigates directly to a role detail URL, the system shall validate role existence and active status before rendering the page.
Assumptions:	Roles and their associated permissions are maintained in the system database.

2.2.3.3 UC-13.3 Create New Role

UC ID and Name	UC-13.3 Create New Role		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026

Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the “Create Role” button on the Role Management page.		
Description:	Allows administrators to create a new role by providing a name and an optional description. Permissions are assigned to the role separately through UC-13.6 Update Role Permissions.		
Preconditions:	**PRE-1** The user is authenticated as an Admin.		
Post-conditions:	**POST-1** A new role record is created in the roles table. **POST-2** The new role appears in the active role list.		
Normal Flow:	**A. Create New Role** 1. Admin clicks the “Create Role” button. 2. The system displays the Create Role form with fields for the role name and an optional description. 3. Admin enters the role name and optional description. 4. Admin clicks the “Save” button. 5. The system validates the input data. 6. The system creates a new record in the roles table. 7. The system redirects the admin to the role detail page of the newly created role.		
Alternative Flows:	**A.4 Admin Cancels Creation** 1. Admin clicks the “Cancel” button (instead of “Save”). 2. The system discards all unsaved data. 3. The system returns to the Role Management page. 4. The use case ends. **A.5 Required Fields Missing** 1. The system detects the role name is empty. 2. The system displays the message “Role name is required.” 3. Return to step A.3. **A.5 Duplicate Role Name** 1. The system detects that a role with the same name already exists. 2. The system displays the message “A role with this name already exists.” 3. Return to step A.3. **A.5 System Role Name Not Allowed** 1. The system detects that the entered name matches a system role name (Admin, Client, or Freelancer). 2. The system displays the message “Cannot use a system role name.” 3. Return to step A.3.		

Exceptions:	**EX-01 Database Connection Failure** 1. The system fails to create the role. 2. The system displays the message “Unable to create role. Please try again later.” 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-RM-07: Only authenticated Admins can create roles. BR-RM-08: Role name is a required field and must be unique across all active roles. BR-RM-09: System role names (Admin, Client, Freelancer) cannot be used for custom roles. BR-RM-10: A role is created without permissions; route access is granted separately via UC-13.6 Update Role Permissions.
Other Information:	The newly created role has no permissions until they are assigned through the Assign Role page.
Assumptions:	The admin remains authenticated throughout the creation process.

2.2.3.4 UC-13.4 Update Role

UC ID and Name	UC-13.4 Update Role		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin selects the “Edit” option on a specific role from the Role Management page.		
Description:	Allows administrators to update a role’s name and description. Permissions are revised separately through UC-13.6 Update Role Permissions.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The selected role exists and has not been soft-deleted.		
Post-conditions:	**POST-1** The role’s name and description are updated in the roles table. **POST-2** The role’s assigned permissions and the users assigned to it are not affected by the update.		

Normal Flow:	**A. Update Role** 1. Admin selects the “Edit” option on a role. 2. The system retrieves the current role information and displays it in an editable form. 3. Admin modifies the role name and/or description. 4. Admin clicks the “Save Changes” button. 5. The system validates the updated input data. 6. The system updates the role record in the roles table. 7. The system displays the updated role detail page.
Alternative Flows:	**A.2 Role Not Found** 1. The system determines the role does not exist or is soft-deleted. 2. The system displays the message “Role not found.” 3. Return to step A.1. **A.4 Admin Cancels Update** 1. Admin clicks the “Cancel” button (instead of “Save Changes”). 2. The system discards all unsaved changes. 3. The system returns to the role detail page. 4. The use case ends. **A.5 Required Fields Missing** 1. The system detects the role name is empty. 2. The system displays the message “Role name is required.” 3. Return to step A.3. **A.5 Duplicate Role Name** 1. The system detects that the updated name conflicts with another existing role. 2. The system displays the message “A role with this name already exists.” 3. Return to step A.3. **A.5 System Role Name Not Allowed** 1. The system detects that the updated name matches a system role name (Admin, Client, or Freelancer). 2. The system displays the message “Cannot use a system role name.” 3. Return to step A.3.
Exceptions:	**EX-01 Database Connection Failure** 1. The system fails to update the role. 2. The system displays the message “Unable to update role. Please try again later.” 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	BR-RM-11: Only authenticated Admins can update roles. BR-RM-12: Role name is a required field and must remain unique across all active roles. BR-RM-13: Updating a role's name or description shall not alter its assigned permissions or its users. BR-RM-14: System role names (Admin, Client, Freelancer) cannot be used for custom roles.
Other Information:	The original role ID remains unchanged after the update. The role name shown for users assigned to the role is updated along with the role record.
Assumptions:	The admin remains authenticated throughout the update process.

2.2.3.5 UC-13.5 Delete Role

UC ID and Name	UC-13.5 Delete Role		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin selects the "Delete" option on a specific role from the Role Management page.		
Description:	Allows administrators to remove an active role from the system by performing a soft delete operation. The role is retained in the database for auditing purposes.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The selected role exists in the system. **PRE-3** The selected role has not already been soft-deleted.		
Post-conditions:	**POST-1** The role is soft-deleted by setting the `deleted_at` field to the current timestamp. **POST-2** The deleted role no longer appears in the active role list. **POST-3** Users previously assigned to the deleted role are no longer granted its permissions.		
Normal Flow:	**A. Delete Role** 1. Admin selects the "Delete" option on a role. 2. The system displays a confirmation dialog: "Are you sure you want to delete this role? This action cannot be undone." 3. Admin confirms the deletion request. 4. The system performs a soft delete by setting the `deleted_at` field of the selected role to the current timestamp. 5. The system removes the role from the active role list. 6. The system redirects the admin to the Role Management page.		

Alternative Flows:	**A.1 Role Not Found** 1. The system determines the selected role does not exist. 2. The system displays the message “Role not found.” 3. Return to step A.1. **A.1 Role Already Deleted** 1. The system determines the role has already been soft-deleted. 2. The system displays the message “This role has already been deleted.” 3. Return to step A.1. **A.1 Role Currently Assigned to Users** 1. The system detects the role is assigned to one or more active users. 2. The system displays the message “This role cannot be deleted while it is assigned to active users.” 3. Return to step A.1. **A.3 Admin Cancels Deletion** 1. Admin clicks the “Cancel” button in the confirmation dialog. 2. The system closes the dialog. 3. The role remains unchanged. 4. The use case ends.
Exceptions:	**EX-01 Database Connection Failure** 1. The system displays the message “Unable to delete role. Please try again later.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-RM-15: Only authenticated Admins can delete roles. BR-RM-16: Role deletion shall be implemented as a soft delete using the `deleted_at` field. BR-RM-17: Soft-deleted roles shall not appear in the active role list. BR-RM-18: A role that is currently assigned to active users shall not be deleted.
Other Information:	Deleted roles remain stored in the database for auditing and recovery purposes.
Assumptions:	The admin remains authenticated throughout the deletion process.

2.2.3.6 UC-13.6 Update Role Permissions

UC ID and Name	UC-13.6 Update Role Permissions		
Created By:	Nguyen Tien Thuan	Date Created:	10/09/2026

Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin selects a role on the Assign Role page and clicks the “Save permissions” button.		
Description:	Allows administrators to assign route access rights to a role by granting or revoking permissions. Each permission represents a server route defined by an HTTP method and path within a module. The system replaces the role’s entire permission set with the newly selected permissions and the change takes effect immediately.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The selected role exists and has not been soft-deleted. **PRE-3** Permissions are predefined in the system.		
Post-conditions:	**POST-1** All existing permissions of the role are replaced with the newly selected set. **POST-2** The role’s route access rights take effect immediately for all users assigned to the role.		
Normal Flow:	**A. Update Role Permissions** 1. Admin navigates to the Assign Role page. 2. Admin selects a role from the role dropdown. 3. The system retrieves the role detail including its current permissions. 4. The system retrieves all active permissions and displays them grouped by module, pre-checking the role’s current permissions. 5. Admin selects or deselects permissions individually or by module. 6. Admin clicks the “Save permissions” button. 7. The system validates that all selected permissions exist and are active. 8. The system removes all existing permission associations of the role. 9. The system creates new permission associations for the selected permissions. 10. The system displays the message “Updated permissions for [role name]” and the role detail reflects the new permission set.		
Alternative Flows:	**A.2 No Role Selected** 1. Admin does not select a role before proceeding. 2. The system displays the message “Choose a role to start assigning permissions.” 3. Return to step A.1. **A.3 Role Not Found** 1. The system determines the selected role does not exist or is soft-deleted. 2. The system displays the message “Role not found.” 3. Return to step A.2.		

	A.6 Clear All Permissions 1. Admin deselects all permissions and clicks the “Save permissions” button. 2. The system validates the empty selection. 3. The system removes all existing permission associations of the role. 4. The role is left with no route access rights. 5. Proceed to step A.10. **A.7 Invalid Permission Selected** 1. The system detects that one or more selected permissions do not exist or are inactive. 2. The system displays the message “One or more selected permissions are invalid.” 3. Return to step A.5.
Exceptions:	**EX-01 Database Connection Failure** 1. The system fails to replace the role’s permission associations. 2. The system displays the message “Unable to update role permissions. Please try again later.” 3. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-RM-19: Only authenticated Admins can update role permissions. BR-RM-20: Each permission maps to a server route defined by an HTTP method and path within a module; assigning permissions grants the role access to those routes. BR-RM-21: Updating a role’s permissions shall replace the role’s entire existing permission set with the newly selected set. BR-RM-22: All selected permission IDs must exist and be active in the permissions table. BR-RM-23: Permission changes take effect immediately for all users currently assigned to the role.
Other Information:	The Assign Role page lists permissions grouped by module and supports filtering by HTTP method and module. An empty permission set clears all route access for the role. Because replacement takes effect immediately, updating a role that is assigned to active users will alter their access rights without further confirmation.
Assumptions:	Permissions are predefined and managed separately in the system. The admin remains authenticated throughout the process.

2.2.3 Manage Freelancer Profile

2.2.3.1 UC-10.1 View Detailed Freelancer Profile

UC ID and Name	UC-10.1 View Detailed Freelancer Profile		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Guest, Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user navigates to a freelancer's profile page.		
Description:	Allows guests or registered users to retrieve and view all profile information of a freelancer, including basic account data and detailed professional records merged from both the profiles and freelancer_profiles tables.		
Preconditions:	**PRE-1** The selected freelancer profile exists and is active in the database.		
Post-conditions:	**POST-1** The freelancer's detailed profile information is successfully displayed.		
Normal Flow:	**A. View Detailed Freelancer Profile** 1. User navigates to the selected freelancer's profile page. 2. The system retrieves the user's basic profile data from the `profiles` table. 3. The system retrieves the user's professional data from the `freelancer_profiles` table. 4. The system merges the retrieved data. 5. The system displays the complete detailed freelancer profile to the user.		
Alternative Flows:	N/A		
Exceptions:	**EX-01 Profile Not Found** 1. The system displays the message "Freelancer profile not found." 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message "Unable to load profile details. Please try again later." 2. The use case ends.		
Priority:	High		
Frequency of Use:	High		
Business Rules:	BR-PROF-01: Both basic details (from profiles) and professional details (from freelancer_profiles) must be presented together in a single view.		
Other Information:	N/A		

Assumptions:	The database correctly links the profiles and freelancer_profiles tables via a unique user identifier.

2.2.3.2 UC-10.2 Update Your Detailed Freelancer Profile

UC ID and Name	UC-10.2 Update Your Detailed Freelancer Profile		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When an authenticated freelancer clicks the "Edit Profile" button on their own profile page.		
Description:	Allows a freelancer to update their personal and professional details such as display name, bio, title, education, certifications, languages, and availability status.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged into the system. **PRE-2** The freelancer is viewing their own profile settings page.		
Post-conditions:	**POST-1** The updated profile information is saved in the profiles and freelancer_profiles tables. **POST-2** The updated information is visible on the freelancer's public profile page.		
Normal Flow:	**A. Update Detailed Freelancer Profile** 1. Freelancer clicks the "Edit Profile" button. 2. The system displays the profile editing form populated with current data. 3. Freelancer modifies fields: display_name, bio, title, education, certifications, languages, and availability_status. 4. Freelancer clicks the "Save Changes" button. 5. The system validates the input data for all modified fields. 6. The system updates the corresponding records in the `profiles` and `freelancer_profiles` tables. 7. The system displays the message: "Profile updated successfully." 8. The system redirects the user to their updated profile page.		
Alternative Flows:	**A.4 Cancel Update** 1. Freelancer clicks the "Cancel" button. 2. The system discards all unsaved changes. 3. Return to step A.1.		
Exceptions:	**EX-01 Invalid Input Data** 1. The system highlights the error fields and displays a warning message: "Please enter valid information for all fields." 2. The use case ends.		

	EX-02 Database Connection Failure 1. The system displays the message: “Profile update failed. Please try again later.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-PROF-02: Freelancers can only update their own profile. BR-PROF-03: Mandatory fields (e.g., display_name, title) cannot be left empty.
Other Information:	Some updates (like availability_status) may immediately impact search visibility for clients.
Assumptions:	The freelancer remains authenticated throughout the process.

2.2.4 Manage Freelancer Skills

2.2.3.1 UC-11.1 View List Freelancer Skill

UC ID and Name	UC-11.1 View List Freelancer Skill		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Guest, Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user accesses the skills section of a freelancer’s profile page.		
Description:	Allows users to retrieve and view the complete list of skills added by a specific freelancer, including their proficiency levels.		
Preconditions:	**PRE-1** The system is connected to the database. **PRE-2** The freelancer profile exists.		
Post-conditions:	**POST-1** The list of skills and corresponding proficiency levels associated with the freelancer is displayed.		
Normal Flow:	**A. View List Freelancer Skill** 1. User accesses the freelancer’s profile or skills section. 2. The system queries the database to retrieve all skills associated with the freelancer’s ID. 3. The system loads the skill names and their respective proficiency levels. 4. The system displays the list of skills to the user.		
Alternative Flows:	N/A		
Exceptions:	**EX-01 No Skills Found** 1. The system displays the message “No skills added yet.” 2. The use case ends.		

	EX-02 Database Connection Failure 1. The system displays the message “Unable to load skills. Please try again later.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-SKILL-01: All skills belonging to the freelancer must be displayed along with their proficiency level.
Other Information:	N/A
Assumptions:	Skills are properly linked to the freelancer’s ID in the database.

2.2.3.2 UC-11.2 Add Freelancer Skill

UC ID and Name	UC-11.2 Add Freelancer Skill		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer submits a new skill through the skill input form.		
Description:	Allows an authenticated freelancer to add a new skill to their profile along with an indicating proficiency level.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The freelancer is on their profile or skills settings page.		
Post-conditions:	**POST-1** A new skill record is created and associated with the freelancer in the database. **POST-2** The newly added skill is displayed in the freelancer’s skill list.		
Normal Flow:	**A. Add Freelancer Skill** 1. Freelancer clicks the “Add Skill” button. 2. The system displays a form to input the skill. 3. Freelancer enters or selects a skill name and selects a proficiency level. 4. Freelancer clicks the “Save” button. 5. The system validates the selected skill and proficiency level. 6. The system checks if the skill already exists in the freelancer’s profile. 7. The system inserts the new skill record into the database. 8. The system displays the message: “Skill added successfully.” 9. The system updates the displayed list of skills.		

Alternative Flows:	**A.4 Cancel Addition** 1. Freelancer clicks the “Cancel” button. 2. The system discards the input. 3. Return to step A.1.
Exceptions:	**EX-01 Missing Required Information** 1. The system displays the message “Please select a skill and a proficiency level.” 2. The use case ends. **EX-02 Skill Already Exists** 1. The system displays the message “This skill has already been added to your profile.” 2. The use case ends. **EX-03 Database Connection Failure** 1. The system displays the message “Unable to add skill. Please try again later.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-SKILL-02: A freelancer cannot add duplicate skills to their profile. BR-SKILL-03: Both skill name and proficiency level are required.
Other Information:	Proficiency levels are typically predefined (e.g., Beginner, Intermediate, Expert).
Assumptions:	The system provides a standardized list of skills.

2.2.3.3 UC-11.3 Delete Freelancer Skill

UC ID and Name	UC-11.3 Delete Freelancer Skill		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer clicks the “Delete” option next to a specific skill.		
Description:	Allows a freelancer to permanently remove (hard delete) a skill from their profile.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The skill to be deleted currently exists in the freelancer’s profile.		

Post-conditions:	**POST-1** The specific skill record is permanently deleted from the database. **POST-2** The skill is removed from the freelancer's displayed skill list.
Normal Flow:	**A. Delete Freelancer Skill** 1. Freelancer clicks the "Delete" option for a specific skill. 2. The system prompts a confirmation dialog. 3. Freelancer clicks the "Confirm" button. 4. The system executes a hard delete operation on the database for the selected skill record. 5. The system displays the message: "Skill removed successfully." 6. The system refreshes the skill list.
Alternative Flows:	**A.3 Cancel Deletion** 1. Freelancer clicks the "Cancel" button in the confirmation dialog. 2. The system closes the dialog. 3. Return to step A.1.
Exceptions:	**EX-01 Skill Not Found** 1. The system displays the message "Skill not found or already deleted." 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message "Unable to remove skill. Please try again later." 2. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-SKILL-04: Skill deletion must be a hard delete operation; the record is permanently removed from the mapping table. BR-SKILL-05: Only the owning freelancer can delete a skill from their profile.
Other Information:	N/A
Assumptions:	Deleting a skill from a freelancer's profile does not delete the core skill definition from the system's global master list of skills.

2.2.5 Manage Freelancer Portfolio

2.2.3.1 UC-12.1 View Portfolio List

UC ID and Name	UC-12.1 View Portfolio List		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer accesses the portfolio section of their profile.		
Description:	Allows the freelancer to retrieve and view the list of portfolio items they have created.		
Preconditions:	**PRE-1** The system is connected to the database. **PRE-2** The freelancer profile exists and the user is authenticated.		
Post-conditions:	**POST-1** A list of the freelancer's active portfolios is displayed.		
Normal Flow:	**A. Get Portfolio List** 1. Freelancer navigates to their portfolio section. 2. The system retrieves all portfolios associated with the freelancer's ID. 3. The system filters out any soft-deleted portfolios (where deleted_at is not null). 4. The system loads portfolio summaries (e.g., thumbnail, title, short description). 5. The system displays the portfolio list.		
Alternative Flows:	N/A		
Exceptions:	**EX-01 No Portfolios Found** 1. The system displays the message "No portfolios available." 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message "Unable to load portfolios. Please try again later." 2. The use case ends.		
Priority:	High		
Frequency of Use:	High		
Business Rules:	BR-PORT-01: Only portfolios with deleted_at = NULL shall be displayed.		
Other Information:	N/A		
Assumptions:	Portfolios are linked correctly to the freelancer ID.		

2.2.3.2 UC-12.2 Add New Portfolio

UC ID and Name	UC-12.2 Add New Portfolio		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A

Trigger:	When the freelancer submits a new portfolio through the creation form.
Description:	Allows an authenticated freelancer to create a new portfolio entry, optionally uploading media files (images, documents, links).
Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The freelancer is viewing their own portfolio management section.
Post-conditions:	**POST-1** A new portfolio record is created in the database. **POST-2** Associated media files are uploaded and linked to the new portfolio. **POST-3** The new portfolio is visible in the freelancer's portfolio list.
Normal Flow:	**A. Add New Portfolio** 1. Freelancer clicks the "Add Portfolio" button. 2. The system displays the Add Portfolio form. 3. Freelancer enters portfolio details (title, description, role, technologies used). 4. Freelancer optionally uploads media files or provides external links. 5. Freelancer clicks the "Save" button. 6. The system validates the input data and media file formats/sizes. 7. The system uploads and stores the media files. 8. The system creates a new portfolio record in the database. 9. The system displays the message: "Portfolio created successfully." 10. The system redirects the freelancer to the updated portfolio list.
Alternative Flows:	**A.5 Cancel Creation** 1. Freelancer clicks the "Cancel" button. 2. The system discards the input and any pending uploads. 3. Return to step A.1.
Exceptions:	**EX-01 Required Fields Missing** 1. The system displays the message "Portfolio title is required." 2. The use case ends. **EX-02 Invalid Media Upload** 1. The system displays the message "Invalid file format or size limit exceeded." 2. The use case ends. **EX-03 Database or Storage Failure** 1. The system displays the message "Unable to create portfolio. Please try again later." 2. The use case ends.
Priority:	High

Frequency of Use:	Medium
Business Rules:	BR-PORT-02: Portfolio title is a required field. BR-PORT-03: Media uploads must conform to system-defined size and type restrictions.
Other Information:	N/A
Assumptions:	The system has sufficient storage to handle media file uploads.

2.2.3.3 UC-12.3 View Portfolio Detail

UC ID and Name	UC-12.3 View Portfolio Detail		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer selects a specific portfolio item from the portfolio list.		
Description:	Allows the freelancer to view detailed information of a specific portfolio, including its full description and attached media.		
Preconditions:	**PRE-1** The selected portfolio item exists. **PRE-2** The portfolio has not been soft-deleted.		
Post-conditions:	**POST-1** The complete details of the portfolio are displayed successfully.		
Normal Flow:	**A. Get Portfolio Detail** 1. Freelancer clicks on a portfolio item from the list. 2. The system retrieves the portfolio details using the portfolio ID. 3. The system retrieves any associated media links/files. 4. The system displays the portfolio title, description, skills used, and media gallery.		
Alternative Flows:	N/A		
Exceptions:	**EX-01 Portfolio Not Found** 1. The system displays the message “Portfolio not found.” 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message “Unable to load portfolio details. Please try again later.” 2. The use case ends.		
Priority:	High		
Frequency of Use:	High		
Business Rules:	BR-PORT-04: Only portfolios with deleted_at = NULL can be accessed.		

Other Information:	N/A
Assumptions:	Media URLs associated with the portfolio are valid and accessible.

2.2.3.4 UC-12.4 Update Portfolio

UC ID and Name	UC-12.4 Update Portfolio		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer submits modified details for an existing portfolio.		
Description:	Allows a freelancer to update the information and/or replace the media files of an existing portfolio.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The freelancer is the owner of the selected portfolio. **PRE-3** The portfolio exists and is not soft-deleted.		
Post-conditions:	**POST-1** The portfolio information is updated in the database. **POST-2** New media is stored, and old removed media is unlinked/deleted. **POST-3** The updated portfolio details are visible.		
Normal Flow:	**A. Update Portfolio** 1. Freelancer clicks the "Edit" option for a portfolio. 2. The system loads the existing portfolio details into an editable form. 3. Freelancer modifies portfolio fields and/or uploads new media. 4. Freelancer clicks the "Save Changes" button. 5. The system validates the updated data and new media files. 6. The system updates the portfolio record in the database. 7. The system updates associated media files. 8. The system displays the message: "Portfolio updated successfully." 9. The system redirects the freelancer to the updated portfolio list view.		
Alternative Flows:	**A.4 Cancel Update** 1. Freelancer clicks the "Cancel" button. 2. The system discards all unsaved changes. 3. Return to step A.1.		
Exceptions:	**EX-01 Required Fields Missing** 1. The system displays the message "Portfolio title is required." 2. The use case ends.		

	EX-02 Invalid Media Upload 1. The system displays the message “Invalid file format or size limit exceeded.” 2. The use case ends. **EX-03 Database Connection Failure** 1. The system displays the message “Unable to update portfolio. Please try again later.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-PORT-05: Only the portfolio owner can edit it. BR-PORT-06: Portfolio title remains a required field.
Other Information:	N/A
Assumptions:	Old media files removed during the update may be permanently deleted from storage to save space.

2.2.3.5 UC-12.5 Delete Portfolio

UC ID and Name	UC-12.5 Delete Portfolio		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer clicks the “Delete” option next to a specific portfolio.		
Description:	Allows a freelancer to remove a portfolio from their profile by performing a soft delete operation.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The freelancer is the owner of the portfolio. **PRE-3** The portfolio exists and is not already deleted.		
Post-conditions:	**POST-1** The portfolio is soft-deleted by updating the deleted_at field. **POST-2** The portfolio is no longer visible in the portfolio list.		
Normal Flow:	**A. Delete Portfolio** 1. Freelancer clicks the “Delete” option next to a portfolio. 2. The system displays a confirmation dialog. 3. Freelancer clicks the “Confirm” button. 4. The system performs a soft delete by updating the deleted_at field for the portfolio record. 5. The system displays the message: “Portfolio deleted successfully.” 6. The system removes the portfolio from the visible list.		

Alternative Flows:	**A.3 Cancel Deletion** 1. Freelancer clicks the “Cancel” button in the confirmation dialog. 2. The system closes the dialog. 3. Return to step A.1.
Exceptions:	**EX-01 Portfolio Not Found** 1. The system displays the message “Portfolio not found.” 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message “Unable to delete portfolio. Please try again later.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-PORT-07: Portfolio deletion must be a soft delete (updating the deleted_at column). BR-PORT-08: Soft-deleted portfolios must not be displayed.
Other Information:	Soft-deleted portfolios can potentially be restored by the freelancer if an archive page is implemented.
Assumptions:	The soft delete does not immediately remove associated media files from storage.

2.2.6 Browse Jobs

2.2.3.1 UC-03.1 View List Job

UC ID and Name	UC-03.1 View List Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user accesses the Jobs page from the navigation menu.		
Description:	As a guest or registered user, I want to view a list of active jobs with pagination support and search/filter capabilities so that I can browse available opportunities or manage my posted jobs.		
Preconditions:	**PRE-1** The system is connected to the database. **PRE-2** At least one active job exists in the jobs table.		
Post-conditions:	**POST-1** A paginated list of active jobs is displayed. **POST-2** Users can click a job to view its details.		

Normal Flow:	**A. View List Job** 1. User accesses the job list page. 2. The system receives the request with default parameters (page 1, no filters, default sorting by created_at DESC). 3. The system queries the database to retrieve the job list based on role permissions. 4. The system calculates total records and total pages. 5. The system returns the first page of job list with pagination information. 6. User views the job list.
Alternative Flows:	**A.1a Search by Keyword** 1. User enters a keyword in the search bar before submitting the request. 2. The system searches jobs where title or description matches the keyword (case-insensitive). 3. The system displays matching jobs. 4. Return to step A.6. **A.1b Filter by Category** 1. User selects a category from the filter dropdown. 2. The system retrieves jobs matching the selected category. 3. The system displays filtered jobs. 4. Return to step A.6. **A.1c Filter by Budget Range** 1. User sets a minimum budget value. 2. The system retrieves jobs with budget greater than or equal to the value. 3. The system displays filtered jobs. 4. Return to step A.6. **A.1d Filter by Skills** 1. User selects one or more skills. 2. The system retrieves jobs requiring the selected skills. 3. The system displays filtered jobs. 4. Return to step A.6. **A.1e Combine Multiple Filters** 1. User applies multiple filters simultaneously (e.g., search + category + budget). 2. The system applies all filter conditions using AND logic. 3. The system displays combined results. 4. Return to step A.6. **A.5a No Jobs Found** 1. The system finds no jobs matching the entered criteria (zero total records). 2. The system displays the message: “No jobs available. Please adjust your filters or try again later.” 3. Return to step A.1.

	A.6a Navigate Between Pages 1. User clicks a page number, Next, or Previous button. 2. The system retrieves the corresponding page of jobs. 3. The system displays the requested page. 4. Return to step A.6.
Exceptions:	**EX-01 Database Connection Failure** 1. The system cannot connect to the database while retrieving job data. 2. The system displays the message: “Unable to load jobs. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	**BR-JOB-01:** Only jobs with status = ’open’ or ’in_progress’ shall be displayed to freelancers and guests. **BR-JOB-02:** Clients shall see all their jobs regardless of status. **BR-JOB-03:** Default sorting shall be by created_at in descending order. **BR-JOB-04:** The system shall support pagination with default page size of 10. **BR-JOB-05:** Search shall match keywords in title and description fields (case-insensitive). **BR-JOB-06:** When multiple filters are applied, all conditions shall be combined using AND logic.
Other Information:	Default page size: 10 jobs per page. Filters available: Category, Budget (min), Skills, Status (for clients only).
Assumptions:	Jobs are created by registered clients. Categories and skills are predefined in the system.

2.2.3.2 UC-03.2 View Job Detail

UC ID and Name	UC-03.2 View Job Detail		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user selects a job from the job list by clicking on the job title or "View Details" button.		
Description:	Allows users to view detailed information about a specific job including title, description, budget, deadline, category, required skills, and client information.		

Preconditions:	**PRE-1** The user is viewing the job list (UC-03.1). **PRE-2** The selected job exists and is active.
Post-conditions:	**POST-1** The selected job information is displayed successfully.
Normal Flow:	**A. View Job Detail** 1. User selects a job from the job list (clicks on job title or "View Details" button). 2. The system receives the request with job_id parameter. 3. The system queries the database to retrieve job information by job_id from the jobs table. 4. The system queries the client information associated with the job. 5. The system displays job details: title, description, budget, deadline, category, required skills, status, client name, and client avatar. 6. User views the job detail information.
Alternative Flows:	N/A
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist or has been deleted). 2. The system displays the message: "Job not found." 3. The use case ends. **EX-02 Job Inactive** 1. The system determines the job is inactive (status = 'closed' or 'deleted') and the requester is not the job owner. 2. The system displays the message: "This job is no longer available." 3. The use case ends. **EX-03 Database Connection Failure** 1. The system cannot retrieve job information due to a database connection failure. 2. The system displays the message: "Unable to load job details. Please try again later." 3. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	**BR-JOB-07:** Job information displayed includes: title, description, budget, deadline, category, required skills, client name, created date, status. **BR-JOB-08:** If the job is inactive (status = 'closed' or 'deleted'), only the job owner can view full details.
Other Information:	N/A
Assumptions:	The job exists and is associated with a valid client account.

2.2.3.3 UC-06.2 Post New Job

UC ID and Name	UC-06.2 Post New Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a client clicks the "Post New Job" or "Create Job" button from the job management page.		
Description:	Allows a registered client to create and publish a new job posting with details including title, description, budget, category, and required skills.		
Preconditions:	**PRE-1** The client is logged in to the system. **PRE-2** The client has an active account.		
Post-conditions:	**POST-1** A new job record is created in the jobs table with status = 'draft'. **POST-2** The job is saved to the database. **POST-3** The client is redirected to the job detail page or job list with a success message.		

Normal Flow:	**A. Post New Job** 1. Client navigates to the job management page. 2. Client clicks the "Post New Job" button. 3. The system displays the job creation form. 4. Client enters the following information: - Job Title (required, max 200 characters) - Description (required, max 5000 characters) - Budget (required, must be > 0) - Category (required, dropdown selection) - Required Skills (optional, tags input) 5. Client clicks the "Save Draft" or "Publish Job" button. 6. The system validates all input fields. 7. The system creates a new job record with: - title, description, budget, category - client_id = current user ID - status = 'draft' (if Save Draft) or 'open' (if Publish Job) - created_at = current timestamp 8. The system saves the skills associated with the job. 9. The system redirects to the job detail page with a success message.
Alternative Flows:	**A.2 Cancel Creation** 1. Client clicks the "Cancel" button. 2. The system discards all input data. 3. The system returns to the job list page. 4. The use case ends. **A.6a Validation Failed** 1. The system validates input fields and finds invalid or incomplete data. 2. The system highlights the errors and displays: "Invalid data. Please check your input." 3. Return to step A.4. **A.6b Budget Invalid** 1. The system finds that budget is less than or equal to 0. 2. The system displays: "Budget must be greater than 0." 3. Return to step A.4. **A.6c Missing Required Fields** 1. The system finds that title or description is empty. 2. The system displays: "Please fill in all required fields." 3. Return to step A.4.
Exceptions:	**EX-01 Database Insert Failure** 1. The system cannot save the new job record due to a database error. 2. The system displays: "Unable to create job. Please try again later." 3. The use case ends.

Priority:	High
Frequency of Use:	Medium
Business Rules:	**BR-JOB-09:** Job title is required and must not exceed 200 characters. **BR-JOB-10:** Job description is required and must not exceed 5000 characters. **BR-JOB-11:** Budget must be a positive number (greater than 0). **BR-JOB-12:** Category must be selected from predefined categories. **BR-JOB-13:** New jobs shall have status = 'draft' by default when saved as draft, or 'open' when published.
Other Information:	N/A
Assumptions:	The client has sufficient permissions to create jobs. Categories and skills are predefined in the system.

2.2.3.4 UC-06.3 Edit Job

UC ID and Name	UC-06.3 Edit Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a client clicks the "Edit" button on a job they own from the job list or job detail page.		
Description:	Allows the client who created a job to modify its details such as title, description, budget, category, skills, and status. Editing is restricted if the job has an active contract.		
Preconditions:	**PRE-1** The client is logged in to the system. **PRE-2** The client is the owner of the job. **PRE-3** The job exists and is not deleted. **PRE-4** The job does not have an active contract.		
Post-conditions:	**POST-1** The job record is updated with new information. **POST-2** The updated job is visible with new details. **POST-3** A log entry is created for the edit action.		

Normal Flow:	**A. Edit Job** 1. Client navigates to the job detail page or job list. 2. Client clicks the "Edit" button on a job they own. 3. The system retrieves the current job data and pre-fills the edit form. 4. Client modifies the following fields as needed: - Job Title - Description - Budget - Category - Required Skills - Status (if applicable) 5. Client clicks the "Save Changes" button. 6. The system validates the updated input. 7. The system updates the job record in the jobs table. 8. The system updates the associated skills. 9. The system logs the edit action with timestamp. 10. The system redirects to the job detail page with a success message.
Alternative Flows:	**A.2 Cancel Edit** 1. Client clicks the "Cancel" button. 2. The system discards all changes. 3. The system returns to the job detail page. 4. The use case ends. **A.6 Validation Failed** 1. The system validates the updated input and finds invalid or incomplete data. 2. The system highlights the errors and displays appropriate error messages. 3. Return to step A.4.
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Unauthorized Access** 1. The system determines the client is not the owner of the job. 2. The system displays: "You do not have permission to edit this job." 3. The use case ends. **EX-03 Database Update Failure** 1. The system cannot update the job record due to a database error. 2. The system displays: "Unable to update job. Please try again later." 3. The use case ends.

Priority:	High
Frequency of Use:	Medium
Business Rules:	**BR-JOB-14:** Only the job owner can edit the job. **BR-JOB-15:** Editing is restricted if the job has an active contract. **BR-JOB-16:** All validation rules from UC-06.2 apply to edits. **BR-JOB-17:** Edit actions shall be logged for auditing purposes.
Other Information:	N/A
Assumptions:	The client has sufficient permissions to edit jobs. The job exists and is accessible.

2.2.3.5 UC-06.4 Delete Job

UC ID and Name	UC-06.4 Delete Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a client clicks the "Delete" button on a job they own from the job list or job detail page.		
Description:	Allows the client who created a job to delete it (soft delete). The job is hidden from public view but retained in the database. The system checks for active proposals or contracts before allowing deletion.		
Preconditions:	**PRE-1** The client is logged in to the system. **PRE-2** The client is the owner of the job. **PRE-3** The job exists and is not already deleted. **PRE-4** No active proposals or contracts exist for this job.		
Post-conditions:	**POST-1** The job is soft-deleted (deleted_at field is set). **POST-2** The job is no longer visible to freelancers or guests. **POST-3** A log entry is created for the delete action.		

Normal Flow:	**A. Delete Job** 1. Client navigates to the job detail page or job list. 2. Client clicks the "Delete" button on a job they own. 3. The system displays a confirmation dialog: "Are you sure you want to delete this job? This action cannot be undone." 4. Client confirms by clicking "Yes, Delete". 5. The system checks for active proposals or contracts. 6. The system updates the job record: - status = 'deleted' - deleted_at = current timestamp 7. The system logs the delete action. 8. The system redirects to the job list with a success message: "Job deleted successfully."
Alternative Flows:	**A.3 Cancel Delete** 1. Client clicks the "Cancel" button in the confirmation dialog. 2. The system closes the dialog. 3. The system returns to the job detail page or job list. 4. The use case ends. **A.5 Job Cannot Be Deleted** 1. The system finds an active proposal or contract associated with the job. 2. The system displays: "This job cannot be deleted because it has active proposals/contracts. Please close them first." 3. The use case ends.
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Unauthorized Access** 1. The system determines the client is not the owner of the job. 2. The system displays: "You do not have permission to delete this job." 3. The use case ends. **EX-03 Database Update Failure** 1. The system cannot update the job record due to a database error. 2. The system displays: "Unable to delete job. Please try again later." 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	**BR-JOB-18:** Only the job owner can delete the job. **BR-JOB-19:** Deletion should be soft delete (set deleted_at timestamp, status = 'deleted'). **BR-JOB-20:** Jobs with active proposals or contracts cannot be deleted. **BR-JOB-21:** Delete actions shall be logged for auditing purposes.
Other Information:	Soft delete: job is hidden from public but retained in database for audit.
Assumptions:	The client has sufficient permissions to delete jobs. The job exists and is accessible.

2.2.3.6 UC-06.5 Change Job Status

UC ID and Name	UC-06.5 Change Job Status		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a client or system action triggers a status change on a job.		
Description:	Allows the job owner to change the status of a job following the workflow: draft → open → in_progress → completed → closed. Each transition has specific constraints and permissions. The system may also auto-transition statuses based on business events.		
Preconditions:	**PRE-1** The client is logged in to the system (for manual changes). **PRE-2** The client is the job owner. **PRE-3** The job exists and is not deleted.		
Post-conditions:	**POST-1** The job status is updated to the new status. **POST-2** A log entry is created for the status change. **POST-3** Notifications may be sent to relevant users.		
Normal Flow:	**A. Change Job Status** 1. Client navigates to the job detail page. 2. Client clicks the "Change Status" button. 3. The system displays the current status and available next statuses based on workflow rules. 4. Client selects the new status from the allowed options. 5. The system validates the transition is allowed. 6. The system updates the job status. 7. The system logs the status change. 8. The system sends notifications if applicable (e.g., to freelancers when job becomes open). 9. The system displays a success message: "Job status updated successfully."		

Alternative Flows:	**A.2a System Auto-Transition** 1. System event occurs (e.g., contract created, job completed). 2. System automatically updates job status based on business rules. 3. System logs the status change. 4. The use case ends. **A.2b Cancel Status Change** 1. Client clicks the "Cancel" button. 2. The system discards the status change. 3. The system returns to the job detail page. 4. The use case ends. **A.5 Invalid Transition** 1. The system validates and finds that the requested status transition is not allowed. 2. The system displays: "Status transition from [current] to [new] is not allowed." 3. Return to step A.4.
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Unauthorized Access** 1. The system determines the client is not the owner of the job. 2. The system displays: "You do not have permission to change job status." 3. The use case ends. **EX-03 Database Update Failure** 1. The system cannot update the job status due to a database error. 2. The system displays: "Unable to update status. Please try again later." 3. The use case ends.
Priority:	High
Frequency of Use:	Medium

Business Rules:	**BR-JOB-22:** Job status workflow: draft → open → in_progress → completed → closed. **BR-JOB-23:** Allowed transitions: - draft → open (client) - open → in_progress (client, when contract created) - open → closed (client, if cancelled) - in_progress → completed (client, when work done) - in_progress → closed (client, if cancelled) - completed → closed (system/auto) - Any status → deleted (soft delete) **BR-JOB-24:** Only the job owner can manually change status. **BR-JOB-25:** Status changes shall be logged for auditing purposes. **BR-JOB-26:** Notifications shall be sent to freelancers when status changes from draft to open.
Other Information:	Status definitions: - draft: Job created but not published - open: Job is visible and accepting proposals - in_progress: Job has been assigned and is being worked on - completed: Work is done, awaiting finalization - closed: Job is finished and closed - deleted: Job is soft-deleted
Assumptions:	The client has sufficient permissions to change job status. The job exists and is accessible.

2.2.3.7 UC-06.6 View List Bookmarked Job

UC ID and Name	UC-06.6 View List Bookmarked Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer accesses the "Bookmarked Jobs" page from the navigation menu or profile.		
Description:	Allows a freelancer to view a list of all jobs they have bookmarked. The list displays job title, budget, client name, and bookmark date with pagination support.		
Preconditions:	**PRE-1** The freelancer is logged in to the system. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A paginated list of bookmarked jobs is displayed. **POST-2** The freelancer can click a job to view its details or remove the bookmark.		

Normal Flow:	**A. View List Bookmarked Job** 1. Freelancer accesses the "Bookmarked Jobs" page. 2. The system receives the request with default parameters (page 1, sorting by bookmark created_at DESC). 3. The system queries the database to retrieve the bookmark list from the bookmarks table for the current freelancer. 4. The system joins with the jobs table to fetch job details. 5. The system calculates total records and total pages. 6. The system returns the first page of bookmarks with pagination information. 7. The system displays: job title, budget, client name, and bookmark date. 8. Freelancer views the bookmark list.
Alternative Flows:	**A.3 No Bookmarks Found** 1. The system queries and finds no bookmarks for the freelancer. 2. The system displays: "You have no bookmarked jobs. Start exploring jobs to save your favorites." 3. The use case ends. **A.8a Navigate Between Pages** 1. Freelancer clicks a page number, Next, or Previous button. 2. The system retrieves the corresponding page of bookmarks. 3. The system displays the requested page. 4. Return to step A.7. **A.8b Sort by Different Criteria** 1. Freelancer selects a sorting option (e.g., by budget, by client name). 2. The system reorders the bookmark list accordingly. 3. The system displays the sorted list. 4. Return to step A.7. **A.8c View Job Detail** 1. Freelancer clicks on a job title in the list. 2. The system navigates to the job detail page. 3. The use case ends. **A.8d Remove Bookmark Directly** 1. Freelancer clicks the "Remove Bookmark" button on a job in the list. 2. The system removes the bookmark and updates the displayed list. 3. Return to step A.7.
Exceptions:	**EX-01 Database Connection Failure** 1. The system cannot retrieve bookmark data due to a database connection error. 2. The system displays: "Unable to load bookmarked jobs. Please try again later." 3. The use case ends.

Priority:	Medium
Frequency of Use:	Medium
Business Rules:	**BR-JOB-27:** Only the freelancer can view their own bookmarks. **BR-JOB-28:** Bookmarks shall be sorted by created_at in descending order by default. **BR-JOB-29:** Displayed information includes: job title, budget, client name, bookmark date. **BR-JOB-30:** The system shall support pagination with default page size of 10.
Other Information:	Default page size: 10 bookmarks per page. Sorting options: by bookmark date (default), by budget, by client name.
Assumptions:	The freelancer is logged in and has a valid account. Bookmarks are created only by freelancers.

2.2.3.8 UC-06.7 Bookmark Job

UC ID and Name	UC-06.7 Bookmark Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer clicks the "Bookmark" button on a job detail page or job list.		
Description:	Allows a freelancer to add a job to their bookmark list for easy tracking. The system automatically checks for duplicates and ignores if the job is already bookmarked.		
Preconditions:	**PRE-1** The freelancer is logged in to the system. **PRE-2** The job exists and is active.		
Post-conditions:	**POST-1** The job is added to the freelancer's bookmark list. **POST-2** The bookmark button changes to "Bookmarked". **POST-3** A success message is displayed.		

Normal Flow:	**A. Bookmark Job** 1. Freelancer views a job detail page or job list. 2. Freelancer clicks the "Bookmark" button on the job. 3. The system receives the request with job_id and freelancer_id parameters. 4. The system checks if the bookmark already exists in the bookmarks table. 5. If not exists, the system creates a new bookmark record with: - freelancer_id = current user ID - job_id = selected job ID - created_at = current timestamp 6. The system updates the button state to "Bookmarked". 7. The system displays the message: "Job bookmarked successfully."
Alternative Flows:	**A.4 Bookmark Already Exists** 1. The system detects that a bookmark already exists for this freelancer and job. 2. The system skips creating a new record. 3. The system displays the message: "This job is already bookmarked." 4. The use case ends.
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Database Connection Failure** 1. The system cannot create the bookmark due to a database error. 2. The system displays: "Unable to bookmark job. Please try again later." 3. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	**BR-JOB-31:** Only freelancers can bookmark jobs. **BR-JOB-32:** Each freelancer can bookmark a job only once (no duplicates). **BR-JOB-33:** If a bookmark already exists, the system silently ignores the duplicate. **BR-JOB-34:** Bookmarks can only be created for active jobs (status = 'open' or 'in_progress').
Other Information:	N/A
Assumptions:	The freelancer is logged in and has a valid account. The job exists and is active.

2.2.3.9 UC-06.8 Remove Bookmark

UC ID and Name	UC-06.8 Remove Bookmark		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer clicks the "Remove Bookmark" or "Unbookmark" button on a job detail page or bookmark list.		
Description:	Allows a freelancer to remove a job from their bookmark list. The system checks if the bookmark exists before deleting and returns a 404 error if not found.		
Preconditions:	**PRE-1** The freelancer is logged in to the system. **PRE-2** The job is bookmarked by this freelancer.		
Post-conditions:	**POST-1** The bookmark is deleted from the bookmarks table. **POST-2** The button state changes to "Bookmark". **POST-3** A success message is displayed.		
Normal Flow:	**A. Remove Bookmark** 1. Freelancer views a job detail page or bookmark list. 2. Freelancer clicks the "Remove Bookmark" button. 3. The system receives the request with job_id and freelancer_id parameters. 4. The system checks if the bookmark exists in the bookmarks table. 5. If exists, the system deletes the bookmark record. 6. The system updates the button state to "Bookmark". 7. The system displays the message: "Bookmark removed successfully."		
Alternative Flows:	**A.1 Remove from Bookmark List** 1. Freelancer clicks the remove button on a bookmark in the bookmark list page. 2. The system performs the same deletion flow as the normal flow. 3. The system updates the bookmark list by removing the deleted item. 4. Return to step A.1.		
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Bookmark Not Found** 1. The system cannot find the bookmark record for the given freelancer and job. 2. The system displays: "Bookmark not found." 3. The use case ends.		

	EX-03 Database Connection Failure 1. The system cannot delete the bookmark due to a database error. 2. The system displays: "Unable to remove bookmark. Please try again later." 3. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	**BR-JOB-35:** Only the freelancer can remove their own bookmarks. **BR-JOB-36:** The system must check if the bookmark exists before deletion. **BR-JOB-37:** If the bookmark does not exist, the system shall return a 404 Not Found error. **BR-JOB-38:** Removing a bookmark is a hard delete from the database.
Other Information:	Returns 404 error if bookmark does not exist.
Assumptions:	The freelancer is logged in and has a valid account. Bookmarks are deleted only by their owners.

2.2.7 Manage Contract

2.2.3.1 UC-09.1 Get Contract List

UC ID and Name	UC-09.1 Get Contract List		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user navigates to the Contracts section from the dashboard or navigation menu.		
Description:	As a Client, Freelancer, or Admin, I want to retrieve a paginated list of contracts with status filters and sorting so that I can easily find and manage my contracts.		
Preconditions:	**PRE-1** The user is successfully logged into the system. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** The system displays a paginated list of contracts relevant to the user's role.		

Normal Flow:	**A. View Contract List** 1. User navigates to the Contract List page. 2. The system retrieves the contracts associated with the user's account (Admin retrieves all). 3. The system applies default pagination and sorting (e.g., newest first). 4. The system displays the list of contracts.
Alternative Flows:	**A.4 Filter and Sort Contracts** 1. User selects a specific status filter (e.g., Pending, Active, Completed) or sorting option. 2. The system retrieves the updated list of contracts based on the selected criteria. 3. The system displays the filtered and sorted list.
Exceptions:	**EX-01: No Contracts Found** 1. If the user has no associated contracts, the system displays the message: "No contracts found." 2. The use case ends. **EX-02: Database Error** 1. If the system fails to fetch data, it displays: "Unable to load contracts. Please try again." 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONTRACT-01: Clients and Freelancers can only view contracts they are a party to. BR-CONTRACT-02: Admins can view all contracts in the system. BR-CONTRACT-03: The list must be paginated to optimize performance.
Other Information:	N/A
Assumptions:	Users have a basic understanding of using filters and sorting tools.

2.2.3.2 UC-09.2 Get Contract Detail

UC ID and Name	UC-09.2 Get Contract Detail		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A

Trigger:	When the user clicks on a specific contract from the Contract List.
Description:	As a Client, Freelancer, or Admin, I want to view full contract details including terms, parties, status, and milestones so that I can track the agreement and its progress.
Preconditions:	**PRE-1** The user is logged in. **PRE-2** The specific contract exists in the database.
Post-conditions:	**POST-1** The system displays all detailed information of the selected contract.
Normal Flow:	**A. View Contract Details** 1. User clicks on a contract item from the Contract List. 2. The system verifies the user's permission to view the contract. 3. The system retrieves contract details by contract ID (including terms, parties, status, and milestones). 4. The system navigates to the Contract Detail page and displays the information.
Alternative Flows:	N/A
Exceptions:	**EX-01: Unauthorized Access** 1. If the user is not a party to the contract (and not an Admin), the system displays: "Access Denied: You do not have permission to view this contract." 2. The use case ends. **EX-02: Contract Not Found** 1. If the contract ID is invalid or deleted, the system displays: "Contract not found or has been removed." 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONTRACT-04: Full terms, current status, and milestone progress must be clearly visible. BR-CONTRACT-05: Confidentiality of contract terms must be maintained via strict access control.
Other Information:	N/A
Assumptions:	Contracts contain at least one milestone or term.

2.2.3.3 UC-09.3 Sign Contract

UC ID and Name	UC-09.3 Sign Contract		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY

Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the "Sign" or "Accept" button on the Contract Detail page.		
Description:	As a Client or Freelancer, I want to digitally sign and accept a contract so that the agreement is formalized and the project work can officially begin.		
Preconditions:	**PRE-1** The user is logged into the system. **PRE-2** The contract is currently in a "Pending Signature" status.		
Post-conditions:	**POST-1** The contract status is updated to reflect the signature (e.g., "Active" if both parties signed). **POST-2** The system records the timestamp and identity of the signer.		
Normal Flow:	**A. Sign Contract** 1. User navigates to the Contract Detail page of a pending contract. 2. User clicks the "Sign Contract" button. 3. The system displays a confirmation popup for the digital signature. 4. User confirms and submits the signature. 5. The system validates the input and updates the contract status in the database. 6. The system displays a success message and notifies the other party.		
Alternative Flows:	**A.4 Cancel Signature** 1. At the confirmation popup, the user clicks "Cancel". 2. The system closes the popup without making any changes. 3. Return to step A.1.		
Exceptions:	**EX-01: Database Update Failure** 1. If the system cannot save the signature due to a database error, it displays: “An error occurred while signing the contract. Please try again.” 2. The use case ends. **EX-02: Contract Already Signed/Expired** 1. If the contract is no longer pending, the system displays: “This contract cannot be signed as it is already active or expired.” 2. The use case ends.		
Priority:	High		

Frequency of Use:	Medium
Business Rules:	BR-CONTRACT-06: A contract becomes "Active" only after all required parties have successfully signed it. BR-CONTRACT-07: The system must store an unalterable log of the signature for legal and audit purposes.
Other Information:	N/A
Assumptions:	Digital signature implies standard user authentication and consent, not necessarily a cryptographic certificate.

2.2.3.4 UC-09.4 Complete Contract

UC ID and Name	UC-09.4 Complete Contract		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the "Mark as Completed" button on the Contract Detail page.		
Description:	As a Client or Freelancer, I want to mark a contract as completed successfully so that the engagement is officially closed and final steps (like reviews) can proceed.		
Preconditions:	**PRE-1** The user is logged into the system. **PRE-2** The contract is in an "Active" status. **PRE-3** All milestones or deliverables associated with the contract have been approved.		
Post-conditions:	**POST-1** The contract status is updated to "Completed". **POST-2** The other party is notified of the contract completion.		
Normal Flow:	**A. Complete Contract** 1. User navigates to the Contract Detail page of an active contract. 2. User clicks the "Complete Contract" button. 3. The system displays a confirmation dialog asking the user to confirm completion. 4. User clicks "Confirm". 5. The system updates the contract status to "Completed" in the database. 6. The system displays a success message and sends a notification to the other party.		

Alternative Flows:	**A.4 Cancel Completion** 1. At the confirmation popup, the user clicks "Cancel". 2. The system closes the popup and no changes are made. 3. Return to step A.1.
Exceptions:	**EX-01: Unmet Milestones** 1. If there are still pending milestones or unresolved disputes, the system displays: “Cannot complete contract. Please ensure all milestones are approved.” 2. The use case ends. **EX-02: System Error** 1. If the system encounters an error while updating the status, it displays: “Failed to mark contract as completed. Please try again.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-CONTRACT-08: A contract can only transition to "Completed" if its current status is "Active". BR-CONTRACT-09: Completion actions must be logged in the Contract Activities history.
Other Information:	N/A
Assumptions:	Both parties agree on the completion parameters off-system or through the milestone approvals before triggering this action.

2.2.3.5 UC-09.5 Cancel Contract

UC ID and Name	UC-09.5 Cancel Contract		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the "Cancel Contract" button on the Contract Detail page.		
Description:	As a Client or Freelancer, I want to cancel a contract before completion so that I can terminate the agreement if issues arise or requirements change.		
Preconditions:	**PRE-1** The user is logged into the system. **PRE-2** The contract is in an "Active" or "Pending" status (not already completed or canceled).		

Post-conditions:	**POST-1** The contract status is updated to "Canceled". **POST-2** The system records the cancellation reason and notifies the other party.
Normal Flow:	**A. Cancel Contract** 1. User navigates to the Contract Detail page. 2. User clicks the "Cancel Contract" button. 3. The system displays a confirmation dialog asking for a cancellation reason. 4. User enters the reason and clicks "Confirm Cancellation". 5. The system updates the contract status to "Canceled" in the database. 6. The system displays a success message and sends a notification to the other party.
Alternative Flows:	**A.4 Abort Cancellation** 1. At the confirmation dialog, the user clicks "Close" or "Back". 2. The system closes the dialog without making any changes. 3. Return to step A.1.
Exceptions:	**EX-01: Invalid Contract Status** 1. If the contract is already completed, the system displays: “This contract has already been completed and cannot be canceled.” 2. The use case ends. **EX-02: Missing Reason** 1. If the user submits without entering a mandatory cancellation reason, the system displays: “Please provide a reason for cancellation.” 2. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-CONTRACT-10: A canceled contract cannot be reactivated; a new contract must be created instead. BR-CONTRACT-11: Any escrowed funds or pending payments must be handled according to the platform’s cancellation policy.
Other Information:	N/A
Assumptions:	Financial settlements for canceled contracts are handled by a separate billing or dispute module.

2.2.3.6 UC-09.6 Contract Activities

UC ID and Name	UC-09.6 Contract Activities		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user navigates to the "Activities" or "History" section within a specific Contract Detail page.		
Description:	As a Client, Freelancer, or Admin, I want to track all activities and updates related to a contract so that I have a transparent history of changes, milestones, and status transitions.		
Preconditions:	**PRE-1** The user is logged into the system. **PRE-2** The user has appropriate permissions to view the contract.		
Post-conditions:	**POST-1** The system displays a chronological list of all events associated with the contract.		
Normal Flow:	**A. View Contract Activities** 1. User navigates to the Contract Detail page. 2. User clicks on the "Activities" or "History" tab. 3. The system queries the database for all logged events related to the specific contract ID. 4. The system displays the activities sorted chronologically (newest first), showing the event type, user responsible, and timestamp.		
Alternative Flows:	N/A		
Exceptions:	**EX-01: Failed to Load Activities** 1. If the system cannot retrieve the activity logs due to a database error, it displays: “Unable to load contract activities. Please refresh the page.” 2. The use case ends.		
Priority:	Medium		
Frequency of Use:	High		
Business Rules:	BR-CONTRACT-12: The activity log is strictly append-only; users cannot edit or delete historical records. BR-CONTRACT-13: Logged events must include status changes, file uploads, milestone updates, and contract signatures.		
Other Information:	N/A		
Assumptions:	The system automatically records a background audit log for every critical action performed on a contract.		

2.2.3.7 UC-09.7 Contract Files

UC ID and Name	UC-09.7 Contract Files		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user navigates to the "Files" or "Attachments" section within a specific Contract Detail page.		
Description:	As a Client or Freelancer, I want to manage files and attachments associated with a contract so that I can easily access deliverables, requirements, and reference documents.		
Preconditions:	**PRE-1** The user is logged into the system. **PRE-2** The user is a participating party in the specific contract.		
Post-conditions:	**POST-1** The system displays a list of all files attached to the contract.		
Normal Flow:	**A. View Contract Files** 1. User navigates to the Contract Detail page. 2. User clicks on the "Files" tab. 3. The system queries the database for all active files related to the specific contract ID. 4. The system displays the list of files, including file name, uploader, size, and upload date.		
Alternative Flows:	**A.4 Download File** 1. User clicks the "Download" icon next to a specific file. 2. The system retrieves the file from the storage server. 3. The system initiates the download to the user's local device.		
Exceptions:	**EX-01: Failed to Load Files** 1. If the system cannot retrieve the file list due to a server error, it displays: "Unable to load contract files. Please try again." 2. The use case ends. **EX-02: File Not Found** 1. If the user attempts to download a file that has been moved or deleted from the server, the system displays: "This file is no longer available." 2. The use case ends.		
Priority:	Medium		
Frequency of Use:	High		
Business Rules:	BR-CONTRACT-14: Only parties directly involved in the contract (or Admins) can view and download associated files.		
Other Information:	N/A		

Assumptions:	Files are stored securely in a cloud storage environment (e.g., AWS S3).

2.2.3.8 UC-09.8 Upload Contract File

UC ID and Name	UC-09.8 Upload Contract File		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the "Upload File" button within the Contract Files section.		
Description:	As a Client or Freelancer, I want to upload files to a contract (e.g., deliverables, documents) so that I can share necessary project resources with the other party.		
Preconditions:	**PRE-1** The user is logged into the system and is on the Contract Files page. **PRE-2** The contract is in a status that permits file uploads (e.g., "Active" or "Pending").		
Post-conditions:	**POST-1** The uploaded file is securely saved and linked to the contract. **POST-2** A system activity log is created, and the other party is notified.		
Normal Flow:	**A. Upload File** 1. User clicks the "Upload File" button. 2. The system prompts the user to select a file from their local device. 3. User selects the file(s) and clicks "Open" or "Submit". 4. The system validates the file size and format. 5. The system uploads the file to the cloud storage and saves the metadata in the database. 6. The system refreshes the file list, displays a success message, and notifies the other party.		
Alternative Flows:	**A.2 Cancel Upload** 1. At the file selection prompt, the user clicks "Cancel". 2. The system closes the prompt without initiating an upload. 3. Return to step A.1.		

Exceptions:	**EX-01: Invalid File Format or Size** 1. If the file exceeds the maximum size limit or is of a restricted format, the system displays: “Invalid file format or file size exceeds the maximum limit.” 2. The use case ends. **EX-02: Upload Failure** 1. If the server connection is lost during upload, the system displays: “File upload failed. Please check your connection and try again.” 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONTRACT-15: Maximum file size allowed for upload is 50MB per file. BR-CONTRACT-16: Executable file types (e.g., .exe, .bat, .sh) are strictly prohibited for security reasons.
Other Information:	N/A
Assumptions:	The system automatically scans all uploaded files for malware or viruses.

2.2.3.9 UC-09.9 Delete Contract File

UC ID and Name	UC-09.9 Delete Contract File		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the "Delete" icon or button next to a specific file in the Contract Files section.		
Description:	As a Client or Freelancer, I want to delete files from a contract so that I can remove outdated, incorrect, or irrelevant documents from the workspace.		
Preconditions:	**PRE-1** The user is logged into the system and is on the Contract Files page. **PRE-2** The user has the necessary permissions to delete the file (e.g., they are the original uploader).		

Post-conditions:	**POST-1** The selected file is permanently removed from the system storage and database. **POST-2** A deletion event is recorded in the Contract Activities log.
Normal Flow:	**A. Delete File** 1. User clicks the "Delete" icon next to the target file. 2. The system displays a confirmation dialog asking: "Are you sure you want to delete this file?" 3. User clicks "Confirm". 4. The system removes the file from cloud storage and deletes its record from the database. 5. The system refreshes the file list and displays a success message.
Alternative Flows:	**A.3 Cancel Deletion** 1. At the confirmation dialog, the user clicks "Cancel". 2. The system closes the dialog without making any changes. 3. Return to step A.1.
Exceptions:	**EX-01: File Not Found** 1. If the file has already been deleted by another user or moved, the system displays: “File not found. It may have already been deleted.” 2. The use case ends. **EX-02: Unauthorized Deletion** 1. If the user attempts to delete a file they do not own (and lacking admin override), the system displays: “You do not have permission to delete this file.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-CONTRACT-17: Users can only delete files they uploaded themselves, unless they have administrative privileges. BR-CONTRACT-18: Deleting a file does not remove its historical trace from the Activity Log.
Other Information:	N/A
Assumptions:	Deleted files cannot be recovered once removed from the cloud storage.

2.2.3.10 UC-09.10 Update Contract Terms

UC ID and Name	UC-09.10 Update Contract Terms		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a Client clicks the "Edit Terms" button on a pending Contract Detail page.		
Description:	As a Client, I want to update contract terms and conditions so that I can modify the scope, timeline, milestones, or payment details before the agreement is finalized.		
Preconditions:	**PRE-1** The user is logged in as a Client. **PRE-2** The contract is currently in a "Pending" status (not yet active or completed).		
Post-conditions:	**POST-1** The contract terms are updated in the database. **POST-2** If the Freelancer had previously signed, their signature is revoked, and they are notified of the updated terms.		
Normal Flow:	**A. Update Contract Terms** 1. Client navigates to the Contract Detail page of a pending contract. 2. Client clicks the "Edit Terms" button. 3. The system displays a form pre-filled with the current contract terms and conditions. 4. Client modifies the necessary fields (e.g., deadlines, milestones, descriptions) and clicks "Save Updates". 5. The system validates the input data. 6. The system updates the contract record in the database. 7. The system logs the change, displays a success message, and notifies the Freelancer to review the new terms.		
Alternative Flows:	**A.4 Cancel Update** 1. While on the edit form, the Client clicks "Cancel". 2. The system discards all unsaved changes. 3. Return to step A.1.		

Exceptions:	**EX-01: Contract Status Invalid** 1. If the contract is already "Active" or "Completed", the system disables the "Edit Terms" button or displays: “Active contracts cannot be edited. Please propose a formal amendment instead.” 2. The use case ends. **EX-02: Validation Error** 1. If the Client leaves mandatory fields blank or inputs invalid dates, the system highlights the errors and displays: “Please correct the highlighted fields before saving.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-CONTRACT-19: Only Clients (the contract creators) have the permission to update the core terms of a pending contract. BR-CONTRACT-20: Any modification to the terms automatically invalidates previous signatures on the contract, requiring re-signing by all parties.
Other Information:	N/A
Assumptions:	The Freelancer will review the updated terms via the standard "Sign Contract" process (UC-09.3).

2.2.3.11 UC-09.11 Create Contract

UC ID and Name	UC-09.11 Create Contract		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a Client clicks the "Create Contract" or "Hire" button from a Freelancer’s profile or an accepted proposal.		
Description:	As a Client, I want to create and initiate a new contract with a freelancer so that we can formalize the work agreement, milestones, and payment terms.		
Preconditions:	**PRE-1** The user is logged in as a Client. **PRE-2** The target Freelancer exists in the system and is available for work.		

Post-conditions:	**POST-1** A new contract record is created in the database with a "Pending" status. **POST-2** The Freelancer receives a notification to review and sign the contract.
Normal Flow:	**A. Create New Contract** 1. Client clicks "Create Contract". 2. The system displays the contract creation form. 3. Client fills in the required details (title, description, timeline, milestones, and payment amount). 4. Client clicks "Submit Contract". 5. The system validates the entered data. 6. The system saves the new contract with a "Pending" status in the database. 7. The system redirects the Client to the Contract Detail page, displays a success message, and notifies the Freelancer.
Alternative Flows:	**A.4 Save as Draft** 1. Client clicks "Save Draft" instead of "Submit Contract". 2. The system saves the entered information as a draft contract without notifying the Freelancer. 3. Return to step A.1.
Exceptions:	**EX-01: Validation Error** 1. If required fields are missing or dates are invalid, the system highlights the errors and displays: “Please fill in all required fields correctly.” 2. The use case ends. **EX-02: System Error** 1. If the system cannot create the contract due to a server issue, it displays: “An error occurred while creating the contract. Please try again.” 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONTRACT-21: Only Clients can initiate and create a new contract on the platform. BR-CONTRACT-22: The total payment amount must be greater than zero and align with the sum of all milestones.
Other Information:	N/A

Assumptions:	The Client and Freelancer have likely discussed project details prior to the formal contract creation.

2.2.3.12 UC-09.12 Progress Update

UC ID and Name	UC-09.12 Progress Update		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a Freelancer clicks "Submit Update" or "Update Progress" on an active contract's milestone.		
Description:	As a Freelancer, I want to submit progress updates on contract milestones so that the Client can track the work and approve completed stages.		
Preconditions:	**PRE-1** The user is logged in. **PRE-2** The contract is in an "Active" status.		
Post-conditions:	**POST-1** The progress update is recorded in the contract history. **POST-2** The Client is notified of the update.		
Normal Flow:	**A. Submit Progress Update** 1. Freelancer navigates to the specific Contract Detail page. 2. Freelancer selects a specific milestone and clicks "Update Progress". 3. The system displays a form to enter comments and attach proof of work (files/links). 4. Freelancer enters the details and clicks "Submit Update". 5. The system validates the input. 6. The system records the update in the database and links it to the milestone. 7. The system displays a success message, refreshes the progress tracker, and notifies the Client.		
Alternative Flows:	**B. Review Progress Update (Client)** 1. Client receives notification and navigates to the Contract Detail page. 2. Client views the submitted progress update and attached files. 3. Client can leave a comment or mark the milestone as approved.		

Exceptions:	**EX-01: Milestone Already Completed** 1. If the selected milestone has already been approved and closed, the system displays: “This milestone is already completed. Updates are no longer accepted.” 2. The use case ends. **EX-02: Missing Information** 1. If the update form is completely blank, the system displays: “Please provide a comment or attach a file to submit an update.” 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONTRACT-23: Only the assigned Freelancer can submit progress updates for their tasks. BR-CONTRACT-24: Progress updates must be time-stamped and permanently logged in the Contract Activities.
Other Information:	N/A
Assumptions:	Milestones are clearly defined during contract creation (UC-09.11) to allow for accurate progress tracking.

2.2.8 Manage Forum

2.2.3.1 UC-04.1 View Forum Categories

UC ID and Name	UC-04.1 View Forum Categories		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When any user accesses the Forum section from the navigation menu.		
Description:	As a guest or registered user, I want to view all active forum categories so that I can browse discussion topics and select a category to explore forum posts.		
Preconditions:	**PRE-1** The system is connected to the database. **PRE-2** At least one active (non-deleted) category exists in the ForumCategory table.		

Post-conditions:	**POST-1** The list of active forum categories is displayed with `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, and `postCount`. **POST-2** Users can select a category to view its details and related posts.
Normal Flow:	**A. View Forum Categories** 1. The User accesses the Forum page from the navigation menu. 2. The system retrieves all active forum categories from the ForumCategory table. 3. The system displays the category name, description, and post count for each active category. 4. The User selects a category from the list. 5. The system navigates the User to the selected category detail page.
Alternative Flows:	**A.1 No Active Categories Found** 1. The User accesses the Forum page. 2. The system determines that no active categories exist in the ForumCategory table. 3. The system displays the message “No forum categories available.” 4. The User remains on the Forum page. 5. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the list of active forum categories from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load forum categories. Please try again later.” 4. The forum category list is not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the forum category data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any category information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-01: Only categories with `deletedAt = NULL` shall be displayed. BR-FORUM-02: Categories shall be ordered by `name` in ascending order. BR-FORUM-03: `postCount` shall count only posts with `deletedAt = NULL` and `moderationStatus = APPROVED`.
Other Information:	The endpoint is public and does not require authentication.
Assumptions:	Forum categories are created and maintained by administrators.

2.2.3.2 UC-04.2 View Forum Category Detail

UC ID and Name	UC-04.2 View Forum Category Detail		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user selects a forum category from the forum category list.		
Description:	Allows users to view detailed information about a selected forum category including its name, slug, description, and creation date.		

Preconditions:	**PRE-1** The user is viewing the forum category list (UC-04.1). **PRE-2** The selected category exists and has `deletedAt = NULL`.
Post-conditions:	**POST-1** The selected category information is displayed successfully including `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, and `postCount`. **POST-2** The user can continue to browse posts belonging to the selected category.
Normal Flow:	**A. View Forum Category Detail** 1. The User selects a forum category from the Forum Categories page. 2. The system validates that the selected category exists and is active. 3. The system retrieves the category information including its name, slug, description, and creation date. 4. The system displays the category details. 5. The system renders the post list page for the selected category.
Alternative Flows:	**A.1 Selected Category Is No Longer Available** 1. The User selects a forum category. 2. The system determines that the selected category has been deleted or is not found. 3. The system displays the message “The selected category is no longer available.” 4. The system redirects the User to the Forum Categories page. 5. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the selected forum category from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load category details. Please try again later.” 4. The category details are not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested category information. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any category information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-04: Only categories with `deletedAt = NULL` shall be accessible. BR-FORUM-05: Category information displayed includes `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, and `postCount`.
Other Information:	The endpoint is public and does not require authentication.
Assumptions:	Forum categories are created and maintained by administrators.

2.2.3.3 UC-04.3 View Post List

UC ID and Name	UC-04.3 View Post List		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user opens a forum category to browse discussion posts.		
Description:	Allows users to view a paginated list of forum posts within a selected category. The list includes post titles, authors, creation dates, like counts, and comment counts.		

Preconditions:	**PRE-1** The selected category exists and is active (`deletedAt = NULL`). **PRE-2** The system is connected to the database.
Post-conditions:	**POST-1** A paginated list of forum posts is displayed with pagination metadata `{ page, limit, total, totalPages }`. **POST-2** Users can select a post to view its details.
Normal Flow:	**A. View Post List** 1. The User opens a forum category from the Forum Categories page. 2. The system validates that the selected category exists and is active. 3. The system retrieves posts associated with the selected category from the ForumPost table, excluding deleted posts and posts that are not approved. 4. The system optionally filters posts by user or search term, and applies pagination. 5. The system displays the post titles, authors, creation dates, like counts, and comment counts. 6. The User selects a post to view its details.
Alternative Flows:	**A.1 No Posts Found** 1. The User opens a forum category. 2. The system determines that no posts exist in the selected category. 3. The system displays the message "No discussion posts available in this category." 4. The User remains on the category page. 5. Return to step 1. **A.5 Navigate Between Pages** 1. The User clicks a page number, "Next," or "Previous" button, or changes the page size. 2. The system retrieves the requested page of posts from the ForumPost table. 3. The system displays the requested page of posts. 4. Return to step 5.

Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the list of posts for the selected category from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load forum posts. Please try again later.” 4. The post list is not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested post data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any post information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-06: Only posts with `deletedAt = NULL` and `moderationStatus = APPROVED` shall be displayed. BR-FORUM-07: Posts shall be sorted by `createdAt` in descending order. BR-FORUM-08: The system shall support pagination and configurable page sizes. BR-FORUM-09: `commentCount` shall count only comments with `deletedAt = NULL`.
Other Information:	Default page size is 10. A search bar and “My Posts” toggle (filter by user) are available on the frontend.
Assumptions:	Forum posts are associated with valid categories and authors.

2.2.3.4 UC-04.4 View Post Detail

UC ID and Name	UC-04.4 View Post Detail		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user selects a post from the post list.		

Description:	Allows users to view the complete details of a forum post including its title, content, author, category, and creation date.
Preconditions:	**PRE-1** The selected post exists. **PRE-2** The post has not been soft-deleted and has `moderationStatus = APPROVED`.
Post-conditions:	**POST-1** The complete post information is displayed including nested category and user profile. **POST-2** Users can view related comments and reactions associated with the post.
Normal Flow:	**A. View Post Detail** 1. The User selects a post from the post list. 2. The system validates that the selected post exists, is not deleted, and is approved. 3. The system retrieves the post information, author information, and category information. 4. The system displays the post detail page with the title, content, author, category, and creation date. 5. The system displays the available comments and reaction count.
Alternative Flows:	**A.1a Selected Post Is No Longer Available (soft-deleted)** 1. The User selects a post. 2. The system determines that the selected post has been soft-deleted. 3. The system displays the message “This post is no longer available.” 4. The system redirects the User to the previous post list page. 5. Return to step 1. **A.1b Selected Post Not Found** 1. The User selects a post. 2. The system determines that the selected post does not exist or is not yet approved. 3. The system displays the message “Post not found.” 4. The system redirects the User to the previous post list page. 5. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the post, author, and category information from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load post details. Please try again later.” 4. The post details are not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested post information. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any post information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-10: Only posts with `deletedAt = NULL` and `moderationStatus = APPROVED` shall be displayed. BR-FORUM-11: Post details shall include title, content, author, category, and created date.
Other Information:	Comments and reactions are displayed together with the post details.
Assumptions:	Each post is associated with a valid author and category.

2.2.3.5 UC-04.5 Create Post

UC ID and Name	UC-04.5 Create Post		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Create Post” button in the Forum section.		
Description:	Allows authenticated users to create a new forum post by providing a title, content, and optional category.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** At least one active forum category exists (optional for assignment).
Post-conditions:	**POST-1** A new post is created in the ForumPost table. **POST-2** The new post appears in the selected category after being approved (if moderation applies).
Normal Flow:	**A. Create Post** 1. The User clicks the “Create Post” button in the Forum section. 2. The system verifies that the User is authenticated. 3. The system displays the Create Post dialog. 4. The User enters the post title and content, and optionally selects a forum category. 5. The User clicks the “Publish” button. 6. The system validates the input data. 7. The system creates a new post in the ForumPost table associated with the authenticated User. 8. If the new post is approved, the system redirects the User to the newly created post detail page; otherwise it shows a message that the post is awaiting admin review.

Alternative Flows:	**A.3 User Not Authenticated** 1. The User clicks the “Create Post” button. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the Create Post dialog. 6. Return to step 3. **A.5 User Cancels Creation** 1. The User clicks the “Cancel” button on the Create Post dialog. 2. The system discards the unsaved data. 3. The system returns the User to the Forum page. 4. The use case ends. **A.4a Required Fields Missing** 1. The User clicks the “Publish” button. 2. The system determines that the title or content is empty. 3. The system displays the message “Please complete all required fields.” 4. Return to step 4. **A.4b Invalid Category Selected** 1. The User submits the post. 2. The system determines that the selected category does not exist. 3. The system displays the message “Invalid category selected.” 4. Return to step 4. **A.8 Post Awaits Moderation** 1. The User submits the post. 2. The post moderation status is PENDING. 3. The system displays the message “Your post has been saved and is awaiting admin review.” 4. The post is not displayed publicly until approved. 5. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to save the new post to the ForumPost table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to create post. Please try again later.” 4. The post is not created. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while submitting the post. 2. The system cannot save the post data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the Create Post dialog without the post being saved. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-12: Only authenticated users can create posts. BR-FORUM-13: Every post must belong to exactly one user and may belong to a category. BR-FORUM-14: Title and content are required fields. BR-FORUM-15: A new post shall default to `moderationStatus = PENDING` unless moderation data is provided.
Other Information:	The post creator is automatically assigned from the currently authenticated user account.
Assumptions:	Forum categories are available and maintained by administrators.

2.2.3.6 UC-04.6 Edit Post

UC ID and Name	UC-04.6 Edit Post		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When the author selects the “Edit Post” option on their post.		
Description:	Allows the author of a post to modify its title, content, or category.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`. **PRE-3** The user is the author of the selected post.
Post-conditions:	**POST-1** The post information is updated successfully, including a regenerated slug when the title changes. **POST-2** The updated post is displayed to users.
Normal Flow:	**A. Edit Post** 1. The User selects the "Edit Post" option on their post. 2. The system verifies that the User is the author of the selected post. 3. The system displays the existing post information in an editable dialog. 4. The User modifies the title, content, or category. 5. The User clicks the "Save Changes" button. 6. The system validates the updated information. 7. The system updates the post record in the ForumPost table, regenerating the slug if the title changed. 8. The system displays the updated post details.
Alternative Flows:	**A.1 User Is Not the Author** 1. The User selects the "Edit Post" option on a post. 2. The system determines that the User is not the author of the selected post. 3. The system displays the message "You are not authorized to edit this post." 4. Return to step 1. **A.5 User Cancels Editing** 1. The User clicks the "Cancel" button on the edit dialog. 2. The system discards the unsaved changes. 3. The system returns the User to the post detail page. 4. The use case ends. **A.4 Required Fields Missing** 1. The User clicks the "Save Changes" button. 2. The system determines that the title or content is empty. 3. The system displays the message "Please complete all required fields." 4. Return to step 4.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the post record in the ForumPost table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to update post. Please try again later.” 4. The post is not updated. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while saving the changes. 2. The system cannot save the updated post data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the edit dialog without the changes being saved. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-FORUM-16: Only the author of a post can edit that post. BR-FORUM-17: Title and content are required fields. BR-FORUM-18: The original post ID remains unchanged after editing; the slug is regenerated if the title changes.
Other Information:	Editing does not trigger re-moderation.
Assumptions:	The user remains authenticated throughout the editing process.

2.2.3.7 UC-04.7 Delete Post

UC ID and Name	UC-04.7 Delete Post		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When the author or an Admin selects the “Delete Post” option on a post.		
Description:	Allows the author of a post or an Admin to remove it from public view by performing a soft delete operation.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`. **PRE-3** The user is the author of the selected post or has the Admin role.
Post-conditions:	**POST-1** The post is soft-deleted by setting the `deletedAt` field. **POST-2** The post is no longer displayed in forum post listings.
Normal Flow:	**A. Delete Post** 1. The User selects the "Delete Post" option on a post. 2. The system verifies that the User is the author of the selected post or has the Admin role. 3. The system displays a confirmation message. 4. The User confirms the deletion request. 5. The system performs a soft delete by updating the `deletedAt` field of the selected post. 6. The system removes the post from public forum listings. 7. The system redirects the User to the category post list page.
Alternative Flows:	**A.1a User Not Authorized** 1. The User selects the "Delete Post" option on a post. 2. The system determines that the User is not the author nor an Admin. 3. The system displays the message "You are not authorized to delete this post." 4. Return to step 1. **A.4 User Cancels Deletion** 1. The User selects the "Cancel" option on the confirmation message. 2. The system cancels the deletion process. 3. The system returns the User to the post detail page. 4. The use case ends. **A.1b Selected Post Not Found or Already Deleted** 1. The User confirms the deletion request. 2. The system determines that the selected post does not exist or is already soft-deleted. 3. The system displays the message "Post not found." 4. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the `deletedAt` field of the selected post in the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to delete post. Please try again later.” 4. The post is not deleted. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the deletion request. 2. The system cannot complete the deletion of the post. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The post remains unchanged. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-FORUM-19: Only the author of a post or an Admin can delete that post. BR-FORUM-20: Post deletion shall be implemented as a soft delete using the `deletedAt` field. BR-FORUM-21: Soft-deleted posts shall not appear in public forum listings.
Other Information:	Deleted posts remain stored in the database and can be restored by administrators.
Assumptions:	The user remains authenticated during the deletion process.

2.2.3.8 UC-04.8 Like Post

UC ID and Name	UC-04.8 Like Post		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Like” button on a forum post.		
Description:	Allows authenticated users to express appreciation for a post by adding a like reaction (toggle action).		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`.
Post-conditions:	**POST-1** A like record is created in the ForumLike table associated with the user and post. **POST-2** The post's reaction count is updated and the "Like" button is highlighted.
Normal Flow:	**A. Like Post** 1. The User clicks the "Like" button on a post. 2. The system verifies that the User is authenticated. 3. The system verifies that the post exists and has not been deleted. 4. The system verifies that the User has not already liked the post. 5. The system creates a new like record associated with the User and the post in the ForumLike table. 6. The system updates the displayed reaction count and highlights the "Like" button.
Alternative Flows:	**A.1a User Not Authenticated** 1. The User clicks the "Like" button. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.1b Selected Post Not Found or Deleted** 1. The User clicks the "Like" button on a post. 2. The system determines that the selected post does not exist or is soft-deleted. 3. The system displays the message "Post not found." 4. Return to step 1. **A.4 Post Already Liked** 1. The User clicks the "Like" button on a post they have already liked. 2. The system determines that the User has already liked the post. 3. The system performs the unlike action (see UC-04.9). 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to create a new like record in the ForumLike table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to process your request. Please try again later.” 4. The like reaction is not saved. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the like request. 2. The system cannot save the reaction. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The reaction count remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-FORUM-22: Only authenticated users can like posts. BR-FORUM-23: A user can have only one active like reaction per post. BR-FORUM-24: The reaction shall be stored in the ForumLike table.
Other Information:	The like action is a toggle and uses optimistic updates on the frontend.
Assumptions:	The selected post is accessible to the user.

2.2.3.9 UC-04.9 Unlike Post

UC ID and Name	UC-04.9 Unlike Post		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks an active “Like” button on a post that they have previously liked.		
Description:	Allows authenticated users to remove their like reaction from a forum post.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`. **PRE-3** The user has previously liked the selected post.
Post-conditions:	**POST-1** The corresponding like record is removed from the ForumLike table. **POST-2** The post's reaction count is updated and the "Like" button is set to its inactive state.
Normal Flow:	**A. Unlike Post** 1. The User clicks the active "Like" button on a post. 2. The system verifies that the User is authenticated. 3. The system verifies the existing like relationship between the User and the post. 4. The system removes the corresponding like record from the ForumLike table. 5. The system updates the displayed reaction count and changes the "Like" button to its inactive state.
Alternative Flows:	**A.1a Selected Post Not Found** 1. The User clicks the "Like" button on a post. 2. The system determines that the selected post does not exist or is soft-deleted. 3. The system displays the message "Post not found." 4. Return to step 1. **A.1b User Not Authenticated** 1. The User attempts to remove a reaction from a post. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.3 Like Record Not Found** 1. The User clicks the "Like" button on a post. 2. The system determines that no like record exists for the User and the selected post. 3. The system treats the action as a like. 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to remove the like record from the ForumLike table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to process your request. Please try again later.” 4. The like reaction is not removed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the unlike request. 2. The system cannot remove the reaction. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The reaction count remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-FORUM-25: Only authenticated users can remove post reactions. BR-FORUM-26: A like reaction can only be removed if a corresponding like record exists. BR-FORUM-27: Removing a reaction shall delete the corresponding record from the ForumLike table.
Other Information:	The like/unlike action is a single toggle endpoint.
Assumptions:	The selected post is accessible to the user.

2.2.3.10 UC-04.10 View Like Post Detail

UC ID and Name	UC-04.10 View Like Post Detail		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks the reaction/like count link on a forum post.		
Description:	Allows users to view the list of people who have liked a specific forum post.		
Preconditions:	**PRE-1** The selected post exists and has `deletedAt = NULL`. **PRE-2** The post detail page is displayed successfully.		

Post-conditions:	**POST-1** The list of likes for the selected post is displayed. **POST-2** Users can view each liker's display name and avatar.
Normal Flow:	**A. View Like Post Detail** 1. The User clicks the reaction count link on the post action bar. 2. The system retrieves all likes for the selected post from the ForumLike table. 3. The system displays a dialog listing all likers with their avatars.
Alternative Flows:	**A.1a No Likes Found** 1. The User clicks the reaction count link. 2. The system determines that no likes exist for the selected post. 3. The system displays the message "No likes yet." 4. The User remains on the post detail page. 5. Return to step 1. **A.1b Selected Post Not Found** 1. The User clicks the reaction count link. 2. The system determines that the selected post does not exist or is soft-deleted. 3. The system displays the message "Post not found." 4. Return to step 1.
Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the list of likes for the post from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message "Unable to load reactions. Please try again later." 4. The like list is not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested like data. 3. The system displays the message "Cannot connect to the server. Please try again later." 4. The User remains on the current page without like information being displayed. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Medium

Business Rules:	BR-FORUM-28: The like detail shall include the user id and user profile information for each liker.
Other Information:	All likes for a post are returned without pagination.
Assumptions:	The selected post is accessible to the user.

2.2.3.11 UC-04.11 View Post Comments

UC ID and Name	UC-04.11 View Post Comments		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user views a forum post detail page (UC-04.4).		
Description:	Allows users to view all non-deleted comments associated with a selected forum post.		
Preconditions:	**PRE-1** The selected post exists and has `deletedAt = NULL`. **PRE-2** The post detail page is displayed successfully.		
Post-conditions:	**POST-1** A paginated list of comments associated with the selected post is displayed. **POST-2** Users can view comment content, authors, creation dates, and like counts.		
Normal Flow:	**A. View Post Comments** 1. The User opens a post detail page (UC-04.4). 2. The system retrieves comments associated with the selected post from the ForumComment table, excluding deleted comments. 3. The system retrieves author information for each comment. 4. The system sorts the comments by creation date and applies pagination. 5. The system displays the comment content, author information, creation date, and like count for each comment.		

Alternative Flows:	**A.1a No Comments Found** 1. The User opens a post detail page. 2. The system determines that no comments exist for the selected post. 3. The system displays the message “No comments available.” 4. The User remains on the post detail page. 5. Return to step 1. **A.1b Associated Post Is No Longer Available** 1. The User attempts to view comments for a post. 2. The system determines that the associated post does not exist or is soft-deleted. 3. The system displays the message “Post not found.” 4. The system redirects the User to the previous page. 5. Return to step 1.
Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the comment list for the selected post from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load comments. Please try again later.” 4. The comment list is not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested comment data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any comment information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-29: Only comments with `deletedAt = NULL` shall be displayed. BR-FORUM-30: Comments shall be sorted by creation date. BR-FORUM-31: For authenticated users, each comment shall indicate whether it has been liked by the current user.

Other Information:	Comments are displayed within the selected post detail page's comment section.
Assumptions:	Each comment is associated with a valid post and author.

2.2.3.12 UC-04.12 Add Comment

UC ID and Name	UC-04.12 Add Comment		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user enters comment content and clicks the "Submit Comment" button.		
Description:	Allows authenticated users to add a new comment to a forum post and participate in community discussions.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`.		
Post-conditions:	**POST-1** A new comment record is inserted into the ForumComment table. **POST-2** The new comment is displayed in the comment list of the post.		
Normal Flow:	**A. Add Comment** 1. The User enters content into the comment input field on a post detail page. 2. The User clicks the "Submit Comment" button. 3. The system validates the comment content. 4. The system creates a new comment record associated with the selected post and the User in the ForumComment table. 5. The system updates the comment list and displays the new comment.		

Alternative Flows:	**A.1a User Not Authenticated** 1. The User attempts to submit a comment. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.1b Empty Comment Content** 1. The User attempts to submit a comment. 2. The system determines that the comment content is empty. 3. The system displays the message “Comment content cannot be empty.” 4. Return to step 1. **A.1c Selected Post Not Found** 1. The User clicks the “Submit Comment” button. 2. The system determines that the selected post does not exist or is soft-deleted. 3. The system displays the message “Post not found.” 4. Return to step 1.
Exceptions:	**EX-01: Database Error** 1. The system attempts to save the new comment to the ForumComment table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to submit comment. Please try again later.” 4. The comment is not created. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while submitting the comment. 2. The system cannot save the comment data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the post detail page without the comment being saved. 5. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	BR-FORUM-32: Only authenticated users can create comments. BR-FORUM-33: Comment content cannot be empty. BR-FORUM-34: Each comment shall be associated with a valid post and user.
Other Information:	Comments are displayed in chronological order based on their creation time.
Assumptions:	The selected post remains available during the comment submission process.

2.2.3.13 UC-04.13 Edit Comment

UC ID and Name	UC-04.13 Edit Comment		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Edit” option on one of their comments.		
Description:	Allows users to modify the content of a comment that they previously created.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected comment exists and has `deletedAt = NULL`. **PRE-3** The user is the owner of the selected comment.		
Post-conditions:	**POST-1** The comment content is updated in the ForumComment table. **POST-2** The updated comment is displayed on the post detail page.		
Normal Flow:	**A. Edit Comment** 1. The User clicks the “Edit” option on their comment. 2. The system verifies that the User is the owner of the selected comment. 3. The system displays the current comment content in an editable form. 4. The User modifies the comment content. 5. The User clicks the “Save Changes” button. 6. The system validates the updated content. 7. The system updates the comment record in the ForumComment table. 8. The system displays the updated comment.		

Alternative Flows:	**A.1a User Not Authorized** 1. The User clicks the “Edit” option on a comment. 2. The system determines that the User is not the owner of the selected comment. 3. The system displays the message “You are not authorized to edit this comment.” 4. Return to step 1. **A.4a Cancel Comment Editing** 1. The User clicks the “Cancel” button on the edit form. 2. The system discards all unsaved changes. 3. The system displays the original comment content. 4. The use case ends. **A.4b Empty Comment Content** 1. The User clicks the “Save Changes” button. 2. The system determines that the updated comment content is empty. 3. The system displays the message “Comment content cannot be empty.” 4. Return to step 4. **A.1b Selected Comment Not Found or Not in Post** 1. The User clicks the “Save Changes” button. 2. The system determines that the selected comment does not exist, is deleted, or does not belong to the post. 3. The system displays the message “Comment not found.” 4. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the comment record in the ForumComment table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to update comment. Please try again later.” 4. The comment is not updated. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while saving the changes. 2. The system cannot save the updated comment data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the edit form without the changes being saved. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-FORUM-35: Only the owner of a comment can edit it. BR-FORUM-36: Comment content cannot be empty. BR-FORUM-37: A comment must belong to the specified post to be edited.
Other Information:	The updated comment remains associated with the original post and author.
Assumptions:	The selected comment has not been deleted or hidden by the system.

2.2.3.14 UC-04.14 Delete Comment

UC ID and Name	UC-04.14 Delete Comment		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks the “Delete” option on one of their comments.		
Description:	Allows the owner of a comment or an Admin to remove it from public view by performing a soft delete operation.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected comment exists and has `deletedAt = NULL`. **PRE-3** The user is the owner of the selected comment or has the Admin role.
Post-conditions:	**POST-1** The selected comment is soft-deleted by setting the `deletedAt` field. **POST-2** The comment is no longer displayed to regular users.
Normal Flow:	**A. Delete Comment** 1. The User clicks the "Delete" option on a comment. 2. The system verifies that the User is the owner of the selected comment or has the Admin role. 3. The system displays a confirmation dialog. 4. The User confirms the deletion request. 5. The system updates the `deletedAt` field of the selected comment. 6. The system removes the comment from the visible comment list and refreshes the comment section.
Alternative Flows:	**A.1a User Not Authorized** 1. The User clicks the "Delete" option on a comment. 2. The system determines that the User is not the owner nor an Admin. 3. The system displays the message "You are not authorized to delete this comment." 4. Return to step 1. **A.4 Cancel Deletion** 1. The User clicks the "Cancel" button on the confirmation dialog. 2. The system closes the confirmation dialog. 3. The comment remains unchanged. 4. The use case ends. **A.1b Selected Comment Not Found or Not in Post** 1. The User confirms the deletion request. 2. The system determines that the selected comment does not exist, is already deleted, or does not belong to the post. 3. The system displays the message "Comment not found." 4. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the `deletedAt` field of the selected comment in the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to delete comment. Please try again later.” 4. The comment is not deleted. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the deletion request. 2. The system cannot complete the deletion of the comment. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The comment remains unchanged. 5. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-FORUM-38: Only the owner of a comment or an Admin can delete it. BR-FORUM-39: Comment deletion shall be implemented as a soft delete using the `deletedAt` field. BR-FORUM-40: Soft-deleted comments shall not be displayed in normal comment listings.
Other Information:	Deleted comments may remain available for administrative review and auditing purposes.
Assumptions:	The selected comment has not been permanently removed from the database.

2.2.3.15 UC-04.15 Like Comment

UC ID and Name	UC-04.15 Like Comment		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Like” button on a comment.		
Description:	Allows authenticated users to express appreciation for a comment by adding a like reaction (toggle action).		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected comment exists and has `deletedAt = NULL`.
Post-conditions:	**POST-1** A like record is created in the ForumLike table associated with the user and comment. **POST-2** The comment's reaction count is updated and the "Like" button is highlighted.
Normal Flow:	**A. Like Comment** 1. The User clicks the "Like" button on a comment. 2. The system verifies that the User is authenticated. 3. The system verifies that the comment exists and has not been deleted. 4. The system verifies that the User has not already liked the comment. 5. The system creates a new like record associated with the User and the comment in the ForumLike table. 6. The system updates the displayed reaction count and highlights the "Like" button.
Alternative Flows:	**A.1a User Not Authenticated** 1. The User clicks the "Like" button on a comment. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.1b Selected Comment Not Found** 1. The User clicks the "Like" button on a comment. 2. The system determines that the selected comment does not exist, is deleted, or does not belong to the post. 3. The system displays the message "Comment not found." 4. Return to step 1. **A.4 Comment Already Liked** 1. The User clicks the "Like" button on a comment they have already liked. 2. The system determines that the User has already liked the comment. 3. The system performs the unlike action (see UC-04.16). 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to create a new like record in the ForumLike table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to process your request. Please try again later.” 4. The like reaction is not saved. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the like request. 2. The system cannot save the reaction. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The reaction count remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-FORUM-41: Only authenticated users can like comments. BR-FORUM-42: A user can have only one active like reaction per comment. BR-FORUM-43: The reaction shall be stored in the ForumLike table.
Other Information:	The like action is a toggle and uses optimistic updates on the frontend.
Assumptions:	The selected comment is accessible to the user.

2.2.3.16 UC-04.16 Unlike Comment

UC ID and Name	UC-04.16 Unlike Comment		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks an active “Like” button on a comment that they have previously liked.		
Description:	Allows authenticated users to remove their like reaction from a forum comment.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected comment exists and has `deletedAt = NULL`. **PRE-3** The user has previously liked the selected comment.
Post-conditions:	**POST-1** The corresponding like record is removed from the ForumLike table. **POST-2** The comment's reaction count is updated and the "Like" button is set to its inactive state.
Normal Flow:	**A. Unlike Comment** 1. The User clicks the active "Like" button on a comment. 2. The system verifies that the User is authenticated. 3. The system verifies the existing like relationship between the User and the comment. 4. The system removes the corresponding like record from the ForumLike table. 5. The system updates the displayed reaction count and changes the "Like" button to its inactive state.
Alternative Flows:	**A.1a Selected Comment Not Found** 1. The User clicks the "Like" button on a comment. 2. The system determines that the selected comment does not exist or is soft-deleted. 3. The system displays the message "Comment not found." 4. Return to step 1. **A.1b User Not Authenticated** 1. The User attempts to remove a reaction from a comment. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.3 Like Record Not Found** 1. The User clicks the "Like" button on a comment. 2. The system determines that no like record exists for the User and the selected comment. 3. The system treats the action as a like. 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to remove the like record from the ForumLike table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to process your request. Please try again later.” 4. The like reaction is not removed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the unlike request. 2. The system cannot remove the reaction. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The reaction count remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-FORUM-44: Only authenticated users can remove comment reactions. BR-FORUM-45: A like reaction can only be removed if a corresponding like record exists. BR-FORUM-46: Removing a reaction shall delete the corresponding record from the ForumLike table.
Other Information:	The like/unlike action is a single toggle endpoint.
Assumptions:	The selected comment is accessible to the user.

2.2.3.17 UC-04.17 Report Content

UC ID and Name	UC-04.17 Report Content		
Created By:	Pham Kha Vy	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	Admin, Moderator
Trigger:	When a user selects the “Report” option for a forum post or comment.		
Description:	Allows authenticated users to report forum posts or comments that violate community guidelines so that moderators can review and take appropriate actions.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post or comment exists and has `deletedAt = NULL`. **PRE-3** The user has not already reported the selected content.
Post-conditions:	**POST-1** A new report record is created in the ForumReport table. **POST-2** The report status is automatically set to PENDING. **POST-3** Administrators and moderators can review the report.
Normal Flow:	**A. Report Content** 1. The User selects the "Report" option on a post or comment. 2. The system verifies that the User is authenticated. 3. The system displays a report dialog. 4. The system checks whether the User has already reported this content. 5. The User enters a report reason in the dialog. 6. The User clicks the "Submit Report" button. 7. The system validates the submitted information. 8. The system creates a new record in the ForumReport table with status set to PENDING. 9. The system displays a confirmation message indicating that the report has been submitted successfully.

Alternative Flows:	**A.1a User Not Authenticated** 1. The User selects the “Report” option on a post or comment. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected content. 6. Return to step 1. **A.4 User Already Reported (Duplicate Report)** 1. The User selects the “Report” option on a post or comment. 2. The system determines that the User has already reported the same content. 3. The system displays the message “You have already reported this content.” 4. The system shows a read-only report dialog. 5. The use case ends. **A.5a User Cancels Report** 1. The User closes the report dialog without submitting. 2. The system discards all entered information. 3. The system returns the User to the current post or comment page. 4. The use case ends. **A.5b Reason Not Entered** 1. The User clicks the “Submit Report” button. 2. The system determines that no report reason was entered. 3. The system displays the message “Please enter a report reason.” 4. Return to step 5. **A.1b Content Not Found** 1. The User clicks the “Submit Report” button. 2. The system determines that the selected post or comment no longer exists or is soft-deleted. 3. The system displays the message “The selected content is no longer available.” 4. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to save the new report to the ForumReport table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to submit your report. Please try again later.” 4. The report is not created. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while submitting the report. 2. The system cannot save the report data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the report dialog without the report being saved. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-FORUM-47: Only authenticated users can submit reports. BR-FORUM-48: Reports must be associated with either a post or a comment. BR-FORUM-49: New reports shall be created with status PENDING. BR-FORUM-50: A user may submit only one report for the same content item.
Other Information:	Reports are reviewed by administrators who can list and update report status.
Assumptions:	Moderators regularly review pending reports and take appropriate moderation actions.

3 Business Rule

Table 62: Business Rules

ID	Rule Definition
BR-PROF-01	Both basic details (from profiles) and professional details (from freelancer_profiles) must be presented together.
BR-PROF-02	Hidden or private fields should not be displayed to unauthorized users (e.g., clients vs. the profile owner).
BR-PROF-03	Freelancers can only update their own profile.
BR-PROF-04	Mandatory fields (e.g., display_name, title) cannot be left empty.
BR-SKILL-01	All skills belonging to the freelancer must be displayed along with their proficiency level.
BR-SKILL-02	A freelancer cannot add duplicate skills.
BR-SKILL-03	Both skill name and proficiency level are required.
BR-SKILL-04	Skill deletion must be a hard delete operation; the record is permanently removed from the mapping table.
BR-SKILL-05	Only the owning freelancer or an administrator can delete a skill from the profile.
BR-PORT-01	Only portfolios with deleted_at = NULL shall be displayed.
BR-PORT-02	Portfolio title is a required field.
BR-PORT-03	Media uploads must conform to system-defined size and type restrictions.
BR-PORT-04	Only portfolios with deleted_at = NULL can be accessed.
BR-PORT-05	Only the portfolio owner can edit it.
BR-PORT-06	Portfolio title remains a required field.
BR-PORT-07	Portfolio deletion must be a soft delete (updating the deleted_at column).
BR-PORT-08	Soft-deleted portfolios must not be visible to clients or guests.
BR-FORUM-01	Only categories with is_active = TRUE shall be displayed.

ID	Rule Definition
BR-FORUM-02	Category information displayed includes name and description.
BR-FORUM-03	Only categories with is_active = TRUE shall be accessible.
BR-FORUM-04	Category information displayed includes name and description.
BR-FORUM-05	Only posts with deleted_at = NULL shall be displayed.
BR-FORUM-06	Posts with is_pinned = TRUE shall appear before regular posts.
BR-FORUM-07	Posts shall be sorted by created_at in descending order within each group.
BR-FORUM-08	The system shall support pagination for post listings.
BR-FORUM-09	Only posts with deleted_at = NULL shall be displayed.
BR-FORUM-10	Post details shall include title, content, author, category, and created_at information.
BR-FORUM-11	Only comments with deleted_at = NULL shall be displayed.
BR-FORUM-12	Comment information shall include content, author, and created_at.
BR-FORUM-13	Only comments with deleted_at = NULL shall be displayed.
BR-FORUM-14	Comment details shall include content, author, and created_at information.
BR-FORUM-15	Only authenticated users can create posts.
BR-FORUM-16	Every post must belong to exactly one category.
BR-FORUM-17	Title and content are required fields.
BR-FORUM-18	Only the author of a post can edit that post.
BR-FORUM-19	Every post must belong to a valid category.
BR-FORUM-20	Title and content are required fields.
BR-FORUM-21	Only the author of a post can delete that post.
BR-FORUM-22	Post deletion shall be implemented as a soft delete using the deleted_at field.

ID	Rule Definition
BR-FORUM-23	Soft-deleted posts shall not appear in public forum listings.
BR-FORUM-24	Only authenticated users can like posts.
BR-FORUM-25	A user can have only one active like reaction per post.
BR-FORUM-26	The reaction shall be stored in the forum_likes table.
BR-FORUM-27	Only authenticated users can remove post reactions.
BR-FORUM-28	A like reaction can only be removed if a corresponding like record exists.
BR-FORUM-29	Removing a reaction shall delete the corresponding record from the forum_likes table.
BR-FORUM-30	Only authenticated users can create comments.
BR-FORUM-31	Comment content cannot be empty.
BR-FORUM-32	Comment content must contain at least 2 characters.
BR-FORUM-33	Each comment shall be associated with a valid post_id and user_id.
BR-FORUM-34	Only the owner of a comment can edit it.
BR-FORUM-35	Comment content cannot be empty.
BR-FORUM-36	Comment content must contain at least 2 characters.
BR-FORUM-37	The system shall record the updated_at timestamp when a comment is modified.
BR-FORUM-38	Only the owner of a comment can delete it.
BR-FORUM-39	Comment deletion shall be implemented as a soft delete using the deleted_at field.
BR-FORUM-40	Soft-deleted comments shall not be displayed in normal comment listings.
BR-FORUM-41	Only authenticated users can like comments.
BR-FORUM-42	A user can have only one active like reaction per comment.
BR-FORUM-43	The reaction shall be stored in the forum_likes table using comment_id.

ID	Rule Definition
BR-FORUM-44	Only authenticated users can remove comment reactions.
BR-FORUM-45	A like reaction can only be removed if a corresponding like record exists.
BR-FORUM-46	Removing a reaction shall delete the corresponding record from the forum_likes table using comment_id.
BR-FORUM-47	Only authenticated users can submit reports.
BR-FORUM-48	Reports must be associated with either a post_id or comment_id.
BR-FORUM-49	New reports shall be created with status = PENDING.
BR-FORUM-50	A user may submit only one active report for the same content item.
BR-JOB-01	Only jobs with status = 'open' or 'in_progress' shall be displayed to freelancers and guests.
BR-JOB-02	Clients shall see all their jobs regardless of status.
BR-JOB-03	Default sorting shall be by created_at in descending order.
BR-JOB-04	The system shall support pagination with default page size of 10.
BR-JOB-05	Search shall match keywords in title and description fields (case-insensitive).
BR-JOB-06	When multiple filters are applied, all conditions shall be combined using AND logic.
BR-JOB-07	Job information displayed includes: title, description, budget, deadline, category, required skills, client name, created date, status.
BR-JOB-08	If the job is inactive (status = 'closed' or 'deleted'), only the job owner can view full details.
BR-JOB-09	Job title is required and must not exceed 200 characters.
BR-JOB-10	Job description is required and must not exceed 5000 characters.
BR-JOB-11	Budget must be a positive number (greater than 0).
BR-JOB-12	Category must be selected from predefined categories.

ID	Rule Definition
BR-JOB-13	New jobs shall have status = 'draft' by default when saved as draft, or 'open' when published.
BR-JOB-14	Only the job owner can edit the job.
BR-JOB-15	Editing is restricted if the job has an active contract.
BR-JOB-16	All validation rules from UC-06.2 apply to edits.
BR-JOB-17	Edit actions shall be logged for auditing purposes.
BR-JOB-18	Only the job owner can delete the job.
BR-JOB-19	Deletion should be soft delete (set deleted_at timestamp, status = 'deleted').
BR-JOB-20	Jobs with active proposals or contracts cannot be deleted.
BR-JOB-21	Delete actions shall be logged for auditing purposes.
BR-JOB-22	Job status workflow: draft → open → in_progress → completed → closed.
BR-JOB-23	Allowed transitions: draft → open (client); open → in_progress (client, when contract created); open → closed (client, if cancelled); in_progress → completed (client, when work done); in_progress → closed (client, if cancelled); completed → closed (system/auto); Any status → deleted (soft delete).
BR-JOB-24	Only the job owner can manually change status.
BR-JOB-25	Status changes shall be logged for auditing purposes.
BR-JOB-26	Notifications shall be sent to freelancers when status changes from draft to open.
BR-JOB-27	Only the freelancer can view their own bookmarks.
BR-JOB-28	Bookmarks shall be sorted by created_at in descending order by default.
BR-JOB-29	Displayed information includes: job title, budget, client name, bookmark date.
BR-JOB-30	The system shall support pagination with default page size of 10.

ID	Rule Definition
BR-JOB-31	Only freelancers can bookmark jobs.
BR-JOB-32	Bookmark is a toggle action. Clicking the button toggles between adding and removing.
BR-JOB-33	If the bookmark does not exist, the system creates it and changes the button to "Bookmarked".
BR-JOB-34	If the bookmark exists, the system deletes it and changes the button to "Bookmark".
BR-JOB-35	Bookmarks can only be created for active jobs (status = 'open' or 'in_progress').
BR-JOB-36	Only the freelancer can remove their own bookmarks.
BR-JOB-37	The system must check if the bookmark exists before deletion.
BR-JOB-38	If the bookmark does not exist, the system shall return a 404 Not Found error.
BR-JOB-39	Removing a bookmark is a hard delete from the database.

4 Functional Requirement

4.1 System Functional Overview

4.1.1 Screen Flow

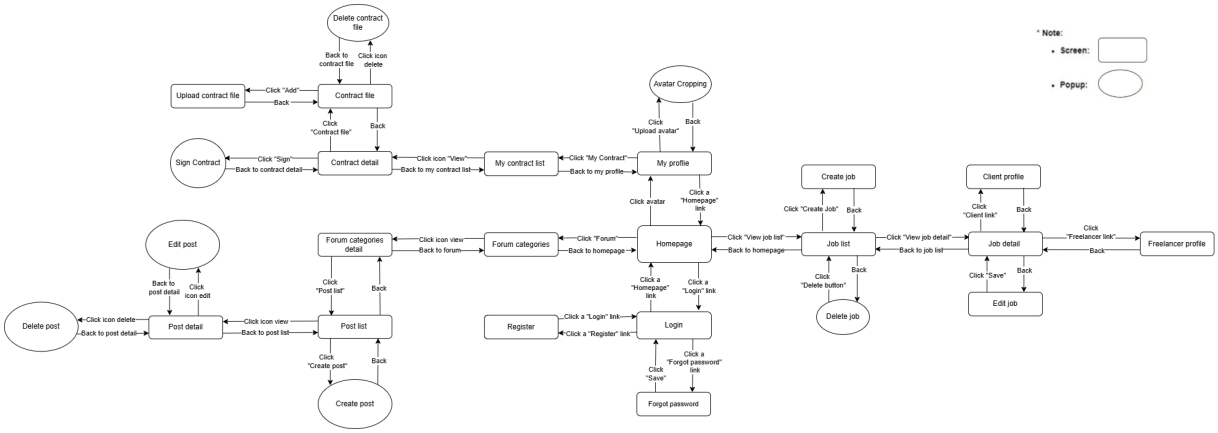


Figure 9: Screen Flow of Client

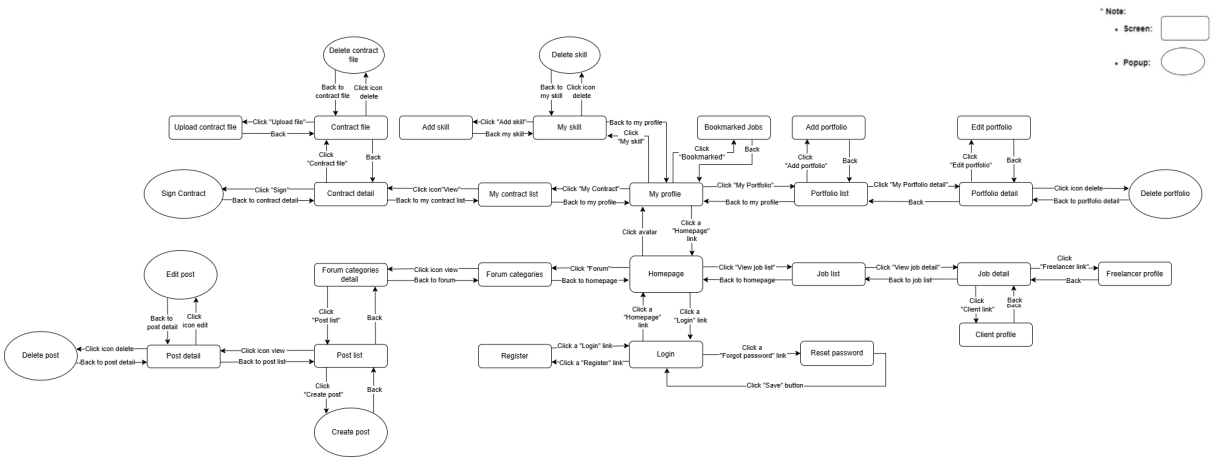


Figure 10: Screen Flow of Freelancer

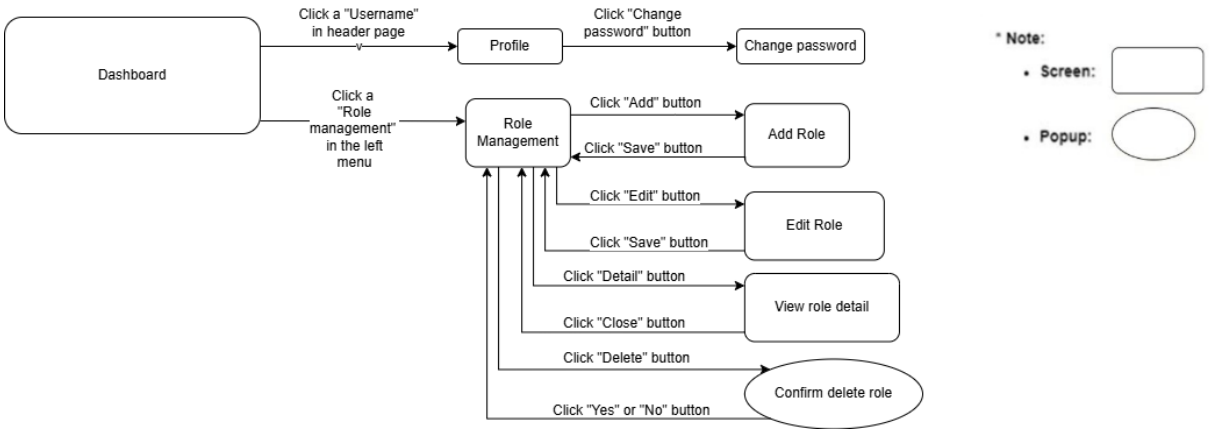


Figure 11: Screen Flow of Admin

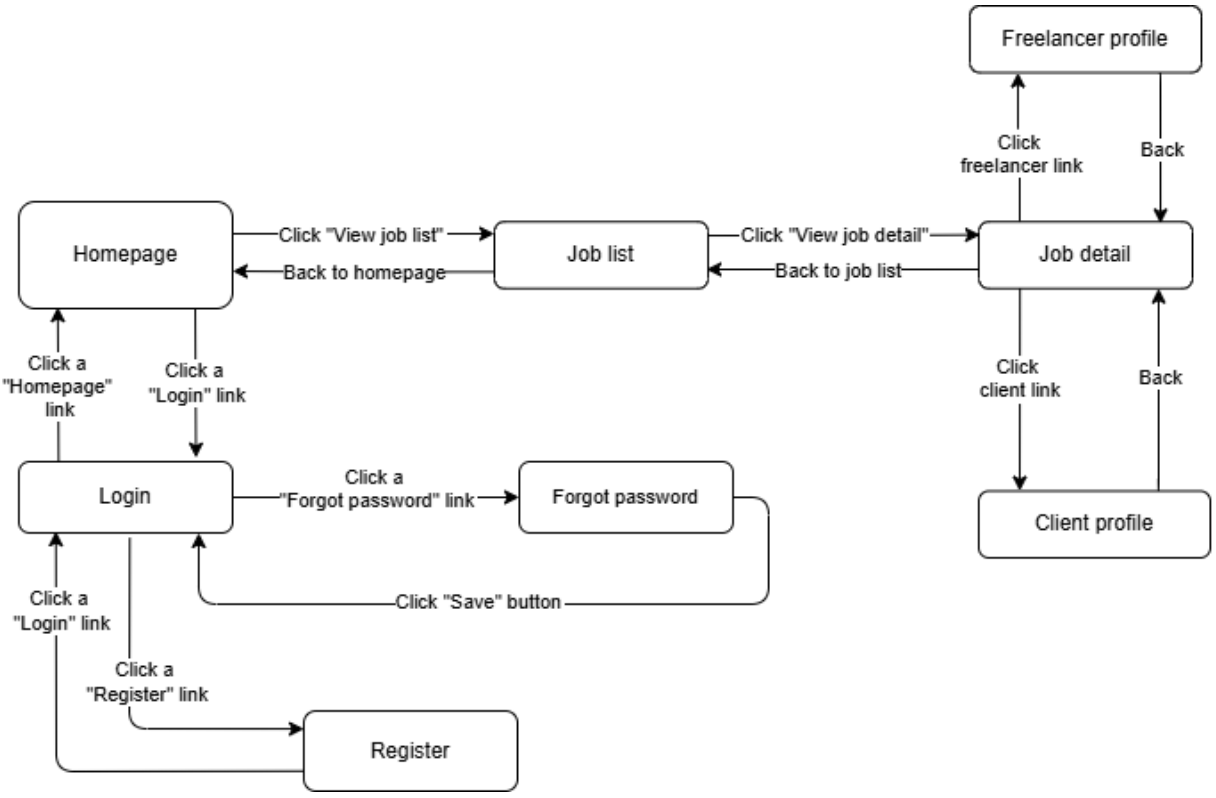


Figure 12: Screen Flow of Guest

4.1.2 Screen Details

No	Actor	Screen Name	Description
1	Guest	Homepage	The main landing page introducing the platform and its features.
2		Login	Allows users to log in using email/password and JWT authentication.
3		Forgot Password	Allows users to request a password reset via email verification link.
4		Register	Allows users to create new accounts using email and password.
5		Job List	Displays a list of available jobs with search and filter capabilities.
6		Job Detail	Displays full details of a specific job posting.
7		Freelancer Profile	Displays a freelancer's public profile including skills and portfolio.
8		Client Profile	Displays a client's public profile including posted jobs and reviews.
9	Freelancer	Google Login	Allows users to login/register using Google OAuth.
10		Login	Allows users to log in using email/password and JWT authentication.
11		Logout	Allows users to logout and terminate authentication session.
12		Register	Allows users to create new accounts using email and password.
13		Forgot Password	Allows users to request password reset via email verification link.
14		My Profile	Main dashboard for freelancer to manage their skills, contracts, and portfolio.

No	Actor	Screen Name	Description
15		My Skill	Displays the list of skills added by the freelancer.
16		Add Skill	Form screen to add a new skill to the profile.
17		Delete Skill	Popup dialog to confirm the deletion of a skill.
18		Bookmarked Jobs	Displays a list of jobs that the freelancer has bookmarked or saved.
19		My Contract List	Displays all active and past contracts associated with the freelancer.
20		Contract Detail	Displays detailed information and terms of a specific contract.
21		Sign Contract	Popup/Action screen allowing the freelancer to digitally sign a contract.
22		Contract File	Displays documents and files attached to a specific contract.
23		Upload Contract File	Form screen to upload new files or documents to a contract.
24		Delete Contract File	Popup dialog to confirm the deletion of a contract file.
25		Portfolio List	Displays a list of projects and works in the freelancer's portfolio.
26		Add Portfolio	Form screen to create and add a new project to the portfolio.
27		Portfolio Detail	Displays full details, descriptions, and media of a specific portfolio project.
28		Edit Portfolio	Form screen to update or modify an existing portfolio project.
29		Delete Portfolio	Popup dialog to confirm the deletion of a portfolio project.

No	Actor	Screen Name	Description
30		Forum Categories	Displays a list of available discussion categories in the forum.
31		Forum Categories Detail	Displays sub-topics and general information of a selected forum category.
32		Post List	Displays a list of discussion threads or posts within a category.
33		Create Post	Form screen allowing the freelancer to create a new forum post.
34		Post Detail	Displays the full content of a forum post along with its replies.
35		Edit Post	Form/Popup to edit the content of an existing forum post.
36		Delete Post	Popup dialog to confirm the deletion of a forum post.
37	Client	Google Login	Allows users to login/register using Google OAuth.
38		Login	Allows users to log in using email/password and JWT authentication.
39		Logout	Allows users to logout and terminate authentication session.
40		Register	Allows users to create new accounts using email and password.
41		Forgot Password	Allows users to request password reset via email verification link.
42		Client Dashboard	Main dashboard for authenticated clients, showing active jobs, contract summaries, and notifications.
43		Create Job Form	Form screen allowing clients to create and publish a new job posting with details (title, description, budget, category, skills).

No	Actor	Screen Name	Description
44		Edit Job	Form screen allowing clients to modify details of an existing job posting.
45		Delete Job	Popup dialog to confirm the soft deletion of a job posting.
46		Change Job Status	Form/Dropdown action to change the status of a job posting (Open, In Progress, Closed, Cancelled).
47		My Contract List	Displays all active and past contracts associated with the client.
48		Contract Detail	Work interface for client to manage contract terms, milestones, and release escrow payments.
49		Create Contract	Form screen allowing the client to initiate and create a new contract with a freelancer.
50		Update Contract Terms	Form/Edit screen to update terms, milestones, and payment settings of a pending contract.
51		Sign Contract	Popup/Action screen allowing the client to digitally sign and fund the contract escrow (Web3).
52		Cancel Contract	Popup/Action screen allowing the client to request contract cancellation and handle escrow refunds.
53		Upload Contract File	Form screen allowing the client to upload files or deliverables to a contract.
54		Delete Contract File	Popup dialog to confirm the deletion of a contract file.
55		View Progress Updates	Screen displaying progress updates and attached proof of work submitted by the freelancer.

No	Actor	Screen Name	Description
56		Hired Freelancers	Screen displaying the list of freelancers currently hired by the client.
57	Admin	Dashboard	Main admin screen displaying system overview and key statistics.
58		Profile	Allows admin to view and update personal account information.
59		Change Password	Allows admin to change their account password.
60		Role Management	Displays a list of all roles along with their assigned permissions.
61		Add Role	Form screen for creating a new role with name, description, and permissions.
62		Edit Role	Form screen for modifying an existing role's name, description, and permissions.
63		View Role Detail	Displays detailed information of a specific role including its permissions.
64		Confirm Delete Role	Popup dialog prompting admin to confirm before permanently deleting a role.

4.1.3 Screen Authorization

No	Screen / Page	Guest	Client	Freelancer	Admin
1	Forum Categories	✓	✓	✓	✓
2	Forum Category Detail	✓	✓	✓	✓
3	Post List	✓	✓	✓	✓
4	Post Detail	✓	✓	✓	✓
5	View Comments	✓	✓	✓	✓
6	View Comment Detail	✓	✓	✓	✓

No	Screen / Page	Guest	Client	Freelancer	Admin
7	Create Post		✓	✓	
8	Edit Post		✓	✓	
9	Delete Post		✓	✓	✓
10	Like Post		✓	✓	
11	Unlike Post		✓	✓	
12	Add Comment		✓	✓	
13	Edit Comment		✓	✓	
14	Delete Comment		✓	✓	✓
15	Like Comment		✓	✓	
16	Unlike Comment		✓	✓	
17	Report Content		✓	✓	
18	View Detailed Freelancer Profile	✓	✓	✓	
19	Update Detailed Freelancer Profile			✓	
20	Get List Freelancer Skill	✓	✓	✓	
21	Add Freelancer Skill			✓	
22	Delete Freelancer Skill			✓	
23	Get Portfolio List			✓	
24	Add New Portfolio			✓	
25	Get Portfolio Detail			✓	
26	Update Portfolio			✓	
27	Delete Portfolio			✓	
28	Create Contract		✓		
29	Update Contract Terms		✓		
30	Complete Contract		✓	✓	
31	Get Contract List		✓	✓	✓
32	Get Contract Detail		✓	✓	✓
33	Sign Contract		✓	✓	

No	Screen / Page	Guest	Client	Freelancer	Admin
34	Cancel Contract		✓	✓	✓
35	Contract Activities		✓	✓	✓
36	Contract Files		✓	✓	✓
37	Upload Contract File		✓	✓	
38	Delete Contract File		✓	✓	✓
39	Progress Update			✓	
40	View Progress Updates		✓	✓	✓
41	Google Login	✓			
42	Login	✓			
43	Logout		✓	✓	✓
44	Register	✓			
45	Forgot Password	✓	✓	✓	✓
46	View Role List				✓
47	View Role Detail				✓
48	Create New Role				✓
49	Update Role				✓
50	Delete Role				✓
51	View List Job	✓	✓	✓	
52	View Job Detail	✓	✓	✓	
53	Post New Job		✓		
54	Edit Job		✓		
55	Delete Job		✓		
56	Change Job Status		✓		
57	View List Bookmarked Job			✓	
58	Bookmark Job			✓	
59	Remove Bookmark			✓	

4.1.4 Non-Screen Functions

#	Feature	System Function	Description
3	**Manage Forum**	[GET] /api/forum/categories [GET] /api/forum/categories/{id} [GET] /api/forum/posts [GET] /api/forum/posts/{id} [GET] /api/forum/comments/{postId} [GET] /api/forum/comments/{id} [POST] /api/forum/posts [PUT] /api/forum/posts/{id} [DELETE] /api/forum/posts/{id} [POST] /api/forum/like-post/{id} [DELETE] /api/forum/unlike-post/{id} [POST] /api/forum/comments [PUT] /api/forum/comments/{id} [DELETE] /api/forum/comments/{id} [POST] /api/forum/like-comments/{id} [DELETE] /api/forum/unlike-comments/{id} [POST] /api/forum/report-contents	The Forum Management feature enables community discussion and knowledge sharing within the platform. Users can browse forum categories, view discussion threads, create and manage posts, add comments, react to content through likes, and report inappropriate content. Guests can access public forum content, while authenticated users can participate in discussions by creating posts, commenting, reacting, and reporting violations. Administrators can moderate forum activities and maintain community standards.

#	Feature	System Function	Description
4	**Manage Contract**	[GET] /api/contracts [GET] /api/contracts/id [GET] /api/contracts/id/files [POST] /api/contracts/id/files [DELETE] /api/contracts/files/id [POST] /api/contracts/id/sign [PUT] /api/contracts/id/terms [POST] /api/contracts/id/milestones/id/complete [POST] /api/contracts/id/cancel	The Contract Management feature enables clients to manage agreements and collaboration with freelancers throughout the project lifecycle. Users can browse contract lists, view contract details, upload and manage contract-related files, and digitally sign contracts to confirm participation. Clients can update contract terms, manage milestone execution, release escrow payments upon successful milestone completion, and request contract cancellation when necessary. The feature ensures secure transactions, transparent collaboration, and controlled contract execution between both parties.

#	Feature	System Function	Description
5	**Manage Freelancer Profile**	[GET] /api/profiles/{id} [PUT] /api/profiles/{id} [GET] /api/profiles/{id}/skills [POST] /api/profiles/{id}/skills [DELETE] /api/profiles/skills/{id} [GET] /api/profiles/{id}/portfolios [POST] /api/profiles/{id}/portfolios [GET] /api/profiles/portfolios/{id} [PUT] /api/profiles/portfolios/{id} [DELETE] /api/profiles/portfolios/{id}	The Freelancer Profile Management feature enables freelancers to manage their professional identity and showcase their expertise to potential clients. Users can view and update their detailed profile information, including their bio, professional title, education, and availability status. Furthermore, freelancers can manage their skill set by adding or removing specific skills along with proficiency levels. The feature also allows freelancers to build a comprehensive portfolio by creating, updating, and deleting portfolio items with attached media files. This robust profile system helps freelancers build credibility, highlight their capabilities, and attract suitable project opportunities.

#	Feature	System Function	Description
6	**Manage Freelancer Skills**	[GET] /api/profiles/{id}/skills [POST] /api/profiles/{id}/skills [DELETE] /api/profiles/skills/{id}	The Freelancer Skills Management feature allows freelancers to define and showcase their professional capabilities. Users can view the complete list of skills associated with a freelancer's profile, add new skills with corresponding proficiency levels (e.g., beginner, intermediate, expert), and permanently remove skills that are no longer relevant. This feature ensures that the freelancer's skill set remains accurate and up-to-date, helping clients easily evaluate their technical and professional qualifications during the talent selection process.

#	Feature	System Function	Description
7	**Manage Freelancer Portfolio**	[GET] /api/profiles/{id}/portfolios [POST] /api/profiles/{id}/portfolios [GET] /api/profiles/portfolios/{id} [PUT] /api/profiles/portfolios/{id} [DELETE] /api/profiles/portfolios/{id}	The Freelancer Portfolio Management feature enables freelancers to visually demonstrate their past work experience and successful projects. Freelancers can create new portfolio entries by providing project details and uploading associated media files or links. The feature also allows users to view detailed information of specific portfolios, update existing portfolio content or media, and soft-delete portfolios that they no longer wish to display publicly. By maintaining a rich and organized portfolio gallery, freelancers can build stronger credibility and significantly increase their chances of being hired by prospective clients.

#	Feature	System Function	Description
8	**Manage Authentication**	[POST] /api/auth/google [POST] /api/auth/login [POST] /api/auth/register [POST] /api/auth/forgot-password [POST] /api/auth/logout	The Authentication Management feature handles user identity verification and access control. It supports multiple authentication methods including traditional email/password login with JWT token generation, Google OAuth for social sign-in, and new account registration. The feature also provides secure password reset via email verification links, and allows authenticated users to terminate their session by logging out. Together, these endpoints ensure a seamless and secure authentication experience for all platform users.
9	**Manage Roles**	[GET] /api/roles [GET] /api/roles/{id} [POST] /api/roles [PUT] /api/roles/{id} [DELETE] /api/roles/{id}	The Role Management feature allows administrators to define and control user permissions within the system. Admin users can retrieve a list of all roles along with their assigned permissions, view detailed information of a specific role, create new roles with custom names and permission sets, update existing roles to modify their access rights, and delete roles that are no longer needed. This feature ensures fine-grained access control, maintaining system security and proper segregation of duties across different user types.

#	Feature	System Function	Description
10	**Manage Job**	[GET] /api/jobs [GET] /api/jobs/{id} [POST] /api/jobs [PUT] /api/jobs/{id} [DELETE] /api/jobs/{id} [PATCH] /api/jobs/{id}/status [GET] /api/jobs/bookmarks [POST] /api/jobs/{id}/bookmark [DELETE] /api/jobs/{id}/bookmark	The Job Management feature enables clients to create and manage job postings, and allows freelancers to browse, search, and bookmark interesting opportunities. Clients can post new jobs with detailed descriptions, budgets, required skills, and timelines; edit or delete their own job postings; and update job status (Open, In Progress, Closed, Cancelled). Freelancers can view paginated job lists with filters and sorting, view full job details, and bookmark jobs for later reference. This feature serves as the core marketplace functionality connecting clients with freelancers.

4.1.5 Entity Relationship Diagram

4.1.1 Authentication & Role-Based Access

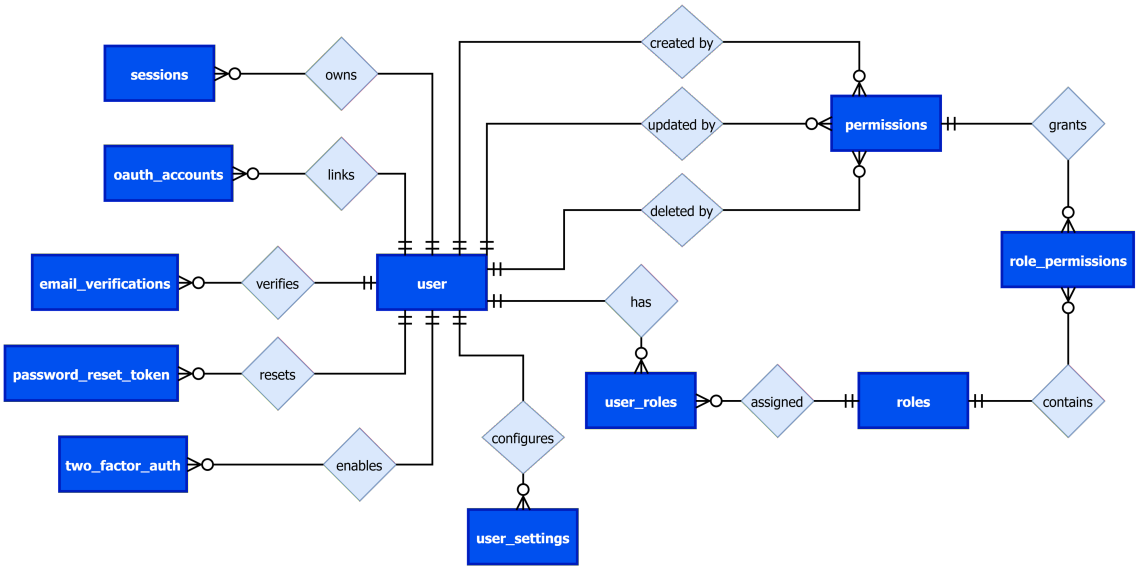


Figure 13: Authentication & Role-Based Access ERD

4.1.2 Manage Profile

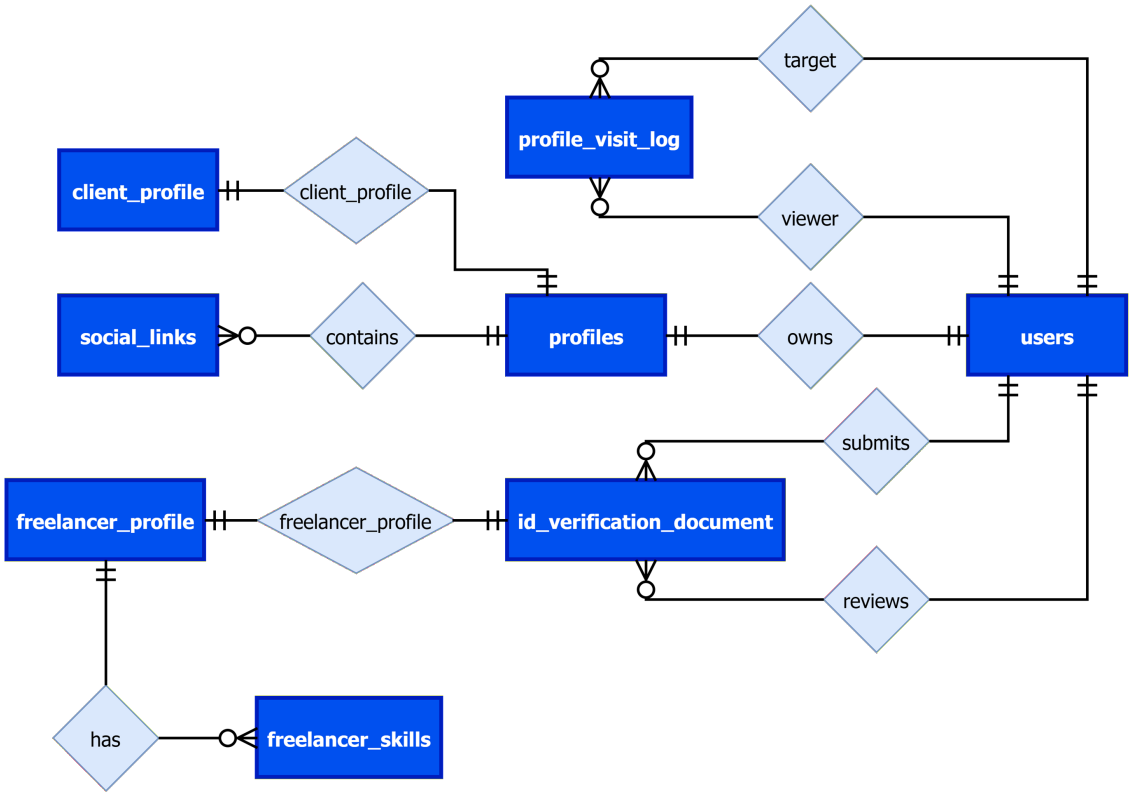


Figure 14: Manage Profile ERD

4.1.3 Portfolio & Freelancer Network

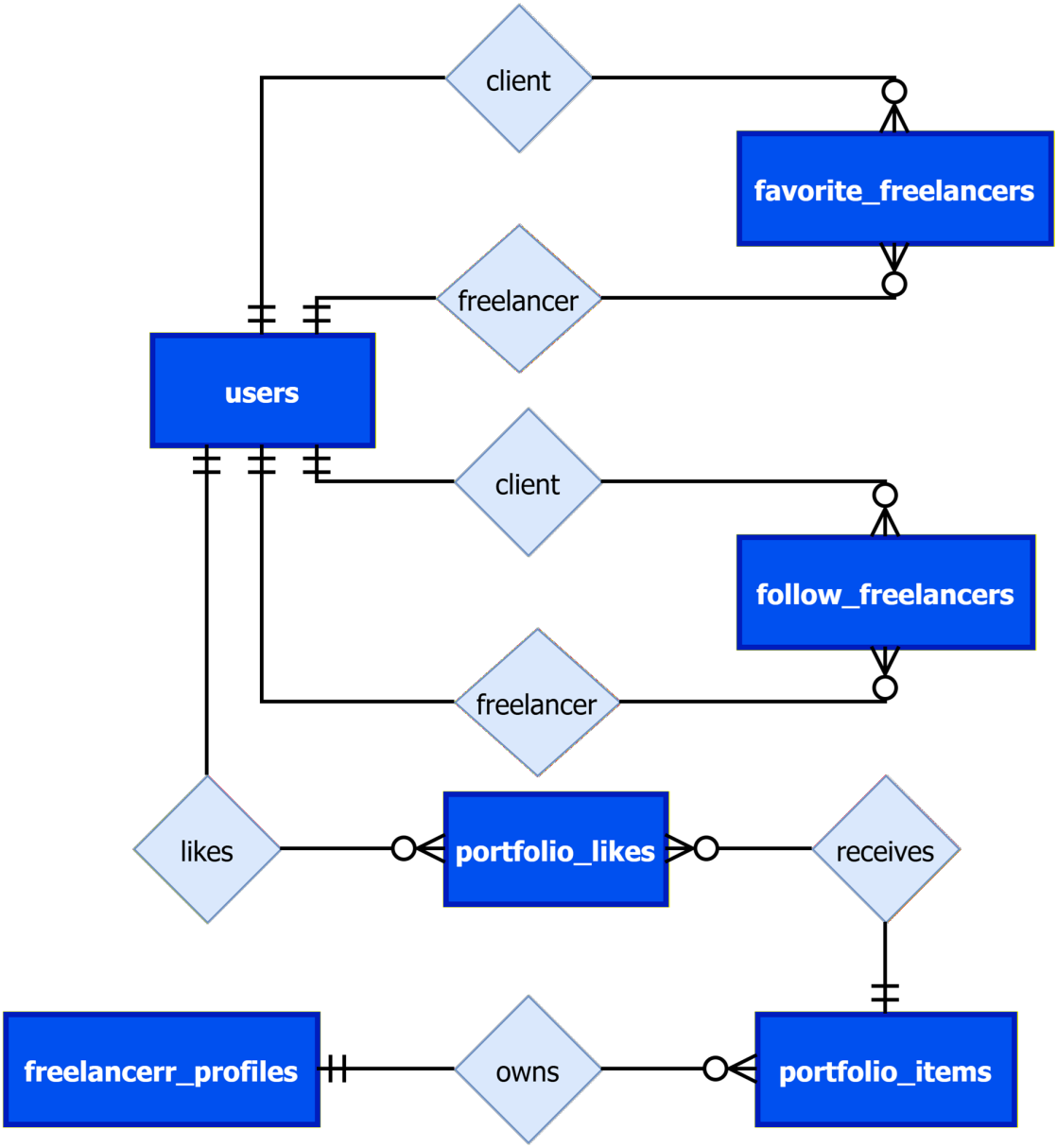


Figure 15: Portfolio & Freelancer Network ERD

4.1.4 Job Posting & Discovery

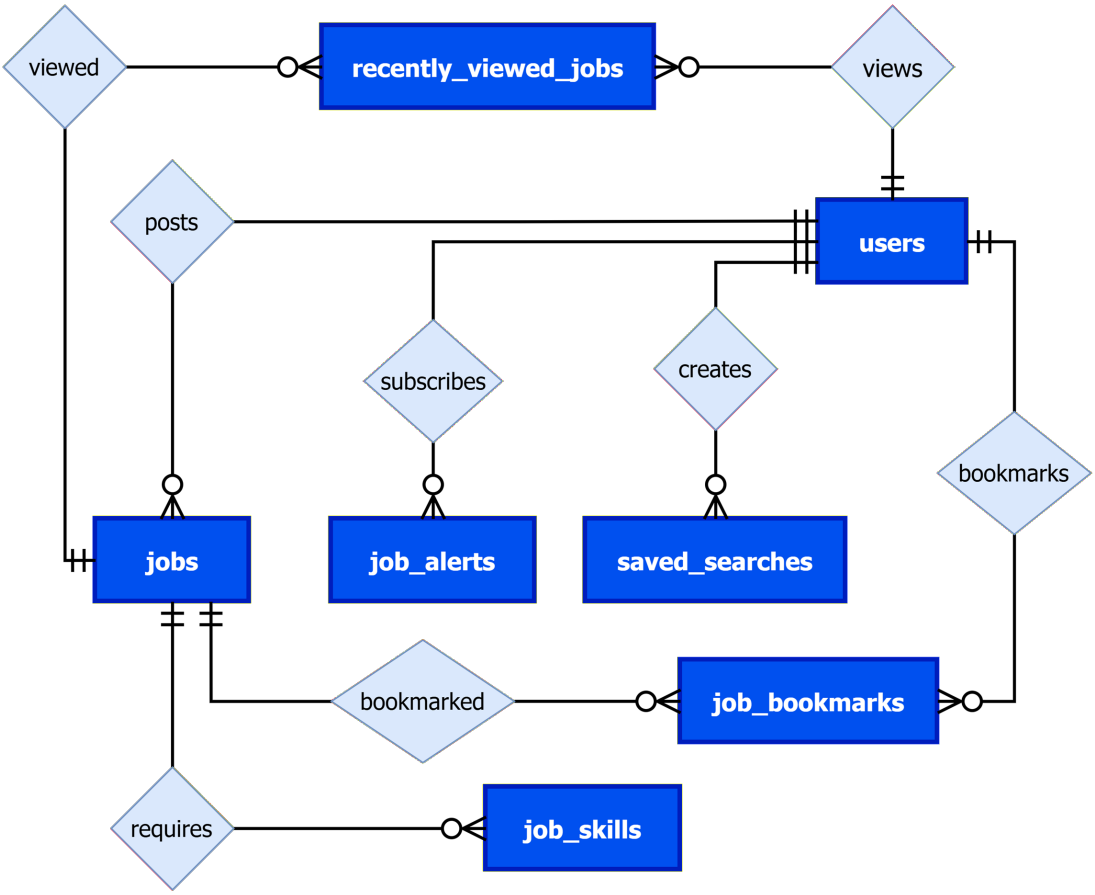


Figure 16: Job Posting & Discovery ERD

4.1.5 Manage Proposal

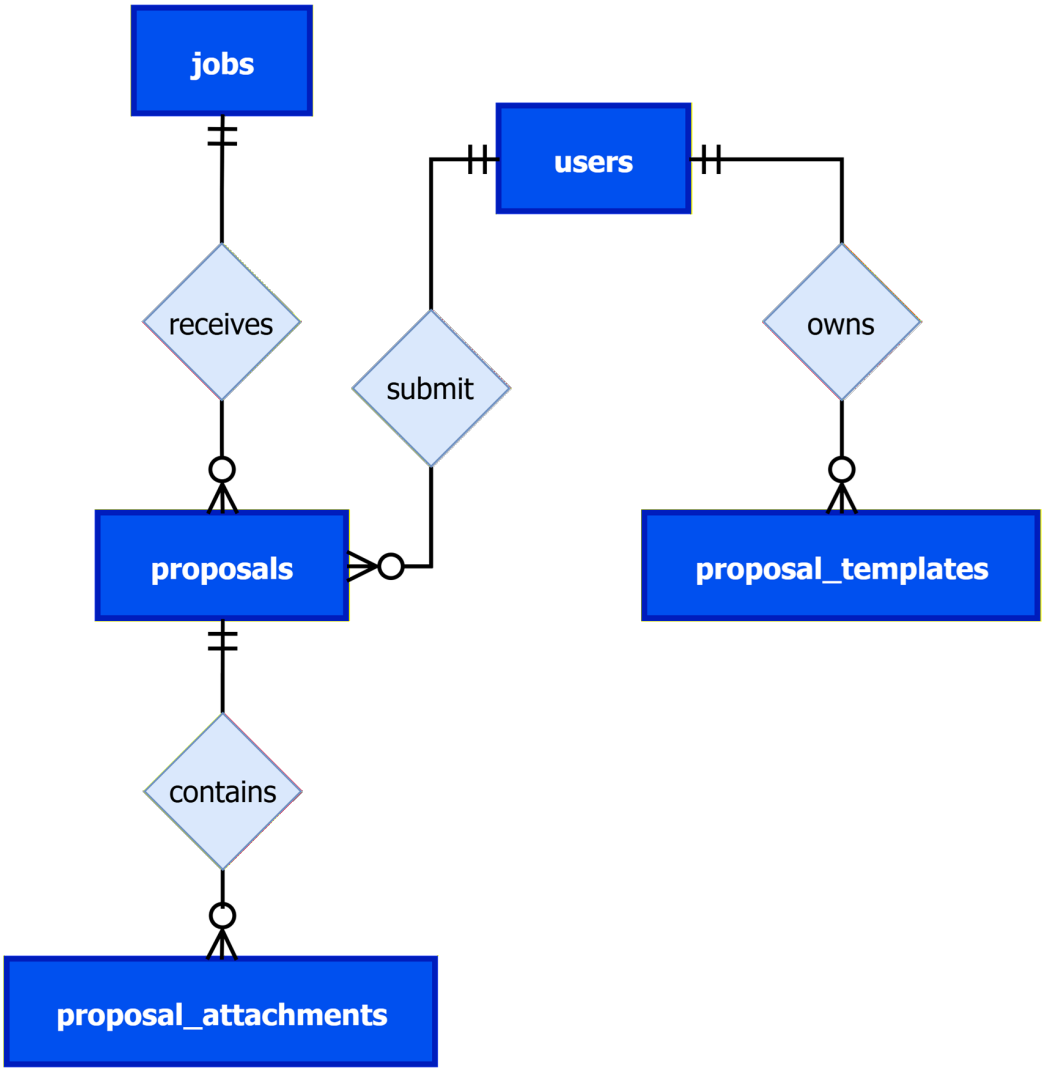


Figure 17: Manage Proposal ERD

4.1.6 Manage Contract

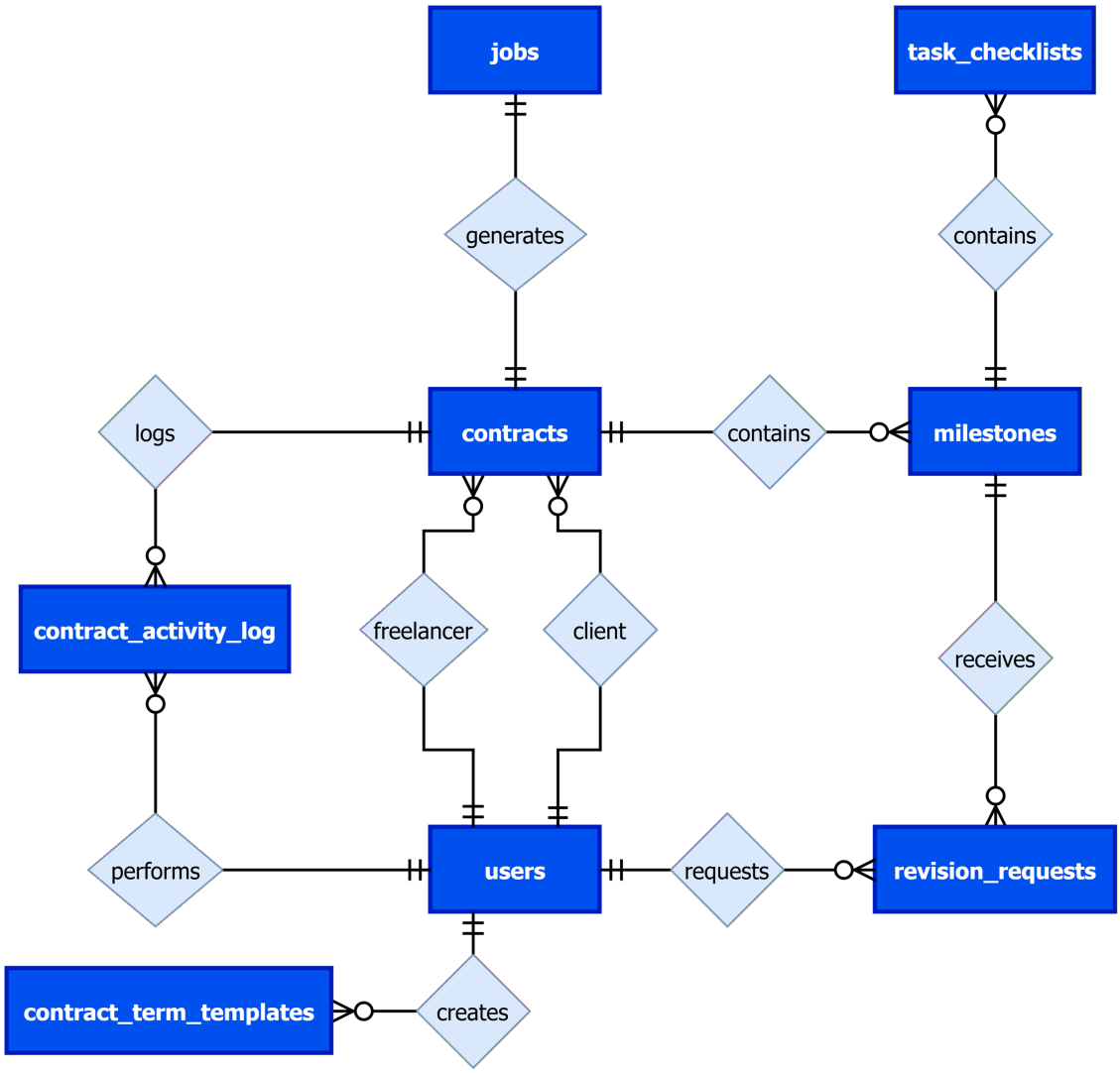


Figure 18: Manage Contract ERD

4.1.7 Escrow, Payments & Dispute Resolution

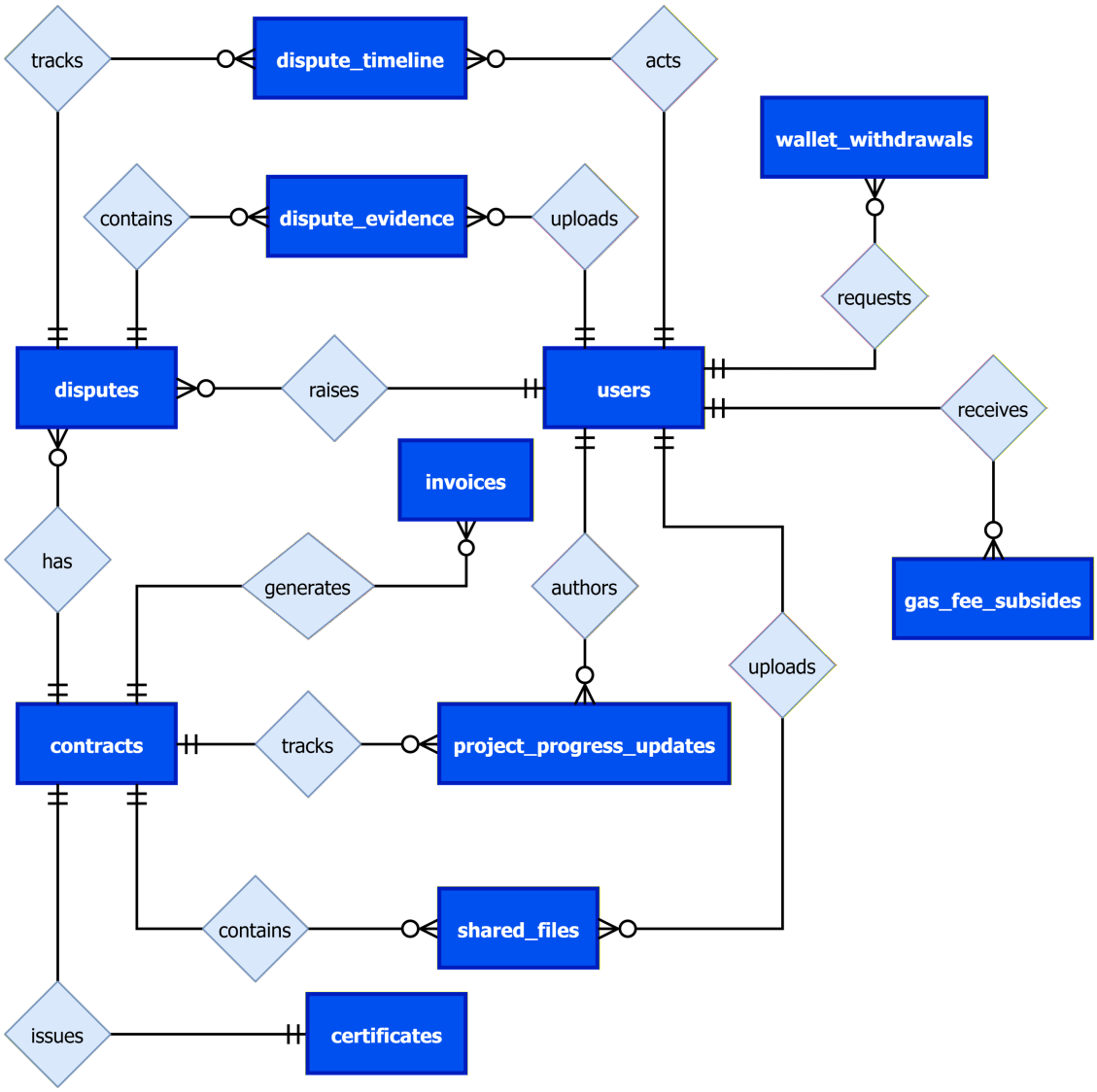


Figure 19: Escrow, Payments & Dispute Resolution ERD

4.1.8 Reviews & Reputation System

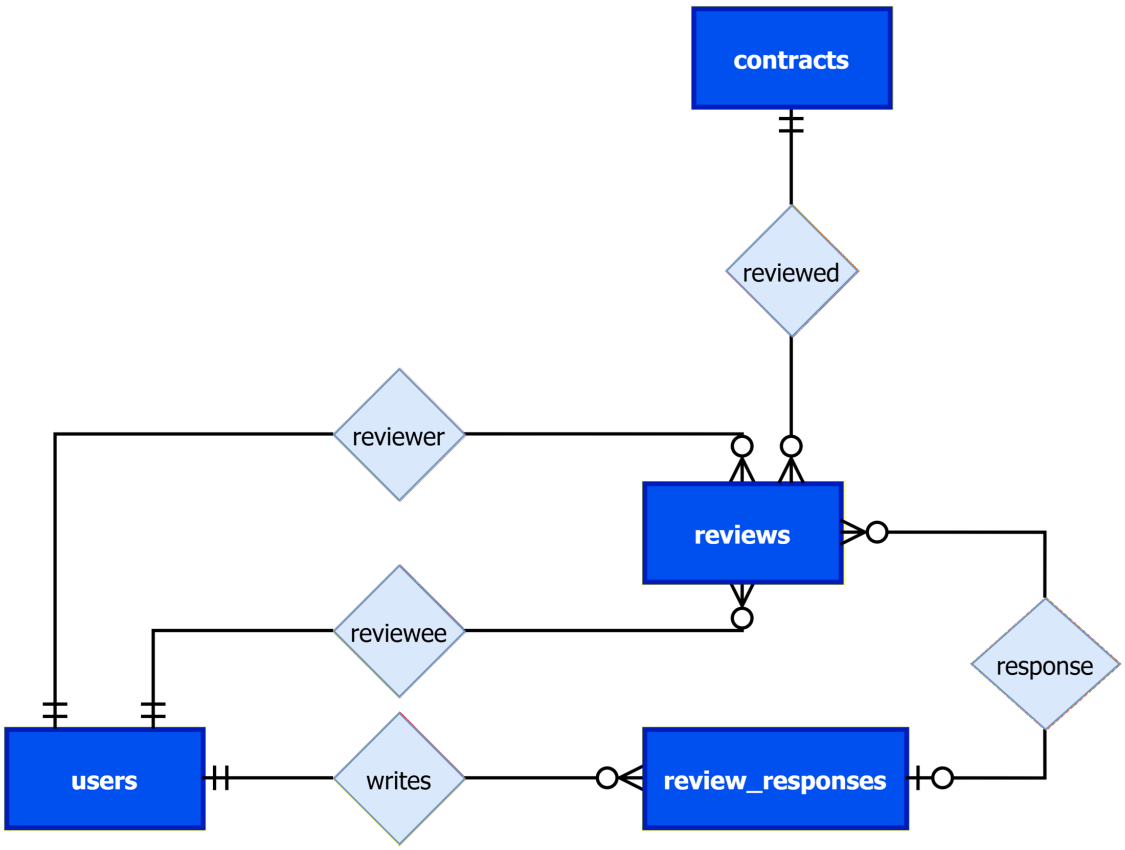


Figure 20: Reviews & Reputation SystemERD

4.1.9 Forum & Community

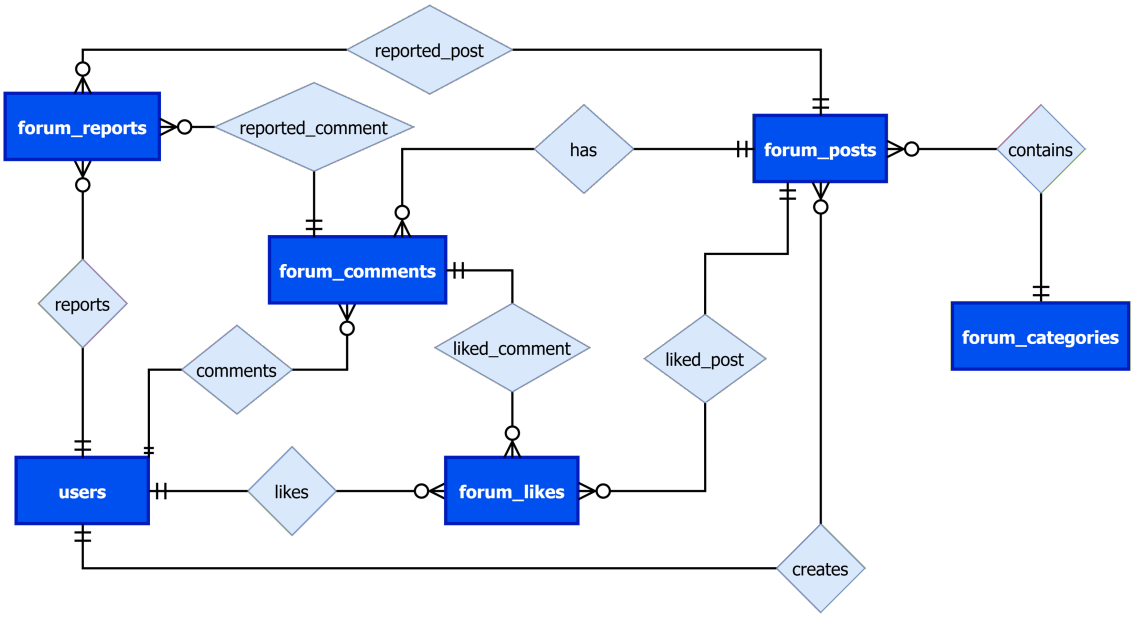


Figure 21: Forum & Community ERD

4.1.10 Messaging & Notifications

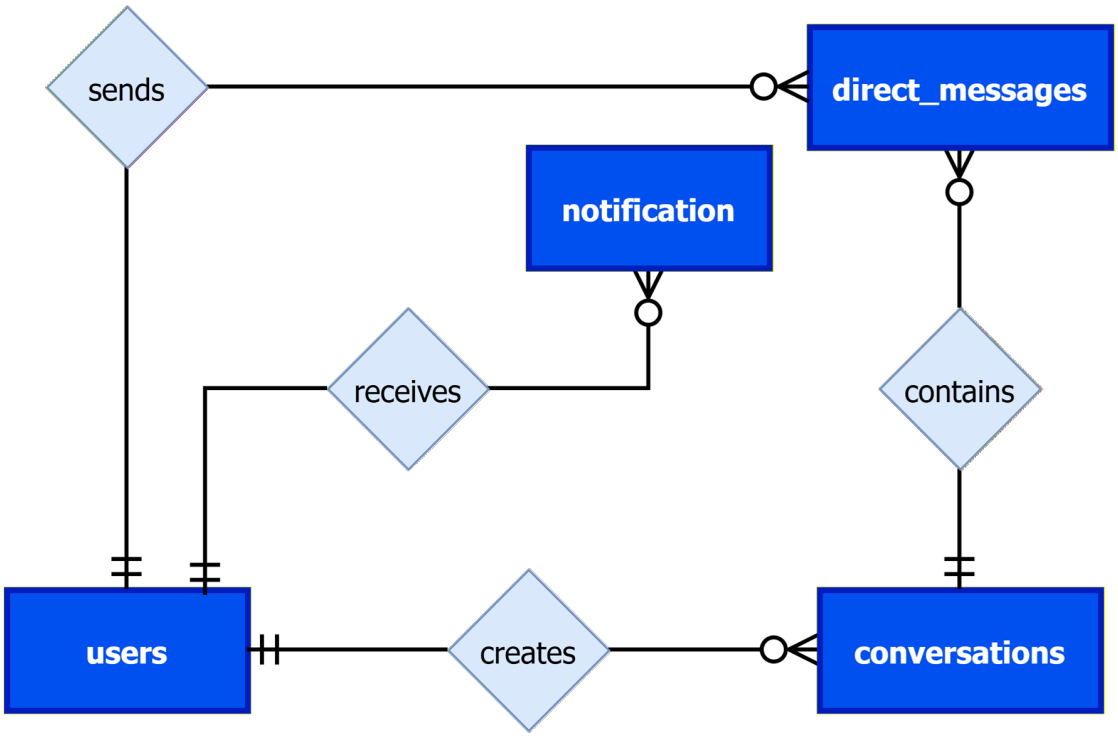


Figure 22: Messaging & Notifications ERD

4.1.11 Administration & Moderation

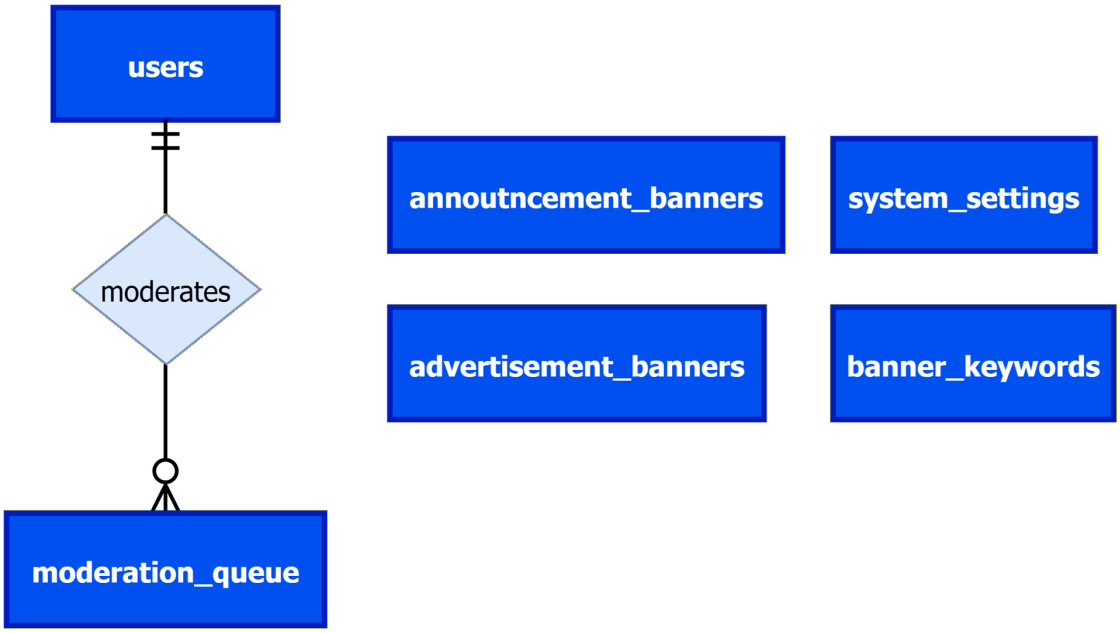


Figure 23: Administration & Moderation ERD

5 Non-Functional Requirement

This section describes the non-functional requirements of the Frevia platform, covering performance, security, availability, usability, and Web3-specific requirements.

5.1 Performance & Scalability

- **Response Time:** The system's API read operations must respond within 200ms under normal load (up to 500 concurrent requests). Write/update operations, excluding blockchain transactions, must complete within 500ms.
- **Page Load Time:** The web application must achieve a Largest Contentful Paint (LCP) of less than 2.5 seconds, adhering to Google's Core Web Vitals guidelines.
- **Concurrency:** The system backend must support at least 1,000 concurrent active users and handle a database connection pool efficiently.
- **Search and Filtering Efficiency:** The job search page must employ debouncing (300ms delay) and pagination (default 10 items per page) to optimize query performance and reduce server load.

5.2 Security & Privacy

- **Authentication and Authorization:** All network communications must be encrypted using HTTPS/TLS 1.3. User passwords must be hashed using bcrypt with a work factor of 12.
- **Session Management:** JWT Access Tokens must have a short lifespan (e.g., 15 minutes) and be stored in memory. Refresh Tokens must have a longer lifespan (e.g., 7 days), be stored in HTTP-only, Secure, and SameSite=Strict cookies, and be invalidated in the database upon user logout.
- **Role-Based Access Control (RBAC):** The system must strictly enforce role boundaries (Guest, Freelancer, Client, Admin). Administrative features, such as changing smart contract addresses or viewing audit logs, must be blocked by a server-side gateway for non-admin accounts.
- **Web3 Wallet and Transaction Safety:** Input wallet addresses must be verified on the client and server side to conform to the standard EVM format (starts with `0x` followed by 40 hexadecimal characters).

5.3 Availability & Reliability

- **System Uptime:** The system core services must maintain a 99.9% monthly uptime, excluding scheduled maintenance.
- **Blockchain Fault Tolerance:** In case of RPC node connection failures, the platform must gracefully fall back to backup RPC endpoints within 3 seconds to avoid breaking the wallet balance display or transaction signing flows.
- **Escrow Reliability:** Funds locked in the Escrow Smart Contracts must be secure. Contracts must be verified on Etherscan/BscScan and use standard libraries (e.g., OpenZeppelin) to prevent vulnerabilities.

5.4 Usability & User Experience

- **Responsive Design:** The user interface must be fully responsive, supporting mobile devices, tablets, and desktop viewports (minimum width 320px).
- **Toast Notification System:** Interactive toast notifications (success, warning, error) must have the highest z-index to overlay all content, auto-dismiss after 3000ms, and must not freeze or block other user interactions.

- **Client-side Image Processing:** The avatar cropping tool must handle images locally before upload. The system must restrict upload sizes to under 5MB and validate formats (.jpg, .jpeg, .png, .webp) before initiating API requests.